

ΠΑΝΕΠΙΣΤΗΜΙΟ ΜΑΚΕΔΟΝΙΑΣ
ΟΙΚΟΝΟΜΙΚΩΝ ΚΑΙ ΚΟΙΝΩΝΙΚΩΝ ΕΠΙΣΤΗΜΩΝ
ΤΜΗΜΑ ΕΦΑΡΜΟΣΜΕΝΗΣ ΠΛΗΡΟΦΟΡΙΚΗΣ

“Εκμάθηση της UML-Ενοποιημένη
Γλώσσα Μοντελοποίησης μέσω
τηλεμαθημάτων στο Internet”

Εκπονήθηκε από:

Ξαπόλυτου Παναγιώτα Α.Μ. 95/00

Επιβλέπων Καθηγητής:
Στεφανίδης Γεώργιος

Τριμελής Επιτροπή:

Στεφανίδης Γεώργιος, Επίκουρος Καθηγητής τμ.

Εφαρμοσμένης Πληροφορικής

Χατζηγεωργίου Αλέξανδρος, Λέκτορας τμ. Εφαρμοσμένης
Πληροφορικής

Φουληράς Παναγιώτης, Λέκτορας τμ. Εφαρμοσμένης
Πληροφορικής

Θεσσαλονίκη, Νοέμβριος 2003



ΠΑΝΕΠΙΣΤΗΜΙΟ ΜΑΚΕΔΟΝΙΑΣ
ΟΙΚΟΝΟΜΙΚΩΝ ΚΑΙ ΚΟΙΝΩΝΙΚΩΝ ΕΠΙΣΤΗΜΩΝ
ΤΜΗΜΑ ΕΦΑΡΜΟΣΜΕΝΗΣ ΠΛΗΡΟΦΟΡΙΚΗΣ

“Εκμάθηση της UML-Ενοποιημένη Γλώσσα Μοντελοποίησης μέσω τηλεμαθημάτων στο Internet”

Εκπονήθηκε από: *Ξαπόλυτου Παναγιώτα Α.Μ. 95/00*

Επιβλέπων Καθηγητής: *Στεφανίδης Γεώργιος*

Τριμελής Επιτροπή:

-  Στεφανίδης Γεώργιος, Επίκουρος Καθηγητής τμ. Εφαρμοσμένης Πληροφορικής
-  Χατζηγεωργίου Αλέξανδρος, Λέκτορας τμ. Εφαρμοσμένης Πληροφορικής
-  Φουληράς Παναγιώτης, Λέκτορας τμ. Εφαρμοσμένης Πληροφορικής

Θεσσαλονίκη, Νοέμβριος 2003

Πίνακας Περιεχομένων

1 Εισαγωγή	1
1.1 Δομή	2
1.2 Σχεδίαση - Πλοήγηση	4
1.3 Τεκμηρίωση Κώδικα	11
2 Κεντρική Σελίδα	13
3 Εισαγωγή στην Αντικειμενοστραφή Σχεδίαση & στη UML	14
3.1 Τι είναι η UML	15
3.2 Διαγράμματα	17
3.2.1 Διάγραμμα Περιπτώσεων Χρήσης	18
3.2.2 Διάγραμμα Αντικειμένων	19
3.2.3 Διάγραμμα Κλάσεων	20
3.2.4 Διάγραμμα Ακολουθίας	21
3.2.5 Διάγραμμα Συνεργασίας	23
3.2.6 Διάγραμμα Καταστάσεων	24
3.2.7 Διάγραμμα Δραστηριοτήτων	26
3.2.8 Διάγραμμα Συστατικών	27
3.2.9 Διάγραμμα Ανάπτυξης	29
3.3 Περιπτώσεις Χρήσης	30
3.4 Αντικείμενα	34
3.5 Κλάσεις	38
3.6 Σχέσεις Ανάμεσα σε Κλάσεις	39
3.6.1 Συσχέτιση	40
3.6.2 Συσσωμάτωση	43
3.7 Σύγκριση Διαγραμμάτων Κλάσης & Αντικειμένου	45
3.8 Γενίκευση και Ειδίκευση	47
3.9 Γενίκευση	48
3.10 Ειδίκευση	51
3.11 Κληρονομικότητα	52
3.12 Πολυμορφισμός	55
4 Βήμα προς βήμα για την μοντελοποίηση του SRS	58
4.1 Σύστημα Εγγραφής Φοιτητών(SRS) - Απαιτούμενα Χαρακτηριστικά	59
4.2 Ενότητα I - Αντικειμενοστραφής Μοντελοποίηση	61
4.2.1 Τελικός στόχος κα φιλοσοφία	62
4.2.2 Η δική μας μέθοδος Μοντελοποίησης με δικά μας λόγια	66
4.2.3 Σκέψεις σχετικά με τα Εργαλεία Προγραμματισμού της Αντικειμενοστραφούς Μοντελοποίησης	68
4.3 Ενότητα II - Τακτοποίηση των Απαιτήσεων	

χρησιμοποιώντας Ανάλυση Περιπτώσεων Χρήσης	71
4.3.1 Εισαγωγή	72
4.3.2 Τι είναι τα "Use Cases"	73
4.3.3 Χρήστες	74
4.3.4 Καθορίζοντας τα "Use Cases"	79
4.3.5 Μοντελοποιώντας ή Όχι?	81
4.4 Ενότητα III - Μοντελοποίηση των Στατικών	
Χαρακτηριστικών Συμπεριφοράς του Συστήματος	82
4.4.1 Εισαγωγή	83
4.4.2 Προσδιορίζοντας τις σωστές Κλάσεις	85
4.4.3 Ανάλυση Ουσιαστικών Προτάσεων	86
4.4.4 Έλεγχος των "Use Cases"	91
4.4.5 Λεξικό Δεδομένων για το SRS	93
4.4.6 Καθορισμός των Συσχετίσεων μεταξύ Κλάσεων	94
4.4.7 Κλάσεις, Ιδιότητες και Μέθοδοι	97
4.4.8 Σχέσεις μεταξύ Κλάσεων	101
4.4.9 Πολλαπλότητα	105
4.4.10 Διάγραμμα Αντικειμένων	107
4.4.11 "Ανακατεύοντας και Ταιριάζοντας" τους Συμβολισμούς	
Σχέσεων	109
4.4.12 Κλάσεις Συσχέτισης	112
4.4.13 Η "Ολοκληρωμένη" Κλάση Συσχέτισης για το SRS	116
4.5 Ενότητα IV - Μοντελοποίηση των Δυναμικών	
Χαρακτηριστικών Συμπεριφοράς του Συστήματος	119
4.5.1 Εισαγωγή	120
4.5.2 Γεγονότα	122
4.5.3 Σενάρια	126
4.5.4 Ελάττωμα στο Μοντέλο	130
4.5.5 Επαναχρησιμοποιώντας Μοντέλα	133
5 Ασκήσεις	136
5.1 Ασκήσεις - Θεωρητικές	137
5.1.1 Απαντήσεις Θεωρητικών Ασκήσεων	138
5.2 Ασκήσεις - Εφαρμογές	142
6 Βιβλιογραφία	143

Εισαγωγή

Η παρούσα πτυχιακή εργασία με τίτλο "Εκμάθηση της UML-Ενοποιημένη Γλώσσα Μοντελοποίησης μέσω τηλεμαθημάτων στο Internet" εκπονήθηκε από την φοιτήτρια:

Ξαπόλυτου Παναγιώτα, Α.Μ. 95/00

υπό την επίβλεψη και καθοδήγηση του Κύριου Στεφανίδη Γεώργιου, Επίκουρου Καθηγητή του τμήματος Εφαρμοσμένης Πληροφορικής του Πανεπιστημίου Μακεδονίας.

Ο σκοπός ανάπτυξης της εργασίας αυτής, είναι να διδάξει τη UML-Ενοποιημένη Γλώσσα Μοντελοποίησης, κάνοντας σταδιακά την εισαγωγή σε βασικές έννοιες της UML και στη συνέχεια παρουσιάζοντας πιο εξειδικευμένα χαρακτηριστικά και δυνατότητες που εφαρμόζονται στη κατασκευή ενός συστήματος SRS - Σύστημα Εγγραφής Φοιτητών.

Ο χρήστης έχει τη δυνατότητα να δει μια ολοκληρωμένη παρουσίαση της UML, που αποτελείται από τρεις βασικές κατηγορίες. Η πρώτη κατηγορία «Εισαγωγή στην Αντικειμενοστραφή Σχεδίαση και στη UML» αποτελείται από βασικές έννοιες και διαγράμματα που βοηθούν τον χρήστη να έρθει σε μία πρώτη επαφή με την UML. Στη συνέχεια η δεύτερη κατηγορία «Βήμα προς βήμα για την μοντελοποίηση του SRS» αποτελείται από τέσσερις ενότητες οι οποίες βασίζονται στη κατασκευή ενός συστήματος SRS-Σύστημα Εγγραφής Φοιτητών με την χρήση και εφαρμογή της UML. Τέλος η τρίτη κατηγορία «Ασκήσεις» περιλαμβάνει θεωρητικές ασκήσεις και εφαρμογές πάνω στα θέματα που είδη παρουσιάστηκαν για να βοηθήσει το χρήστη να ταξινομήσει τα όσα έχουν λεχθεί.

Για το σχεδιασμό του ιστοχώρου αυτού χρησιμοποιήθηκε ως βασικό εργαλείο το Microsoft Frontpage 5.0 σε παράλληλη χρήση και του Dreamweaver 3.0 της Macromedia. Η επεξεργασία εικόνων έγινε με τα προγράμματα Paint των Microsoft Windows και Adobe Photoshop 7.0. Επίσης το εργαλείο που βοήθησε σημαντικά για την κατασκευή των σχεδιαγραμμάτων της UML είναι το Microsoft Vision Professional 2002.

Δομή της Πτυχιακής Εργασίας & Συνοπτική παράθεση της δομής του ιστοχώρου

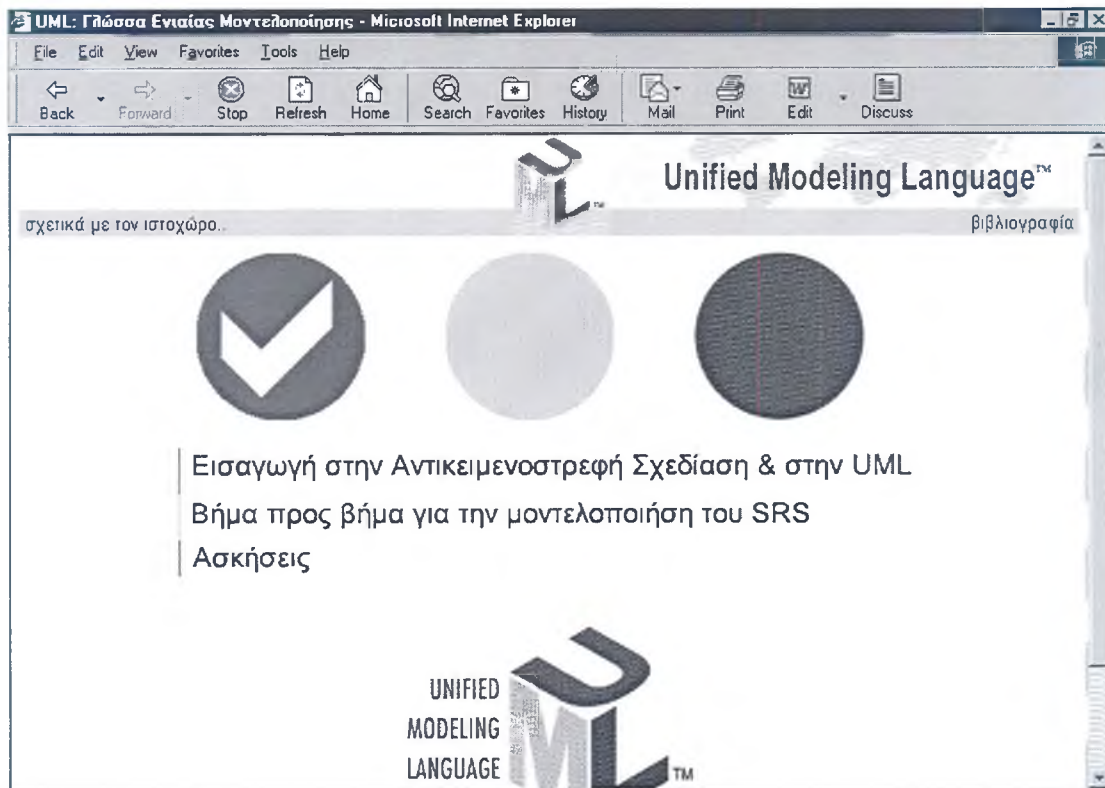
Οι επιλογές που σας δίνονται μέσα από τον ιστοχώρο μας είναι οι παρακάτω:

- ✦ Εισαγωγή στην Αντικειμενοστραφή Σχεδίαση & στην UML - 1.htm
 - Τι είναι η UML - 1_1.htm
 - Διαγράμματα - 1_2.htm
 - ◆ Διάγραμμα Περίπτωσης Χρήσης - usecase_diagram.htm
 - ◆ Διάγραμμα Αντικειμένων - object_diagram.htm
 - ◆ Διάγραμμα Κλάσεων - class_diagram.htm
 - ◆ Διάγραμμα Ακολουθίας - sequence_diagram.htm
 - ◆ Διάγραμμα Συνεργασίας - collaboration_diagram.htm
 - ◆ Διάγραμμα Καταστάσεων - state_diagram.htm
 - ◆ Διάγραμμα Δραστηριοτήτων - activity_diagram.htm
 - ◆ Διάγραμμα Συστατικών - component_diagram.htm
 - ◆ Διάγραμμα Ανάπτυξης - deployment_diagram.htm
 - Περιπτώσεις Χρήσης - 1_3.htm
 - Αντικείμενα - 1_4.htm
 - Κλάσεις - 1_5.htm
 - Σχέσεις Ανάμεσα σε Κλάσεις - 1_6.htm
 - ◆ Συσχέτιση - 1_6_1.htm
 - ◆ Συσσωμάτωση - 1_6_2.htm
 - Σύγκριση Διαγραμμάτων Κλάσης και Αντικειμένων - 1_7.htm
 - Γενίκευση και Ειδίκευση - 1_8.htm
 - ◆ Γενίκευση - 1_8_1.htm
 - ◆ Ειδίκευση - 1_8_2.htm
 - Κληρονομικότητα - 1_9.htm
 - Πολυμορφισμός - 1_10.htm
- ✦ Βήμα προς βήμα για την μοντελοποίηση του SRS - 2.htm
 - SRS-Σύστημα Εγγραφής Φοιτητών - 2_1.htm
 - Ενότητα I- Αντικειμενοστραφής Μοντελοποίηση - 2_2.htm
 - ◆ Τελικός στόχος και φιλοσοφία της Αντικειμενοστραφής Μοντελοποίησης - 2_2_1.htm
 - ◆ Η δική μας μέθοδος Μοντελοποίησης με λίγα λόγια - 2_2_2.htm
 - ◆ Σκέψεις σχετικά με τα Εργαλεία προγραμματισμού της Αντικειμενοστραφής Μοντελοποίησης - 2_2_3.htm
 - Ενότητα II- Τακτοποίηση των Απαιτήσεων χρησιμοποιώντας Ανάλυση Περιπτώσεων Χρήσης. - 2_3.htm
 - ◆ Εισαγωγή - 2_3_1.htm
 - ◆ Τι είναι τα "Use Cases" - 2_3_2.htm

- ◆ Χρήστες (Actors) - 2_3_3.htm
- ◆ Καθορίζοντας τα "Use Cases" - 2_3_4.htm
- ◆ Μοντελοποιώντας ή Όχι? - 2_3_5.htm
- Ενότητα III- Μοντελοποίηση των Στατικών Χαρακτηριστικών Συμπεριφοράς του Συστήματος - 2_4.htm
 - ◆ Εισαγωγή - 2_4_1.htm
 - ◆ Προσδιορίζοντας τις σωστές Κλάσεις - 2_4_2.htm
 - ◆ Ανάλυση Ουσιαστικών Προτάσεων - 2_4_3.htm
 - ◆ Έλεγχος των "Use Cases" - 2_4_4.htm
 - ◆ Λεξικό Δεδομένων για το SRS - 2_4_5.htm
 - ◆ Καθορισμός των Συσχετίσεων μεταξύ Κλάσεων - 2_4_6.htm
 - ◆ Κλάσεις, Ιδιότητες και Μέθοδοι - 2_4_7.htm
 - ◆ Σχέσεις μεταξύ κλάσεων - 2_4_8.htm
 - ◆ Πολλαπλότητα - 2_4_9.htm
 - ◆ Διάγραμμα Αντικειμένων - 2_4_10.htm
 - ◆ " Ανακατεύοντας και Ταιριάζοντας " τους Συμβολισμούς Σχέσεων - 2_4_11.htm
 - ◆ Κλάσεις Συσχέτισης - 2_4_12.htm
 - ◆ Η " Ολοκληρωμένη " Κλάση Συσχέτισης για το SRS - 2_4_13.htm
- Ενότητα IV- Μοντελοποίηση των Δυναμικών Χαρακτηριστικών Συμπεριφοράς του Συστήματος - 2_5.htm
 - ◆ Εισαγωγή - 2_5_1.htm
 - ◆ Γεγονότα (Events) - 2_5_2.htm
 - ◆ Σενάρια (Scenarios) - 2_5_3.htm
 - ◆ Ελάττωμα στο Μοντέλο - 2_5_4.htm
 - ◆ Επαναχρησιμοποιώντας Μοντέλα - 2_5_5.htm
- ✚ Ασκήσεις - 3.htm
 - Θεωρητικές Ασκήσεις - 3_1.htm
 - ◆ Απαντήσεις Ασκήσεων - 3_1_1.htm
 - Εφαρμογές - 3_2.htm
- ✚ Σχετικά με τον ιστοχώρο
- ✚ Βιβλιογραφία

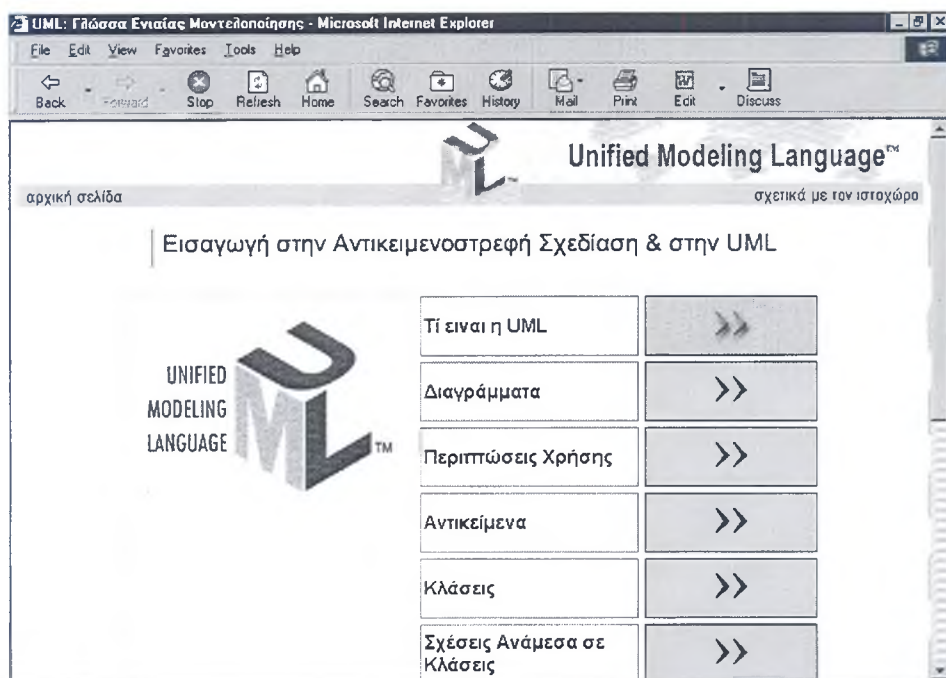
Σχεδίαση - Πλοήγηση

Κεντρική Σελίδα

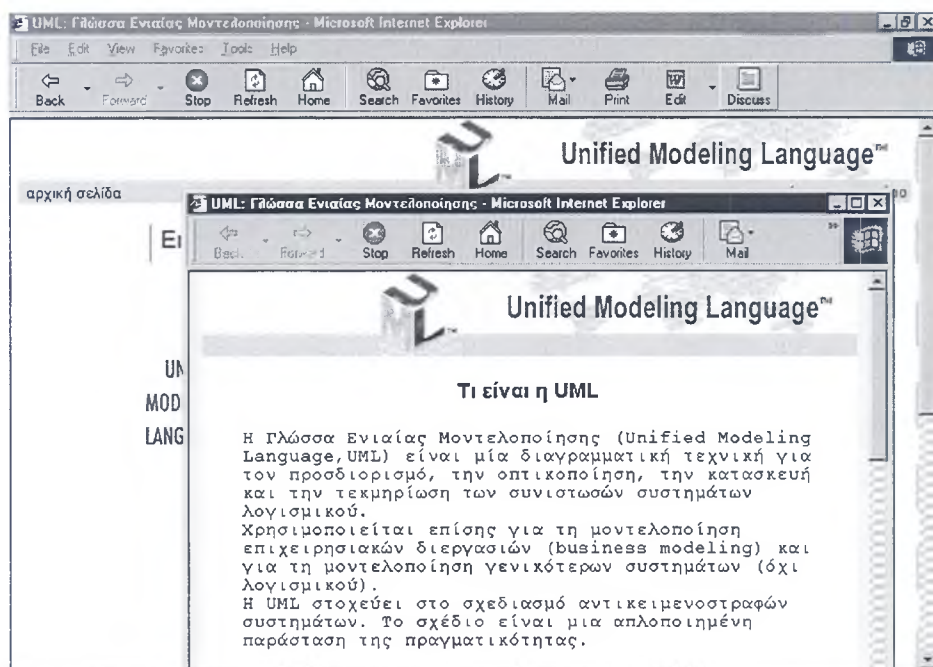


Η κεντρική σελίδα αποτελείται από τρεις βασικές επιλογές πλοήγησης στον ιστοχώρο :

- ❖ **Εισαγωγή στην Αντικειμενοστρεφή Σχεδίαση & στην UML:** με την επιλογή αυτή εμφανίζεται μία νέα ιστοσελίδα, στην οποία παρουσιάζονται βασικές εισαγωγικές έννοιες και σχεδιαγράμματα σχετικά με την UML- Ενοποιημένη Γλώσσα Μοντελοποίησης. Η μορφή της ιστοσελίδας αυτής ακολουθεί παρακάτω:



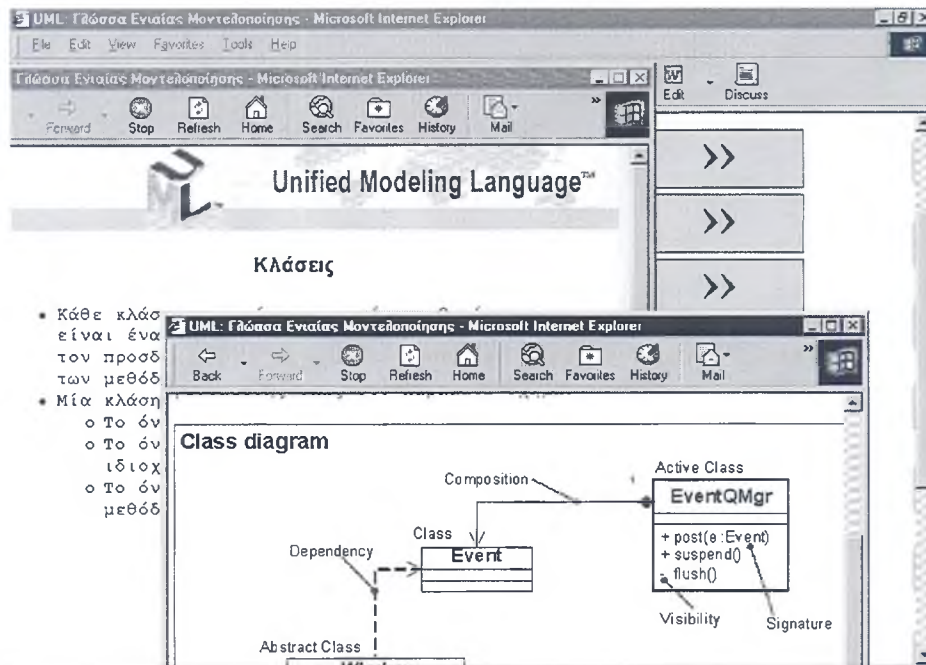
Όπως βλέπουμε στη παραπάνω εικόνα έχει επιλεγθεί το πρώτο κουμπί που παραπέμπει στην ιστοσελίδα «Τι είναι η UML». Η ιστοσελίδα παίρνει την παρακάτω μορφή:



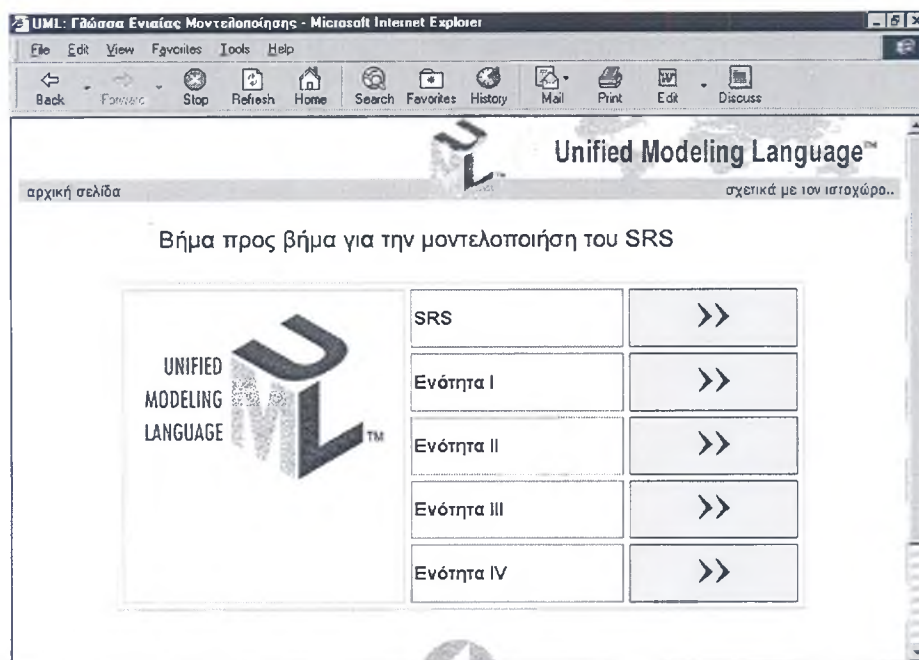
Επίσης παρατηρούμε πώς με την επιλογή κάποιου κουμπιού, αυτό αλλάζει μορφή πριν ενεργοποιηθεί. Αυτό οφείλεται στο javascript το οποίο βοηθά στο

να εναλλάσσονται δύο εικόνες δίνοντας την αίσθηση της κίνησης κάθε φορά που επιλέγουμε με το ποντίκι κάποιο κουμπί. Αυτό υλοποιήθηκε με εργαλείο το Dreamweaver 3.0 όπου απλά δηλώσαμε σε κάθε κουμπί την αντίστοιχη συμπεριφορά με την ενεργοποίηση του.

Είναι σκόπιμο το γεγονός ότι κάθε ιστοσελίδα που προέρχεται από κάποια βασική κατηγορία, εμφανίζεται σε νέο παράθυρο για το λόγο ότι έτσι βοηθάμε τον χρήστη κατά την εκμάθηση του να έχει ανοικτά όσα παράθυρα πιστεύει πως του χρειάζονται. Για παράδειγμα ο χρήστης μπορεί να έχει ανοικτές τις ιστοσελίδες «Κλάσεις» και «Διάγραμμα Κλάσεων» ταυτόχρονα ούτως ώστε να μπορεί πιο εύκολα να κάνει παρατηρήσεις και συγκρίσεις μεταξύ των δύο. Αυτή την περίπτωση την βλέπουμε παρακάτω:

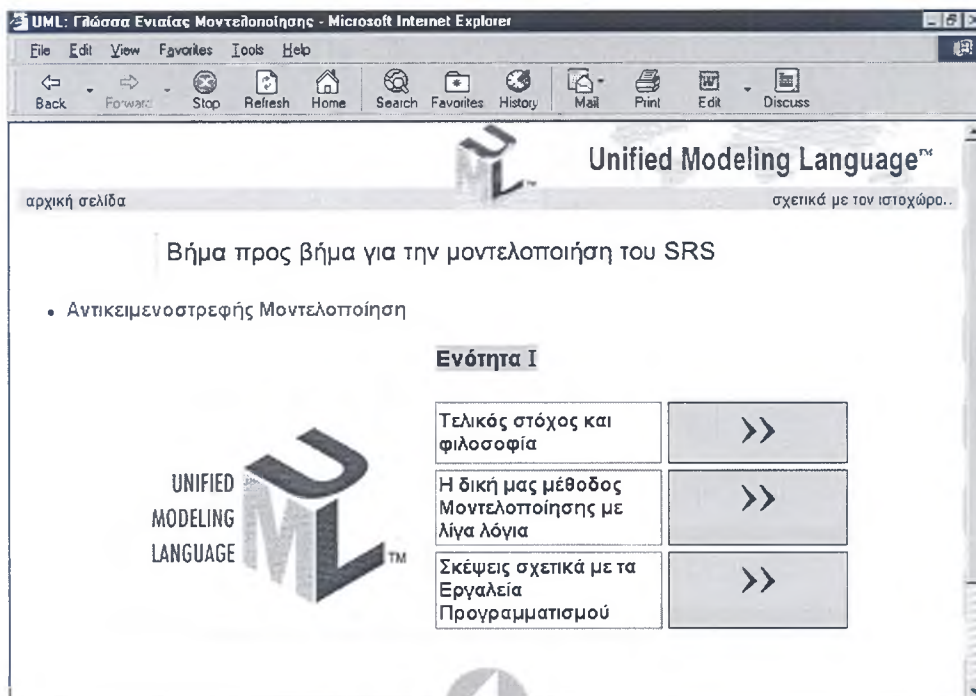


- ❖ **Βήμα προς βήμα για την μοντελοποίηση του SRS:** με την επιλογή αυτή εμφανίζεται μία νέα ιστοσελίδα, η οποία περιλαμβάνει τέσσερις ενότητες οι οποίες βασίζονται στη κατασκευή ενός συστήματος SRS-Σύστημα Εγγραφής Φοιτητών με την χρήση και εφαρμογή της UML. Η μορφή της ιστοσελίδας αυτής ακολουθεί παρακάτω:

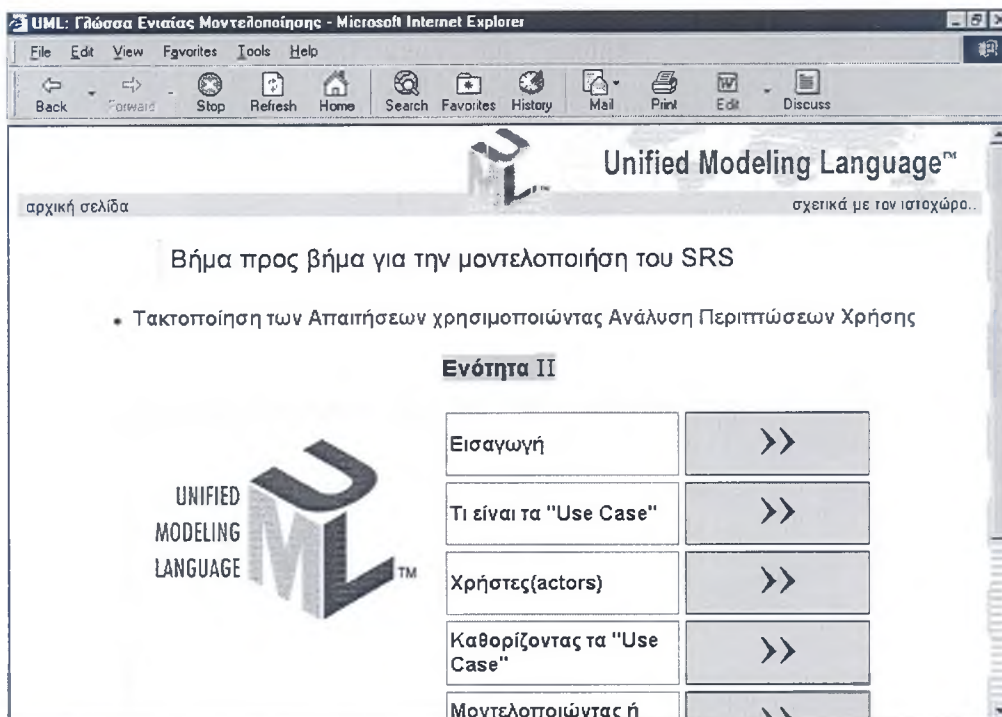


Το υλικό που περιέχεται σε κάθε ενότητα έχει σκοπό την μοντελοποίηση του SRS-Σύστημα Εγγραφής Φοιτητών.

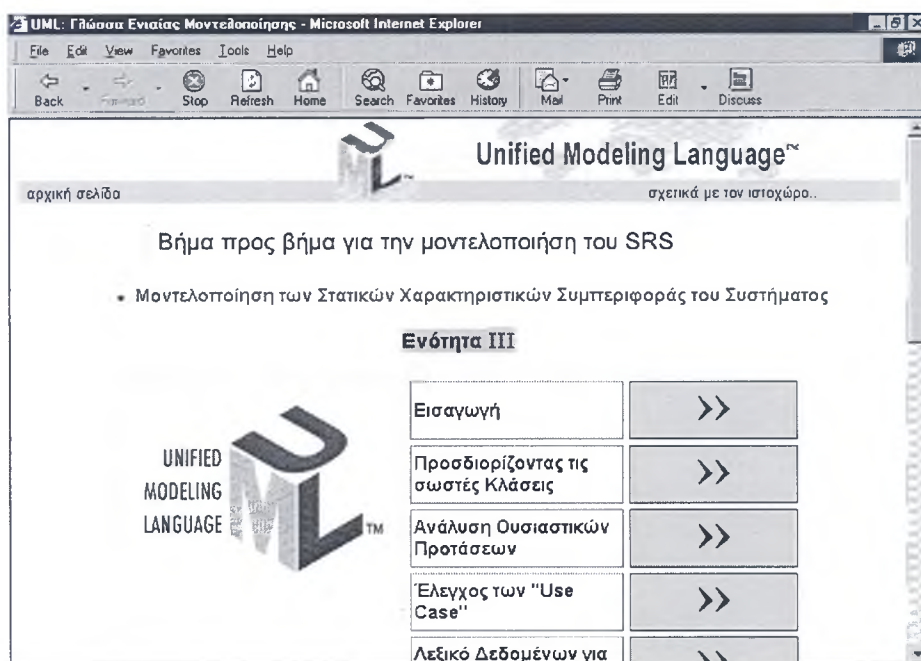
Η Ενότητα I χωρίζεται σε τρεις υποενότητες και έχει σαν γενικό τίτλο "Αντικειμενοστρεφής Μοντελοποίηση"



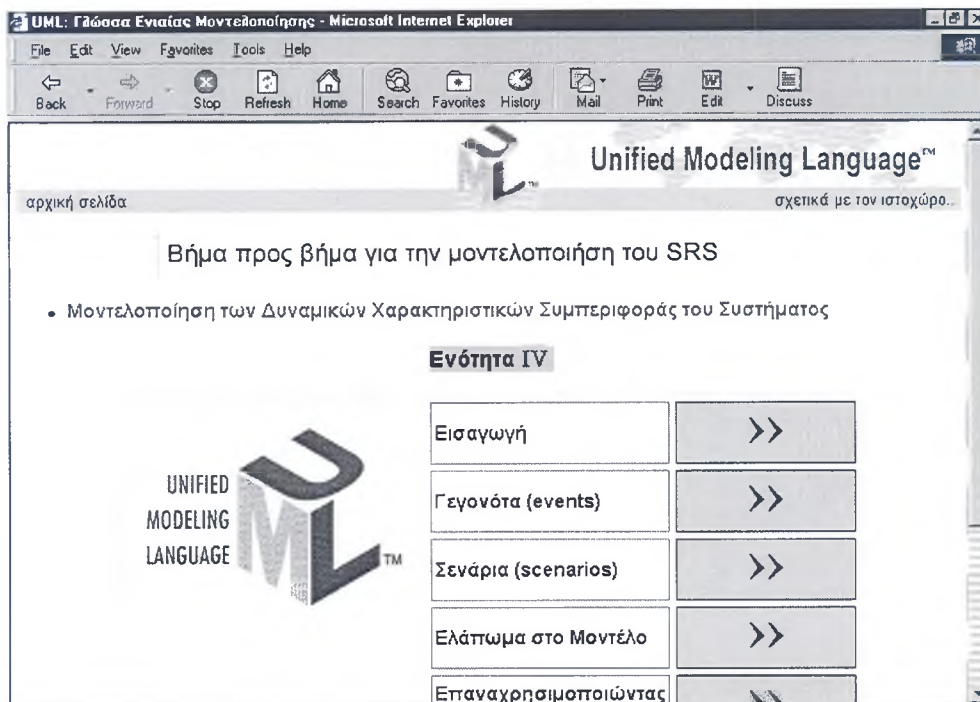
Η Ενότητα II χωρίζεται σε πέντε υποενότητες και έχει σαν γενικό τίτλο "Τακτοποίηση των Απαιτήσεων χρησιμοποιώντας Ανάλυση Περιπτώσεων Χρήσης"



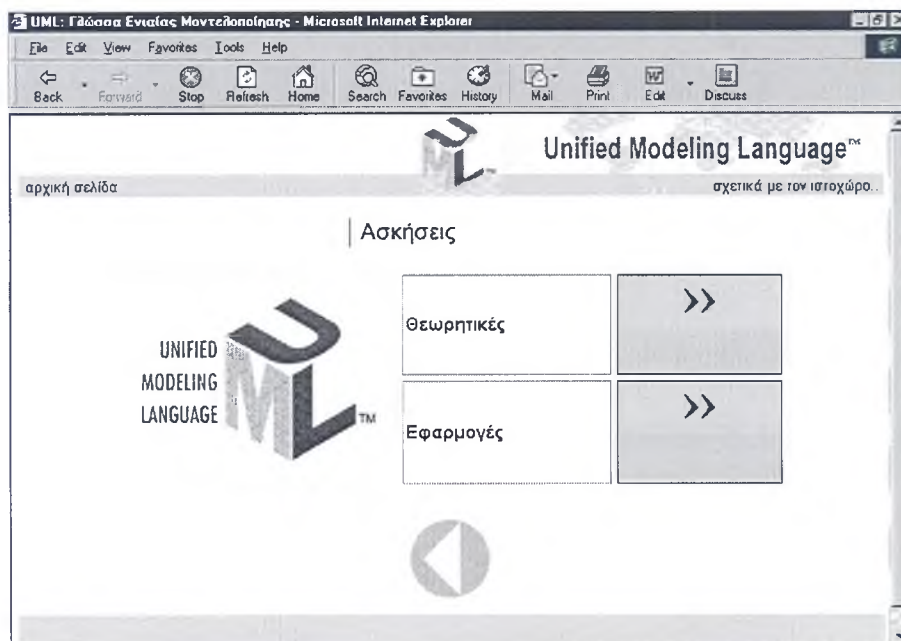
Η Ενότητα III χωρίζεται σε δεκατρείς υποενότητες και έχει σαν γενικό τίτλο "Μοντελοποίηση των Στατικών Χαρακτηριστικών Συμπεριφοράς του Συστήματος"



Η Ενότητα IV χωρίζεται σε πέντε υποενότητες και έχει σαν γενικό τίτλο "Μοντελοποίηση των Δυναμικών Χαρακτηριστικών Συμπεριφοράς του Συστήματος"

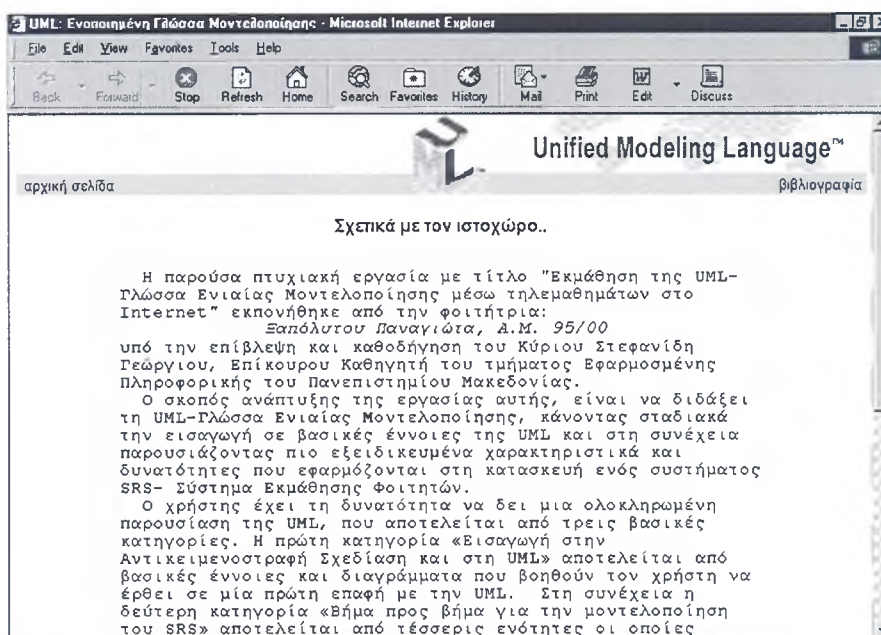


- ❖ **Ασκήσεις:** με την επιλογή αυτή εμφανίζεται μία νέα ιστοσελίδα, η οποία αποτελείται από Θεωρητικές Ασκήσεις με τις αντίστοιχες λύσεις τους και Εφαρμογές που ο χρήστης είναι σε θέση να λύσει ούτως ώστε να προβληματιστεί στα όσα έχει διαβάσει.



και από δύο links που παραπέμπουν:

- ❖ **Σχετικά με τον ιστοχώρο:** Στην ιστοσελίδα αυτή περιέχονται γενικές πληροφορίες για τον ιστοχώρο, για την ομάδα εργασίας και για τα προγράμματα που έχουν χρησιμοποιηθεί.



- ❖ **Βιβλιογραφία:** Στην ιστοσελίδα αυτή υπάρχουν όλες οι βιβλιογραφικές αναφορές και οι ηλεκτρονικές διευθύνσεις που βοήθησαν στην κατασκευή του ιστοχώρου.

Με την επιλογή κάποιας από τις τρεις βασικές επιλογές που προαναφέραμε πιο πάνω «Εισαγωγή στην Αντικειμενοστραφή Σχεδίαση & στην UML», «Βήμα προς βήμα για την μοντελοποίηση του SRS» και «Ασκήσεις» -εικόνες σχεδιασμένες στο Paint των Microsoft Windows και Adobe Photoshop 7.0- ενεργοποιείται η swapImage και με την βοήθεια του javascript εναλλάσσονται δύο εικόνες δίνοντας την αίσθηση της κίνησης κάθε φορά που επιλέγουμε με το ποντίκι κάποια από αυτές τις επιλογές και εμφανίζεται ένα άσπρο τικ πάνω στον κύκλο που αντιστοιχεί στην συγκεκριμένη επιλογή.

Επίσης σε όλες τις ιστοσελίδες εμφανίζεται στο κάτω μέρος της σελίδας και συγκεκριμένα στο scrollit το μήνυμα "Καλώς ήρθατε στην ιστοσελίδα εκμάθησης της UML- Ενοποιημένη Γλώσσα Μοντελοποίησης!" , το οποίο υλοποιείται με τη βοήθεια ενός javascript, το οποίο διατίθεται δωρεάν, για μη εμπορική χρήση, στην ηλεκτρονική διεύθυνση: <http://javascript.internet.com>

Τεκμηρίωση Κώδικα

Ο παρακάτω κώδικας χρωματίζει το scrollbar που βρίσκεται στα δεξιά κάθε σελίδας με χρώματα τις αρεσκείας μας. Διατίθεται δωρεάν, για μη εμπορική χρήση, στην ηλεκτρονική διεύθυνση:

```
<STYLE TYPE="text/css"><!--
BODY {
  scrollbar-arrow-color:33ff99;
  scrollbar-track-color:99ffcc;
  scrollbar-shadow-color:33cc99;
  scrollbar-face-color:33cc99;
  scrollbar-highlight-color:006666;
  scrollbar-darkshadow-color:006666;
  scrollbar-3dlight-color:33cc99;
}
//--></STYLE>
```

Με τη βοήθεια του παρακάτω κώδικας εμφανίζεται στο κάτω μέρος κάθε σελίδας ένα κινούμενο λογότυπο "Καλώς ήρθατε στην ιστοσελίδα εκμάθησης της UML- Ενοποιημένη Γλώσσα Μοντελοποίησης!", το οποίο υλοποιείται με τη βοήθεια ενός javascript, το οποίο διατίθεται δωρεάν, για μη εμπορική χρήση, στην ηλεκτρονική διεύθυνση: <http://javascript.internet.com>

```
<!-- Begin
function scrollit(seed) {
var m1 = "Καλώς ήρθατε στην ιστοσελίδα";
var m2 = " εκμάθησης της";
var m3 = " UML - Ενοποιημένη Γλώσσα Μοντελοποίησης!";
var m4 = "";
var msg=m1+m2+m3+m4;
var out = " ";
var c = 1;
if (seed > 100) {
seed--;
cmd="scrollit("+seed+")";
timerTwo=window.setTimeout(cmd,100);
}
else if (seed <= 100 && seed > 0) {
for (c=0 ; c < seed ; c++) {
out+=" ";
}
out+=msg;
seed--;
window.status=out;
cmd="scrollit("+seed+")";
timerTwo=window.setTimeout(cmd,100);
}
else if (seed <= 0) {
if (-seed < msg.length) {
out+=msg.substring(-seed,msg.length);
seed--;
window.status=out;
cmd="scrollit("+seed+")";
timerTwo=window.setTimeout(cmd,100);
}
else {
window.status=" ";
```

```

timerTwo=window.setTimeout("scrollit(100)",75);
    }
}
}
// End -->

```

Ο κώδικας που ακολουθεί συντελεί στο να ενεργοποιεί κάποια εικόνα και να την αντικαθιστά με μία άλλη ούτως ώστε να φαίνεται πως υπάρχει κίνηση με το πάτημα του ποντικιού σε ένα συγκεκριμένο σημείο. Δημιουργήθηκε με το εργαλείο Dreamweaver 3.0 και την ιδιότητα behaviour(συμπεριφορά).

```

<!--
function MM_preloadImages() { //v3.0
    var d=document; if(d.images){ if(!d.MM_p) d.MM_p=new Array();
    var i,j=d.MM_p.length,a=MM_preloadImages.arguments; for(i=0; i<a.length; i++)
        if (a[i].indexOf("#")!=0){ d.MM_p[j]=new Image; d.MM_p[j++].src=a[i];}}
    }

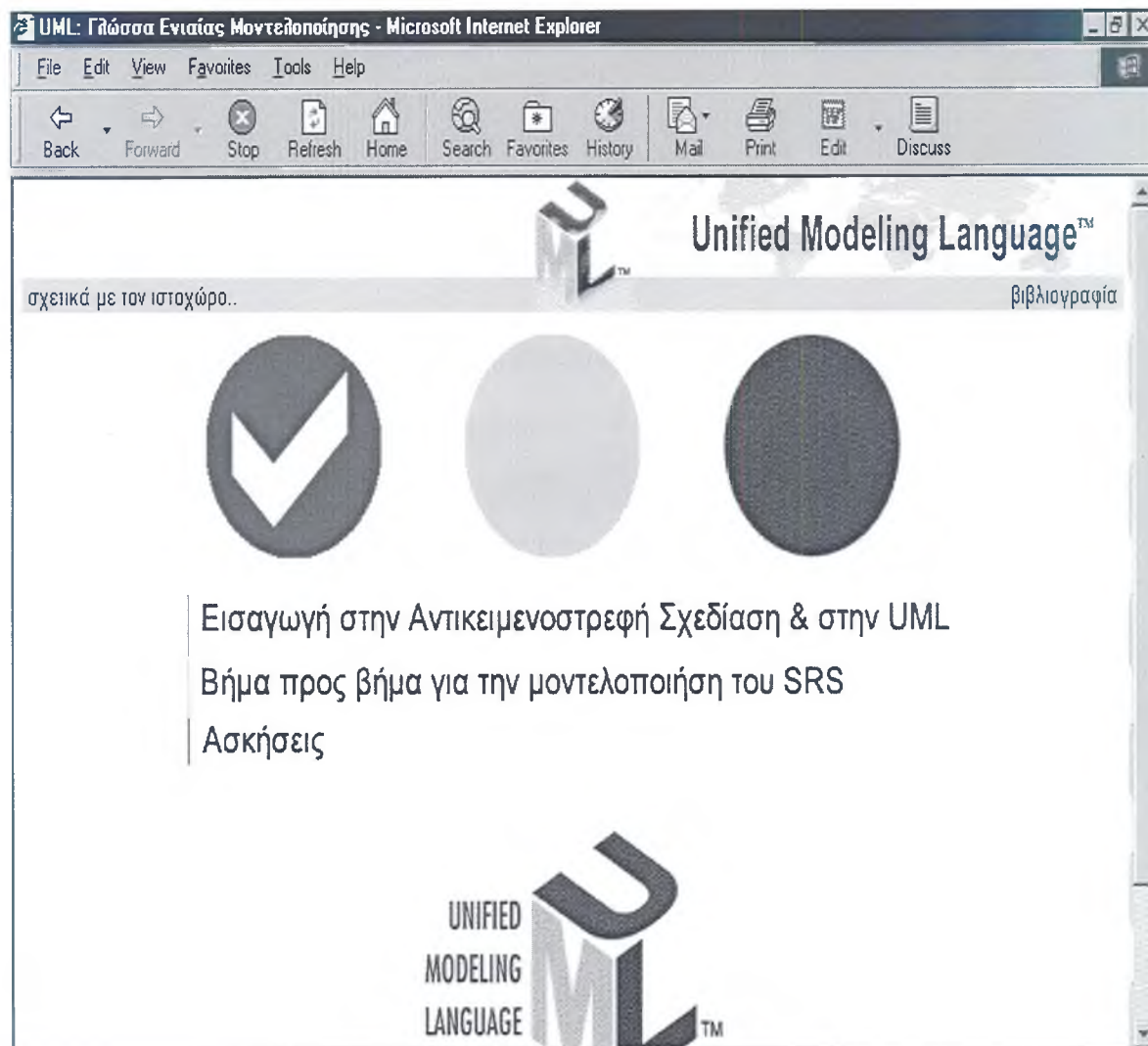
function MM_swapImgRestore() { //v3.0
    var i,x,a=document.MM_sr; for(i=0;a&&i<a.length&&(x=a[i])&&x.oSrc;i++)
x.src=x.oSrc;
}

function MM_findObj(n, d) { //v4.0
    var p,i,x; if(!d) d=document;
    if((p=n.indexOf("?"))>0&&parent.frames.length) {
        d=parent.frames[n.substring(p+1)].document; n=n.substring(0,p);}
    if(!(x=d[n])&&d.all) x=d.all[n]; for (i=0;!x&&i<d.forms.length;i++)
x=d.forms[i][n];
    for(i=0;!x&&d.layers&&i<d.layers.length;i++)
x=MM_findObj(n,d.layers[i].document);
    if(!x && document.getElementById) x=document.getElementById(n); return x;
}

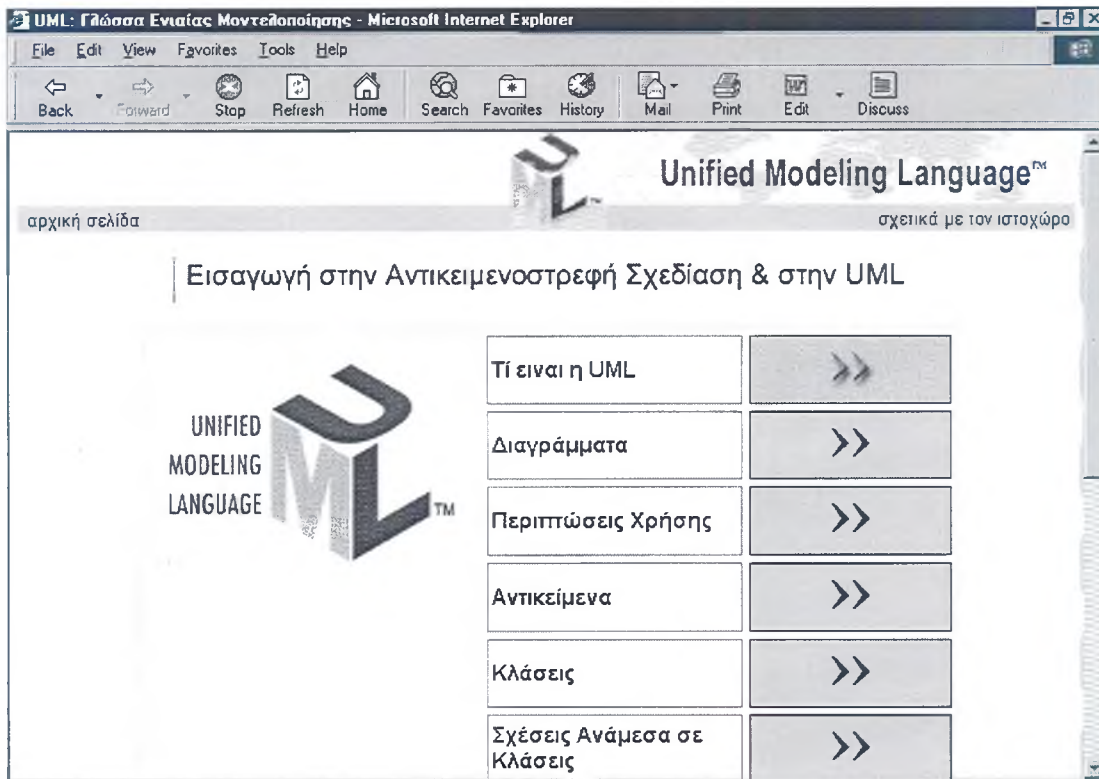
function MM_swapImage() { //v3.0
    var i,j=0,x,a=MM_swapImage.arguments; document.MM_sr=new Array;
    for(i=0;i<(a.length-2);i+=3)
        if ((x=MM_findObj(a[i]))!=null){document.MM_sr[j++]=x; if(!x.oSrc)
x.oSrc=x.src; x.src=a[i+2];}
    }
//-->

```


Κεντρική Σελίδα



Εισαγωγή στην Αντικειμενοστρεφή Σχεδίαση & στη UML



Περιλαμβάνει τα εξής κουμπιά επιλογής:

- Τι είναι η UML
- Διαγράμματα
- Περιπτώσεις Χρήσης
- Αντικείμενα
- Κλάσεις
- Σχέσεις Ανάμεσα σε Κλάσεις
- Σύγκριση Διαγραμμάτων Κλάσης και Αντικειμένου
- Γενίκευση και Ειδίκευση
- Κληρονομικότητα
- Πολυμορφισμός

Τι είναι η UML

Η Ενοποιημένη Γλώσσα Μοντελοποίησης (Unified Modeling Language, UML) είναι μία διαγραμματική τεχνική για τον προσδιορισμό, την οπτικοποίηση, την κατασκευή και την τεκμηρίωση των συνιστωσών συστημάτων λογισμικού.

Χρησιμοποιείται επίσης για τη μοντελοποίηση επιχειρησιακών διεργασιών (business modeling) και για τη μοντελοποίηση γενικότερων συστημάτων (όχι λογισμικού).

Η UML στοχεύει στο σχεδιασμό αντικειμενοστραφών συστημάτων. Το σχέδιο είναι μια απλοποιημένη παράσταση της πραγματικότητας.

Σχεδιάζουμε για να μπορέσουμε να καταλάβουμε το σύστημα που αναπτύσσουμε. Έτσι δημιουργώντας ένα σχέδιο επιτυγχάνουμε τέσσερις στόχους:

1. παριστάνουμε οπτικά το σύστημα που έχουμε ή θέλουμε να κατασκευάσουμε,
2. προσδιορίζουμε τη δομή και τη συμπεριφορά του συστήματος,
3. δημιουργούμε ένα πρότυπο για να βασίσουμε την κατασκευή του συστήματος,
4. τεκμηριώνουμε τις αποφάσεις που λάβαμε.

Σε όλους τους τεχνολογικούς τομείς ο σχεδιασμός βασίζεται σε τέσσερις βασικές αρχές:

- η επιλογή του είδους του σχεδίου έχει επίπτωση στον τρόπο και την μορφή επίλυσης του προβλήματος,
- όλα τα σχέδια εκφράζονται σε διαφορετικές βαθμίδες ακρίβειας,
- τα καλύτερα σχέδια σχετίζονται με την πραγματικότητα,
- ένα είδος σχεδίων δεν είναι ποτέ αρκετό.

Η UML περιλαμβάνει τρία βασικά στοιχεία:

1. Οντότητες
2. Σχέσεις
3. Διαγράμματα

Η UML δεν είναι:

- Μια αντικειμενοστραφής θεωρία, αν και ενσωματώνει και υποστηρίζει βασικές αρχές της αντικειμενοστραφούς τεχνολογίας.

■ Ένα εργαλείο υποστήριξης της ανάπτυξης λογισμικού (case tool), αν και υπάρχουν πολλά τέτοια εργαλεία που υποστηρίζουν την UML (π.χ. τα προϊόντα της Rational).

■ Μεθοδολογία ανάπτυξης, αν και έχει στενή σχέση με την ενιαία διεργασία ανάπτυξης λογισμικού (Unified Software Development Process, USDP) και την ενιαία διεργασία της Rational (Rational Unified Process, RUP).

Αλλά:

- Είναι ανεξάρτητη από κάποια συγκεκριμένη γλώσσα προγραμματισμού.
- Είναι επεκτάσιμη έχοντας ένα καλά ορισμένο πυρήνα.
- Έχει μια τυπική (formal) βάση.
- Είναι σε θέση να υποστηρίξει έννοιες υψηλού επιπέδου, όπως τα μοτίβα σχεδίασης (design patterns) και τα σχεδιαστικά πλαίσια (design frameworks).
- Ορίζεται με τρόπο αναδρομικό χρησιμοποιώντας ένα μετα-μοντέλο.
- Επιτρέπει την ανάπτυξη κατάλληλων εργαλείων λογισμικού.

Διαγράμματα

Η UML ορίζει τα παρακάτω διαγράμματα:

Διάγραμμα Περιπτώσεων Χρήσης (Use Case diagrams)

Διαγράμματα Δομής (Static Structure diagrams)

- ο Διάγραμμα Αντικειμένων (Object diagram)
- ο Διάγραμμα Κλάσεων (Class diagram)

Διαγράμματα Αλληλεπίδρασης (Interaction diagrams)

- ο Διάγραμμα Ακολουθίας (Sequence diagram)
- ο Διάγραμμα Συνεργασίας (Collaboration diagram)

Διάγραμμα Καταστάσεων (Statechart diagram)

Διάγραμμα Δραστηριοτήτων (Activity diagram)

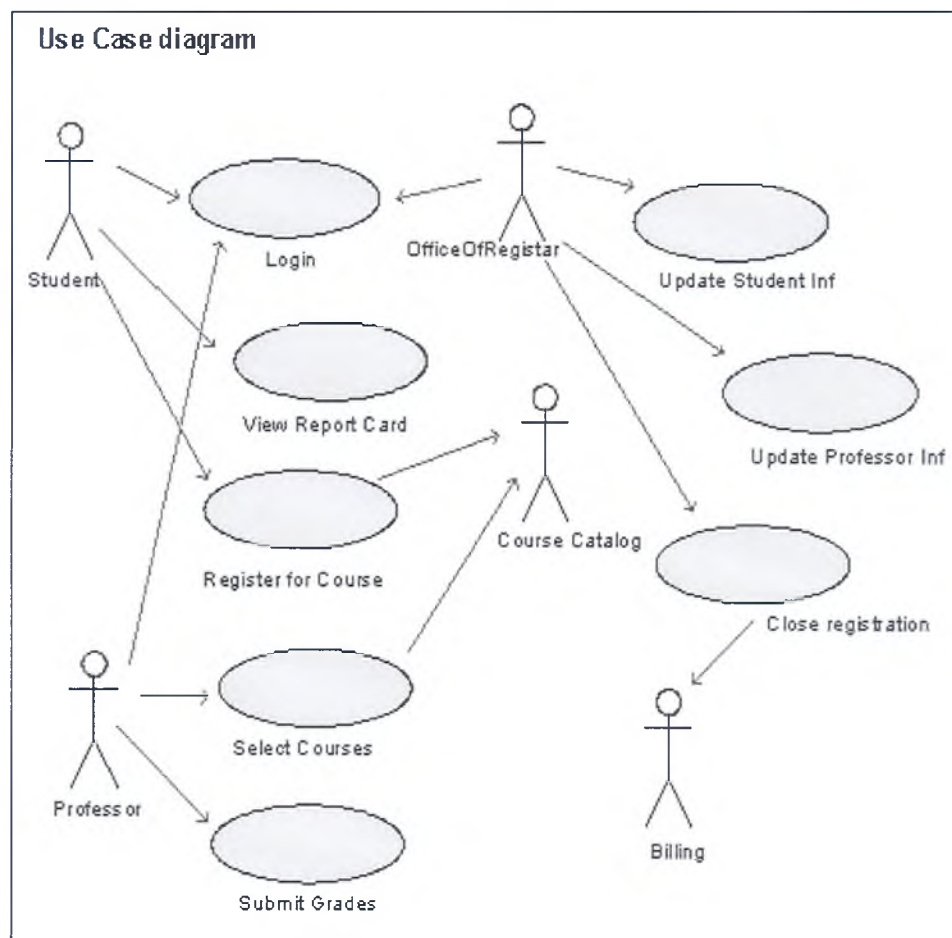
Διαγράμματα Δομής Υλοποίησης (Implementation diagrams)

- ο Διάγραμμα Συστατικών (Component diagram)
- ο Διάγραμμα Ανάπτυξης (Deployment diagram)

Διάγραμμα Περιπτώσεων Χρήσης

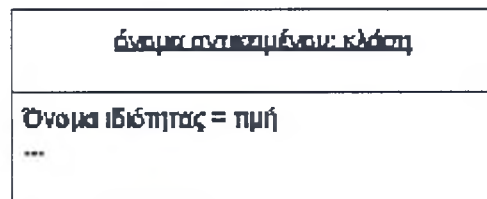
Το *διάγραμμα περιπτώσεων* χρήσης χρησιμοποιείται για να μοντελοποιήσει το πλαίσιο λειτουργίας του συστήματος καθώς και τις προδιαγραφές του. Περιλαμβάνει:

- ✦ Περιπτώσεις χρήσης
- ✦ Χρήστες(actors)
- ✦ Σχέσεις εξάρτησης, γενίκευσης, σύνδεσης
- ✦ Τα όρια του συστήματος

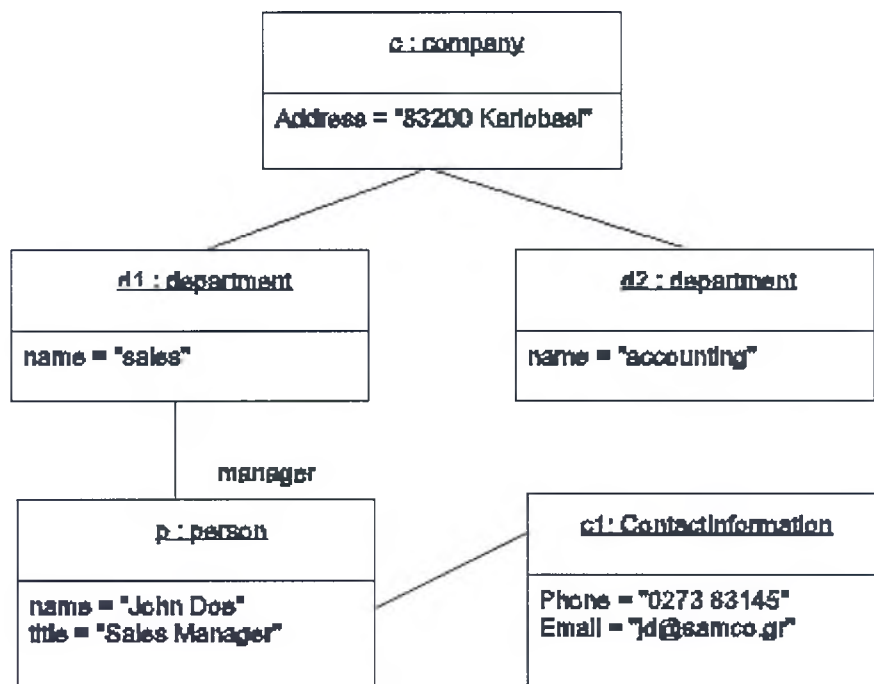


Διάγραμμα Αντικειμένων

Τα διαγράμματα αντικειμένων είναι *διαγράμματα δομής* που χρησιμοποιούνται για το σχεδιασμό της στατικής κατάστασης του συστήματος κατά μια συγκεκριμένη χρονική στιγμή. Κάθε αντικείμενο σχεδιάζεται ως ένα ορθογώνιο με την παρακάτω μορφή:

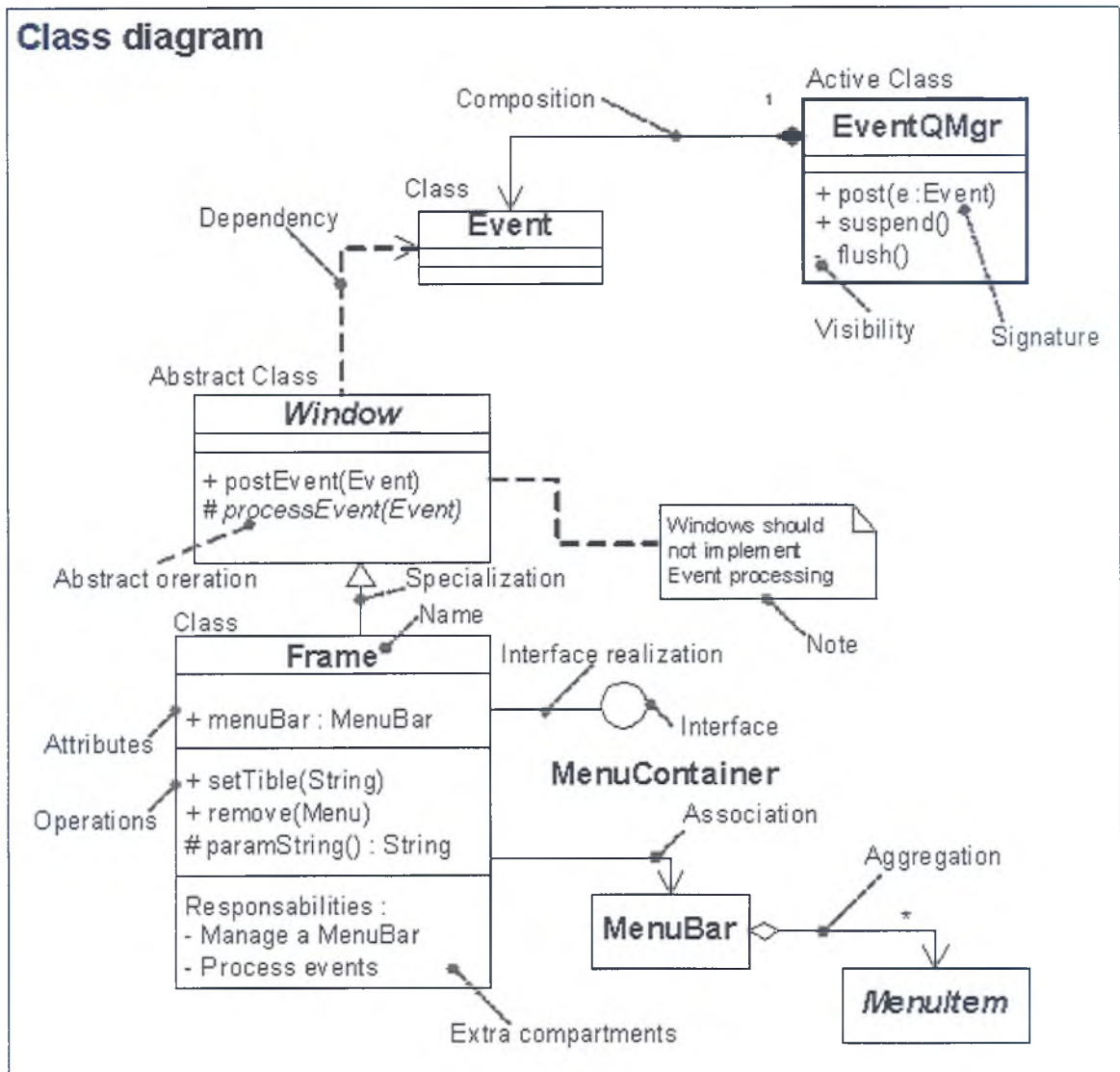


Το σύνολο των αντικειμένων σχεδιάζεται με βάση τους συνδέσμους που ορίζονται πάνω σε αυτό.



Διάγραμμα Κλάσεων

Το **διάγραμμα κλάσεων** ενός συστήματος είναι ένα *διάγραμμα δομής* που περιέχει τις κλάσεις μαζί με του αντίστοιχους δεσμούς εξάρτησης, γενίκευσης και σύνδεσης. Έτσι ένα διάγραμμα κλάσεων μπορεί να απεικονίσει τη χρήση της κληρονομικότητας στο σχεδιασμό με τη χρήση δεσμών γενίκευσης όπως στο παρακάτω σχήμα:



Διάγραμμα Ακολουθίας

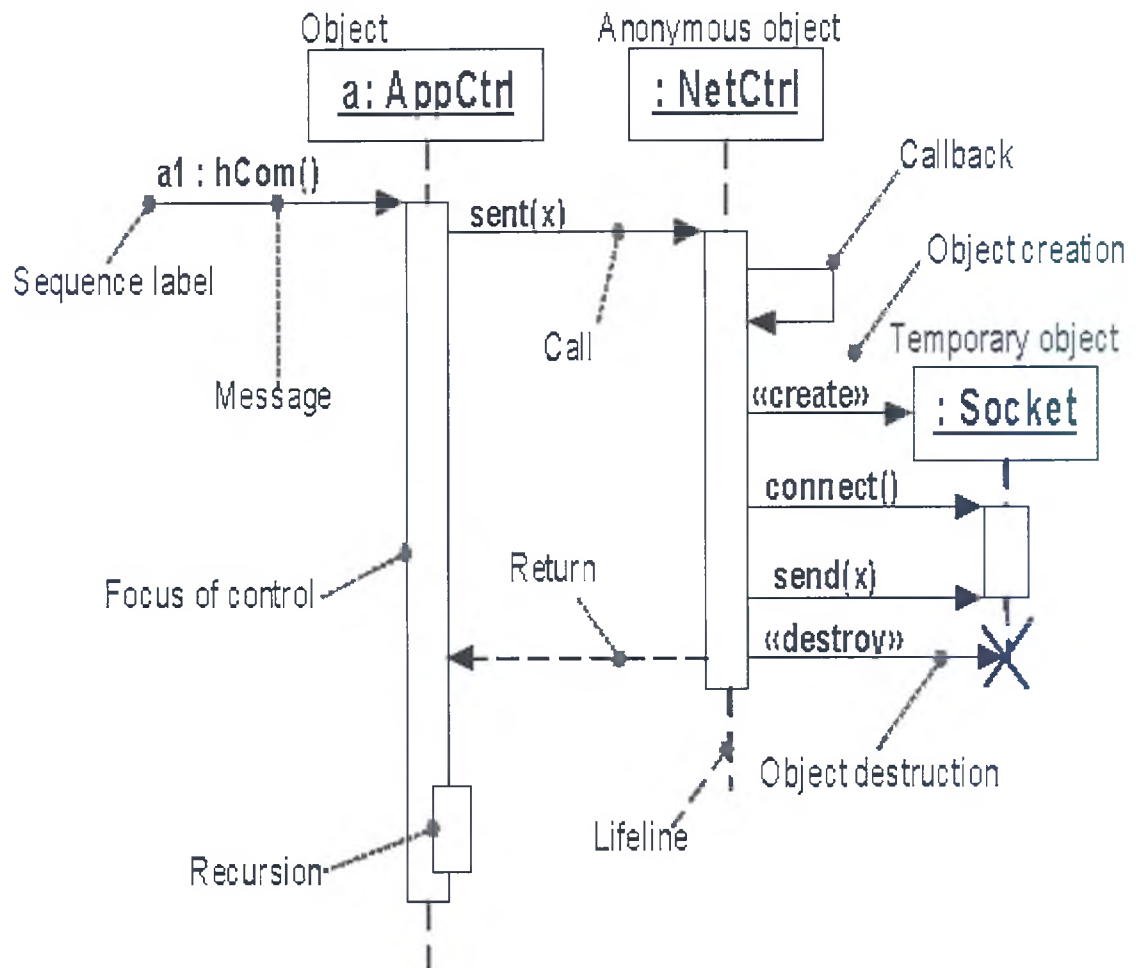
Το **διάγραμμα ακολουθίας**(Sequence) είναι ένα *διάγραμμα αλληλεπίδρασης* (συμπεριφοράς) που παρουσιάζει τον τρόπο που διαφορετικά αντικείμενα συνεργάζονται μεταξύ τους σε μια χρονική ακολουθία.

Στα διαγράμματα ακολουθίας τα αντικείμενα παριστάνονται με κουτιά ή/και ρόλους τα οποία παρατάσσονται στο χώρο (συνήθως από αριστερά προς τα δεξιά). Τα αντικείμενα ανταλλάσσουν μηνύματα τα οποία παριστάνονται με κατευθυνόμενα βέλη των οποίων η φορά δείχνει την κατεύθυνση του μηνύματος (ποιο αντικείμενο κάνει την κλήση και σε ποιο αντικείμενο γίνεται η κλήση). Ο χρόνος παριστάνεται από πάνω προς τα κάτω στο διάγραμμα. Δηλαδή τα μηνύματα που βρίσκονται πιο πάνω στο διάγραμμα προηγούνται χρονικά αυτών που βρίσκονται πιο χαμηλά.

Ένα διάγραμμα ακολουθίας περιλαμβάνει:

- ✚ Αντικείμενα
- ✚ Σχέσεις μεταξύ αντικειμένων
- ✚ Μηνύματα
- ✚ Τη διάρκεια ζωής κάθε αντικειμένου
- ✚ Την περιοχή ελέγχου για κάθε αντικείμενο

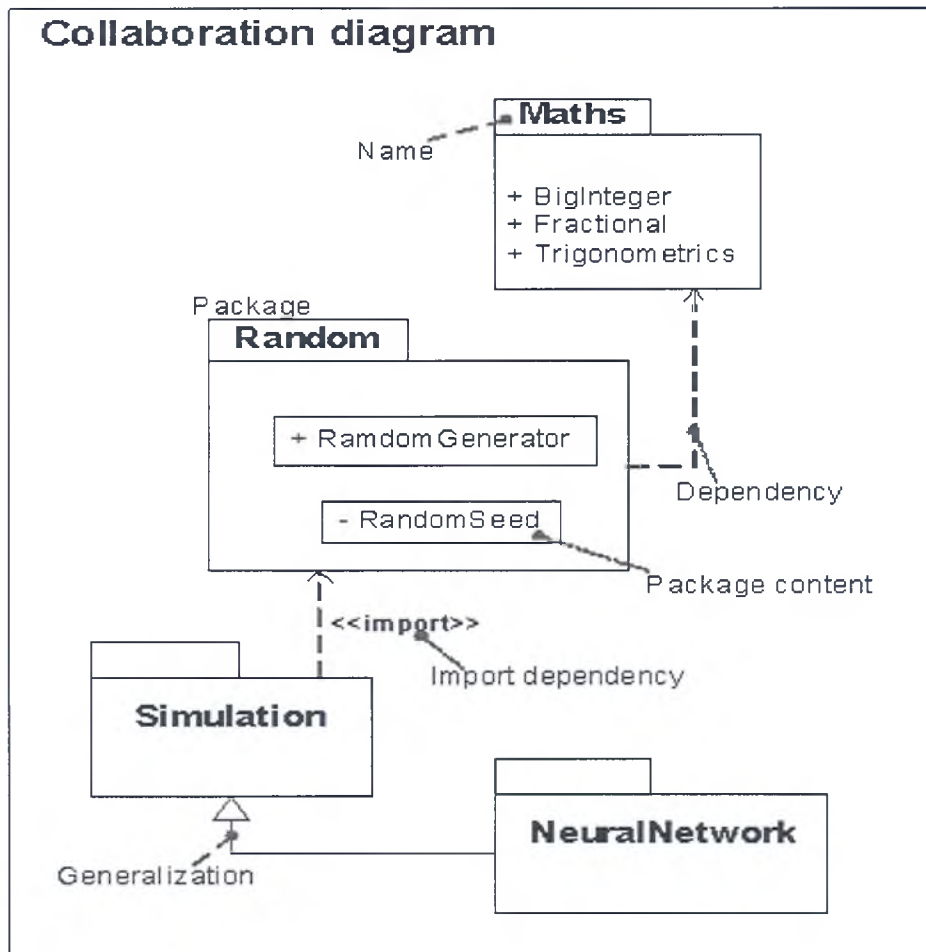
Sequence diagram



Διάγραμμα Συνεργασίας

Το *διάγραμμα συνεργασίας* είναι ένα *διάγραμμα αλληλεπίδρασης* (συμπεριφοράς) που παρουσιάζει τον τρόπο που διαφορετικά αντικείμενα σχετίζονται και ανταλλάσσουν μηνύματα μεταξύ τους. Περιλαμβάνει:

- ✦ Αντικείμενα
- ✦ Σχέσεις μεταξύ αντικειμένων
- ✦ Αριθμημένα μηνύματα



Διάγραμμα Καταστάσεων

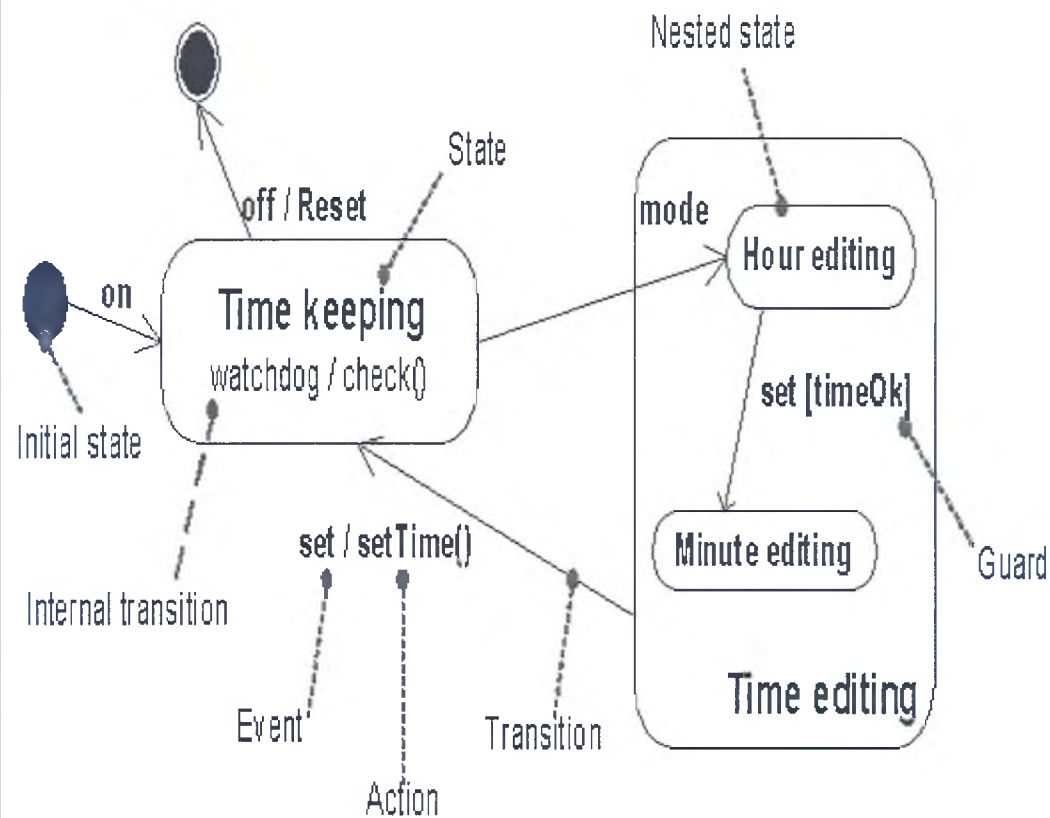
Το **διάγραμμα καταστάσεων** είναι ένα *διάγραμμα συμπεριφοράς* που εμφανίζει μια μηχανή καταστάσεων με έμφαση στις μεταπτώσεις μεταξύ καταστάσεων από διάφορα γεγονότα.

Ουσιαστικά τα διαγράμματα κατάστασης περιγράφουν όλες τις πιθανές καταστάσεις (States) στις οποίες μπορεί να βρεθούν τα αντικείμενα μιας τάξης και τον τρόπο που αλλάζει η κατάσταση αυτών των αντικειμένων σαν ανάδραση σε συμβάντα που προκαλούνται από άλλα αντικείμενα και φτάνουν στα αντικείμενα της τάξης την δυναμική συμπεριφορά της οποίας και περιγράφουμε.

Έτσι μπορούμε να πούμε, ότι τα διαγράμματα κατάστασης δείχνουν την ιστορία της ζωής των αντικειμένων μιας δεδομένης τάξης, τα συμβάντα (events) που προκαλούν τις μεταβάσεις από μια κατάσταση σε κάποια άλλη κατάσταση, και τις ενέργειες (actions) που είναι αποτέλεσμα αυτών των αλλαγών κατάστασης.

Δεν δημιουργούμε διαγράμματα κατάστασης για όλες τις τάξεις ενός συστήματος αλλά μόνο για εκείνες των οποίων τα αντικείμενα επιδεικνύουν μια έντονα δυναμική συμπεριφορά.

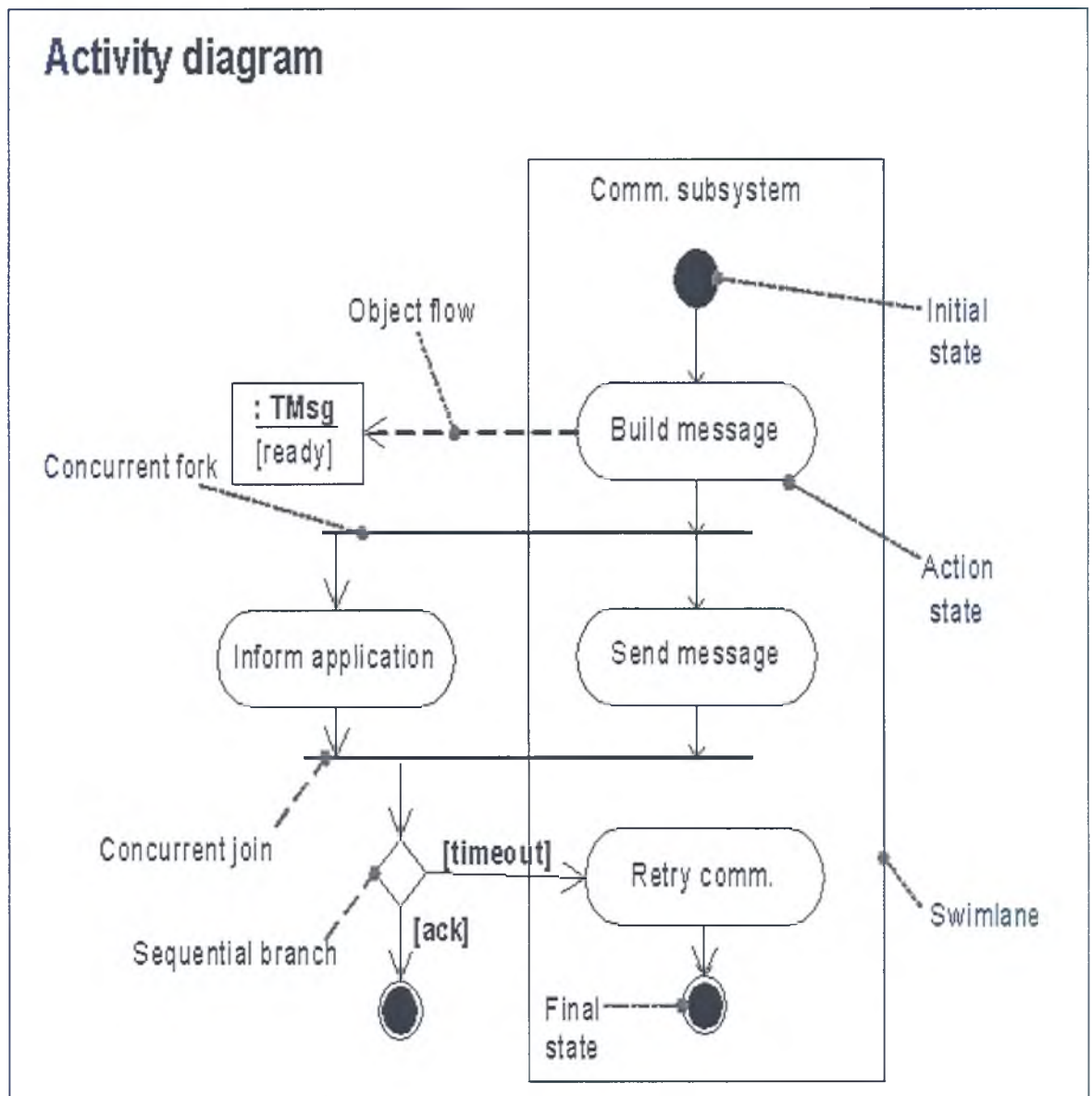
State diagram



Διάγραμμα Δραστηριοτήτων

Όπως και το διάγραμμα καταστάσεων έτσι και το **διάγραμμα δραστηριοτήτων** είναι ένα *διάγραμμα συμπεριφοράς* που εμφανίζει μεταπτώσεις. Η διαφορά του από το διάγραμμα καταστάσεων είναι πως οι μεταπτώσεις είναι ανάμεσα σε διαφορετικές δραστηριότητες εκφράζοντας π.χ. τη ροή εργασιών(workflow). Εκτός από τα στοιχεία του διαγράμματος καταστάσεων περιλαμβάνει συχνά ακόμα:

- ✦ Διακλάδωση υπό συνθήκη(branch)
- ✦ Διάσπαση(fork)
- ✦ Ένωση(join)



Διάγραμμα Συστατικών

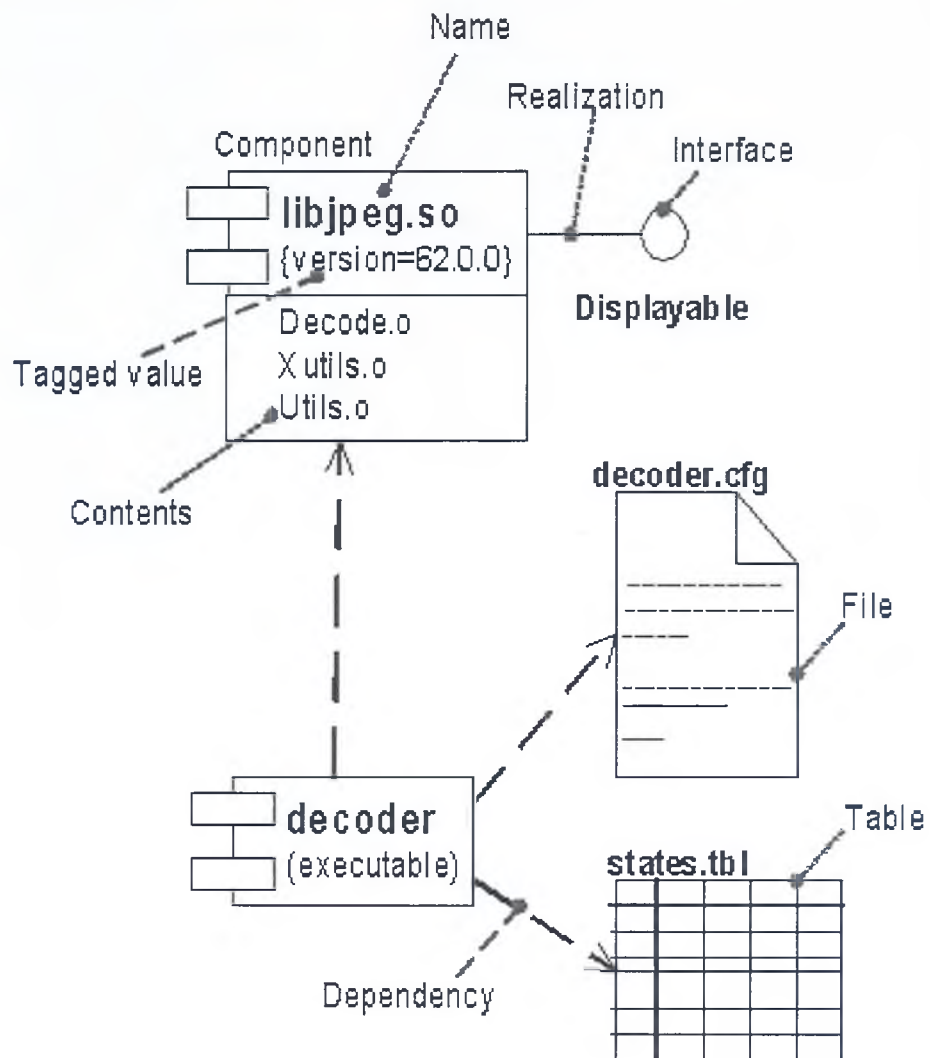
Το διάγραμμα συστατικών είναι ένα *διάγραμμα δομής υλοποίησης* που χρησιμοποιείται για να μοντελοποιήσει:

- ✦ Πηγαίο κώδικα
- ✦ Εκτελέσιμες εκδόσεις
- ✦ Βάσεις δεδομένων
- ✦ Δυναμικά προσαρμοζόμενα συστήματα

Περιλαμβάνει:

- ✦ Συστατικά
- ✦ Διεπαφές(interfaces)
- ✦ Σχέσεις εξάρτησης, γενίκευσης, σύνδεσης και υλοποίησης

Component diagram

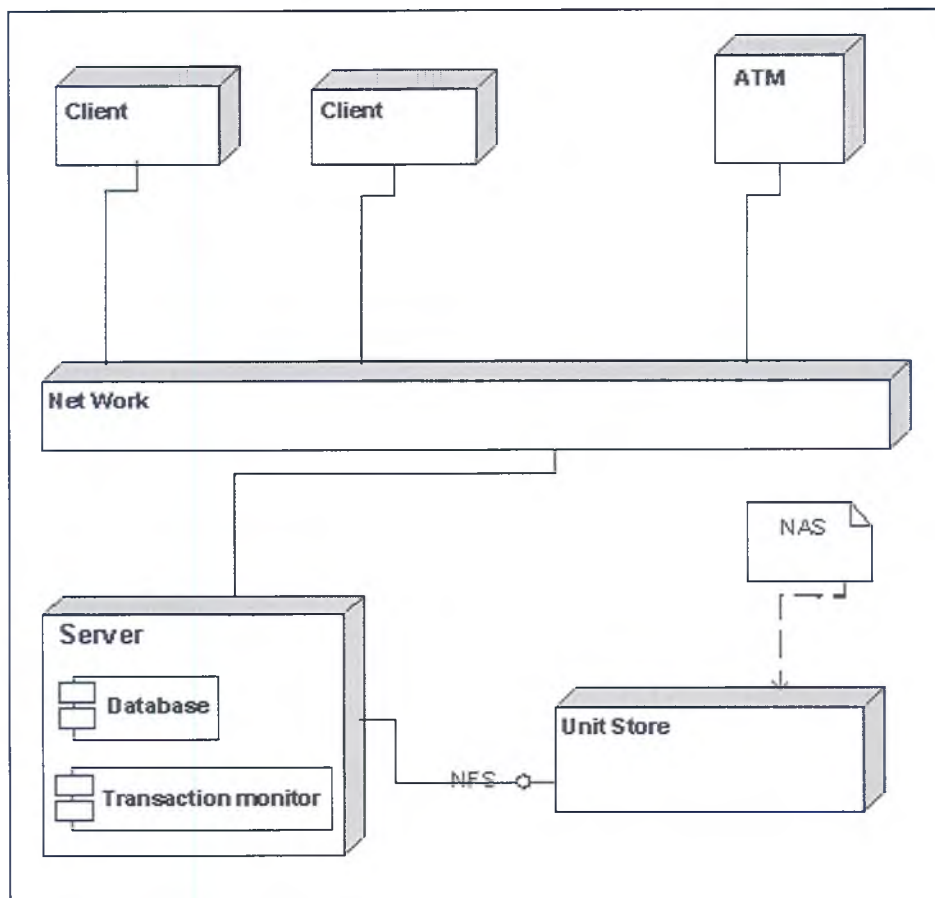


Διάγραμμα Ανάπτυξης

Το **διάγραμμα ανάπτυξης** είναι ένα *διάγραμμα δομής υλοποίησης* που παρουσιάζει τον τρόπο διαμόρφωσης των υπολογιστικών κόμβων του συστήματος κατά τη λειτουργία του. Περιλαμβάνει:

- ✦ Υπολογιστικούς κόμβους
- ✦ Σχέσεις εξάρτησης και σύνδεσης
- ✦ Συστατικά (μέσα σε κόμβους)
- ✦ Πακέτα (που ομαδοποιούν κόμβους)

Παράδειγμα:



Περιπτώσεις Χρήσης

Τι είναι η μοντελοποίηση με περιπτώσεις χρήσης;

Μοντέλο περιπτώσεων χρήσης: μία άποψη του συστήματος που αποδίδει έμφαση στην λειτουργικότητα ενός συστήματος, όπως αυτή είναι ορατή από τους εξωτερικούς χρήστες του συστήματος. Μια περίπτωση χρήσης διαμερίζει την λειτουργικότητα ενός συστήματος σε συναλλαγές («περιπτώσεις χρήσης») που έχουν νόημα για τους χρήστες του συστήματος («ρόλους»).

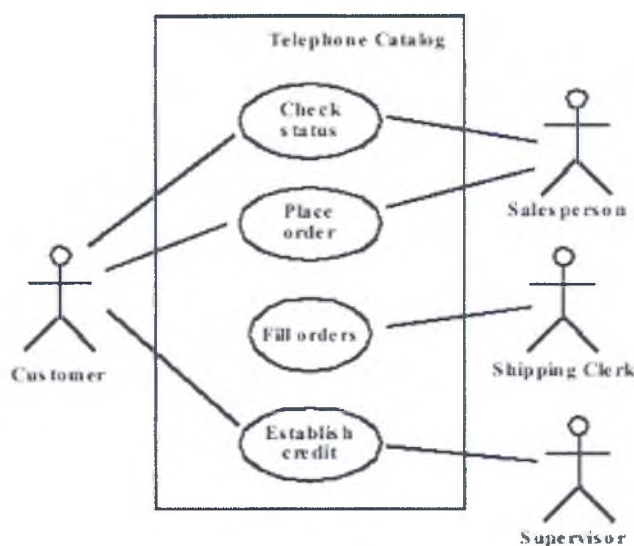
Ο ακόλουθος πίνακας συνοψίζει τις βασικές έννοιες που συναντάμε σε ένα διάγραμμα περιπτώσεων χρήσης.

ΚΑΤΑΣΚΕΥΗ	ΠΕΡΙΓΡΑΦΗ	ΣΥΝΤΑΚΤΙΚΟ
Περίπτωση χρήσης	Μία ακολουθία ενεργειών, συμπεριλαμβανομένων και των παραλλαγών τους, που μπορεί να επιτελέσει ένα σύστημα σε αλληλεπίδραση με τους ρόλους που υπάρχουν στο σύστημα αυτό	 Use Case Name
Ρόλος	Ένας ρόλος είναι ένας ρόλος που μπορεί να παίξει ένας χρήστης του συστήματος όταν αλληλεπιδρά με τις περιπτώσεις χρήσης του συστήματος	 Actor Name
Συσχέτιση	Δηλώνει την συμμετοχή ενός ρόλου σε μια περίπτωση χρήσης	
Επέκταση	Η σχέση μιας επεκταμένης και μιας περίπτωσης χρήσης βάσης, που προσδιορίζει πως θα γίνει αυτή η επέκταση (τα σημεία επέκτασης).	
Γενίκευση	Μια συσχέτιση μιας πιο γενικής περίπτωσης χρήσης με μια πιο ειδική περίπτωση χρήσης.	
Περιεκτικότητα	Μία τέτοια συσχέτιση δείχνει ότι μια περίπτωση χρήσης περιλαμβάνει τις λειτουργίες	

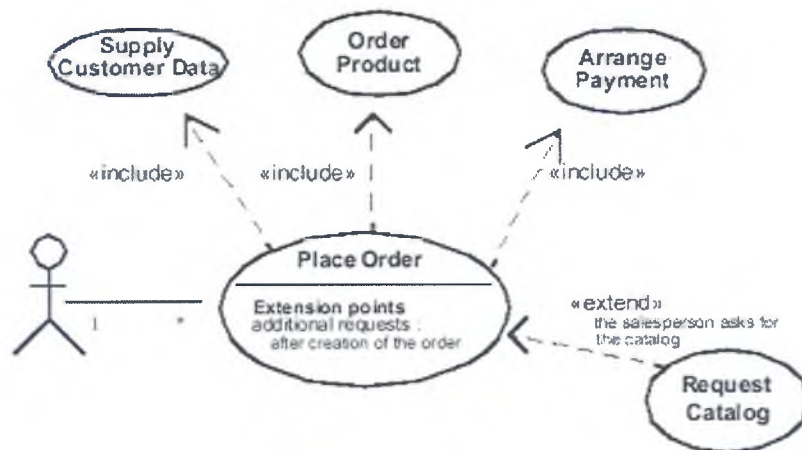
	μιας άλλης περίπτωσης χρήσης. Δηλαδή η λειτουργικότητα της περιλαμβανόμενης περίπτωσης χρήσης εισάγεται στην περίπτωση χρήσης βάσης.	
--	--	--

Μερικά διαγράμματα περιπτώσεων χρήσης είναι τα ακόλουθα:

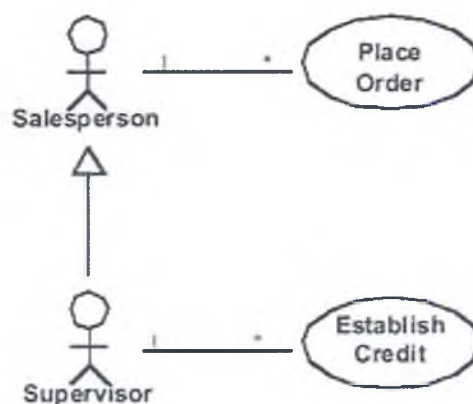
Το ακόλουθο είναι ένα διάγραμμα που απεικονίζει ένα Πελάτη (Customer) που συνδιαλέγεται με ένα σύστημα αγοράς εξ αποστάσεως (μέσω τηλεφώνου ή και μέσω Internet). Ο πελάτης θέλει να μπορεί να ελέγξει την κατάσταση σε ότι αφορά ένα προϊόν (Check Status), να μπορεί να δώσει μια παραγγελία (Place Order) και να μπορεί να δώσει και να επιβεβαιώσει τα στοιχεία της πιστωτικής του κάρτας. Ο πωλητής (Salesperson - υποθέτουμε ότι η εταιρεία πέρα από τηλεφωνικές πωλήσεις κάνει και άλλες πωλήσεις μέσω πωλητών) θέλει να μπορεί να κάνει τα ίδια που κάνει και ένας πελάτης με την διαφορά ότι δεν χρειάζεται να δώσει τις λεπτομέρειες της πιστωτικής του κάρτας. Ο υπάλληλος αποστολής (Shipping Clerk) θέλει να μπορεί να εκδώσει τις παραγγελίες (Fill Orders), και τέλος ο Επόπτης (Supervisor) θέλει να μπορεί να εξετάσει την πιστωτική εικόνα των πελατών (Establish Credit).



Το ακόλουθο είναι ένα παράδειγμα των σχέσεων περιεκτικότητας («includes») και επέκτασης («extend»).



Επίσης μπορούν να υπάρξουν και σχέσεις μεταξύ των ρόλων ενός διαγράμματος περιπτώσεων χρήσης όπως δείχνει το ακόλουθο διάγραμμα:



Συμβουλές μοντελοποίησης με περιπτώσεις χρήσης

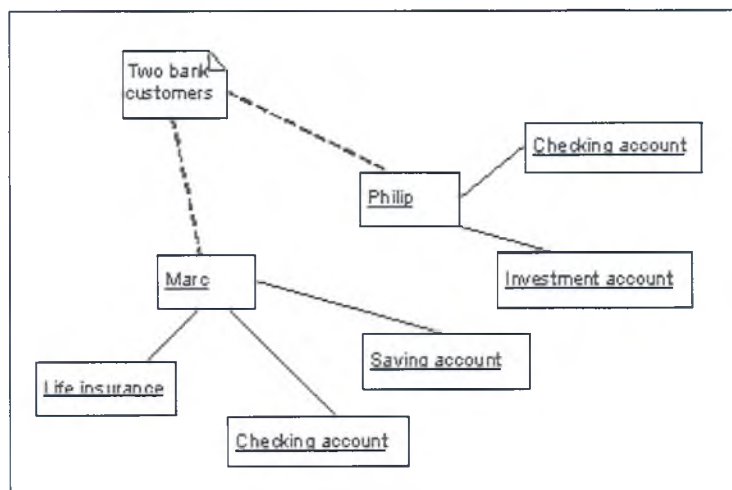
- Εξασφαλίστε ότι κάθε περίπτωση χρήσης περιγράφει ένα σημαντικό τμήμα της λειτουργικότητας του συστήματος που είναι κατανοητό από τους ειδικούς του χώρου ανάπτυξης (domain experts) αλλά και από τους προγραμματιστές.
- Όταν ορίζετε τις περιπτώσεις χρήσης με κείμενο να είστε ακριβείς και συνεπείς με τα ρήματα και τα ουσιαστικά. Αυτό θα βοηθήσει στην παραγωγή των αντικειμένων και των διαγραμμάτων αλληλεπίδρασης αργότερα
- Εξάγετε τις περιπτώσεις χρήσης που απαιτούνται από άλλες περιπτώσεις χρήσης:
 - Αν η περίπτωση χρήσης που περιλαμβάνεται είναι υποχρεωτική χρησιμοποιείτε σχέση περιεκτικότητας («include»).

- Αν η περίπτωση χρήσης βάσης είναι πλήρης και η χρήση της περιλαμβανόμενης περίπτωσης χρήσης προαιρετική χρησιμοποιείτε τη σχέση της επέκτασης («extend»).
- Ένα διάγραμμα περιπτώσεων χρήσης θα πρέπει:
 - Να περιέχει μόνο περιπτώσεις χρήσης που ανήκουν στο ίδιο επίπεδο αφαίρεσης
 - Να περιλαμβάνει μόνο ρόλους που απαιτούνται
- Αν σε κάποιο έργο υπάρχουν πολλές περιπτώσεις χρήσης τότε αυτές θα πρέπει να οργανωθούν σε πακέτα.

Αντικείμενα

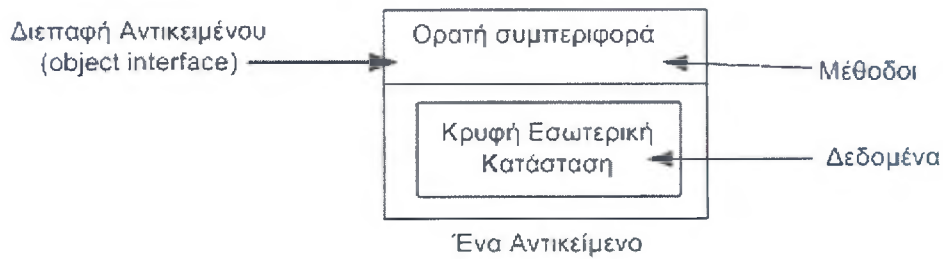
- Ένα αντικείμενο δηλώνεται έχοντας το όνομα του υπογραμμισμένο και παριστάνεται με ένα ορθογώνιο παραλληλόγραμμο. Οι γραμμές ανάμεσα στα αντικείμενα φανερώνουν την ύπαρξη συνδέσμου ανάμεσα τους.
- Ένα αντικείμενο μπορεί να συνδέεται με ένα σημείωμα(note) όπου:
 - Παριστάνεται με ένα ορθογώνιο που έχει διπλωμένη την πάνω δεξιά γωνία του.
 - Περιέχει προαιρετική πληροφορία σε ελεύθερη μορφή με σκοπό τη διευκόλυνση της κατανόησης του διαγράμματος.
 - Συνδέεται με τα υπόλοιπα στοιχεία ενός διαγράμματος με διακεκομμένη γραμμή.

Παράδειγμα: Ένα διάγραμμα που παριστάνει τους πελάτες μίας τράπεζας και τους λογαριασμούς που αυτοί έχουν.



- Βασικά χαρακτηριστικά ενός αντικειμένου:
 - Ένα αντικείμενο είναι συγκεκριμένο **στιγμιότυπο**(instance) μίας κλάσης.
 - Έχει μία συγκεκριμένη **κατάσταση**(state)
 - ⊕ Αναπαριστάνεται από τις τιμές των ιδιοτήτων του.
 - Κατέχει χώρο στην κύρια μνήμη και επομένως έχει μία μοναδική **ταυτότητα**(identity).
 - ⊕ 2 αντικείμενα είναι ξεχωριστά έστω και αν έχουν την ίδια ταυτότητα.

- Έχει μία συμπεριφορά (behaviour) που προσδιορίζεται από τις μεθόδους της κλάσης στην οποία ανήκει.



Άρα

- Βασικά χαρακτηριστικά ενός αντικειμένου= Κατάσταση + Ταυτότητα + Συμπεριφορά

⊕ Κατάσταση Αντικειμένου

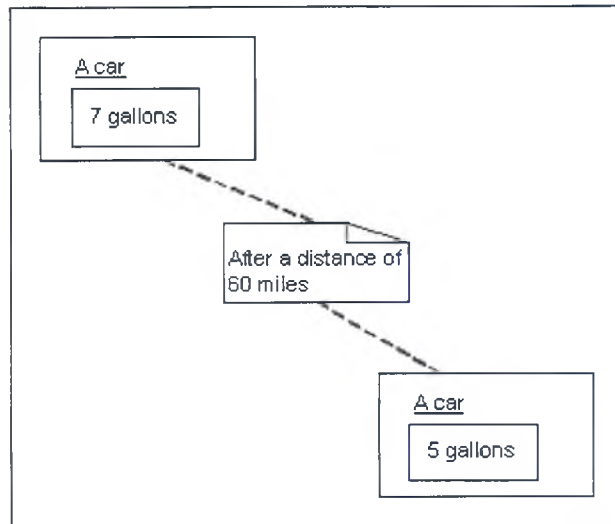
- * Αποτελείται από τις τιμές όλων των ιδιοτήτων ενός αντικειμένου μια συγκεκριμένη χρονική στιγμή.

Παράδειγμα: Ένα αυτοκίνητο με τιμές για το χρώμα, τη μάζα και την ιπποδύναμη του.



- * Η κατάσταση αλλάζει με το χρόνο
- * Μερικές ιδιότητες μπορεί να έχουν σταθερές τιμές.
- * Συνήθως η κατάσταση ενός αντικειμένου είναι μεταβλητή και μπορεί να θεωρεί ως αποτέλεσμα της συμπεριφοράς του στο πρόσφατο παρελθόν.

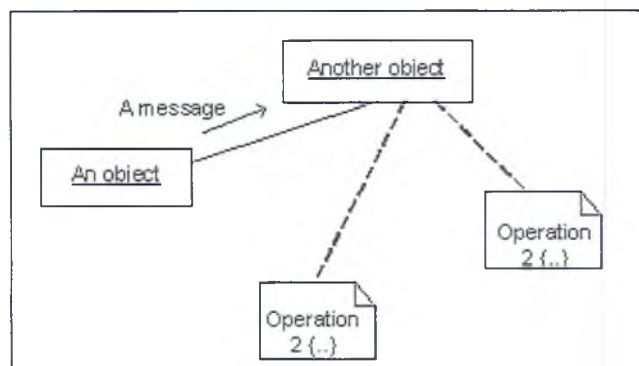
Παράδειγμα: Ένα αυτοκίνητο χρησιμοποιείται για 60 μίλια. Καθώς κινείται η τιμή που παριστάνει την ποσότητα των καυσίμων ελαττώνεται.



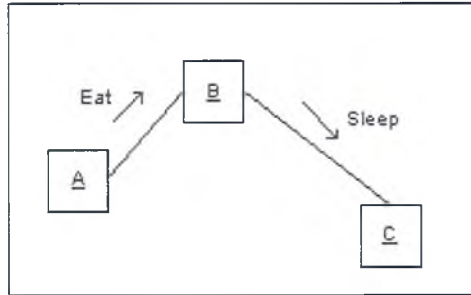
✦ Συμπεριφορά Αντικειμένου

- * Περιλαμβάνει όλες τις ικανότητες ενός αντικειμένου.
- * Περιγράφει τις ενέργειες ενός αντικειμένου και τον τρόπο που αυτό αντιδρά.
- * Αποτελείται από τις **μεθόδους**(methods) ενός αντικειμένου.
- * Μια μέθοδος εκτελείται όταν ένα αντικείμενο λάβει ένα κατάλληλο **μήνυμα**(message) από κάποιο άλλο αντικείμενο.

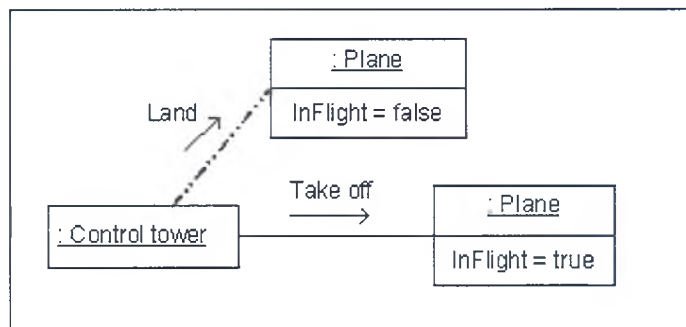
Παράδειγμα: Ανάλογα με το είδος του μηνύματος ενεργοποιείται η μέθοδος 1 ή 2.



- * Οι αλληλεπιδράσεις μεταξύ αντικειμένων παριστάνονται με τη χρήση διαγραμμάτων στα οποία τα αντικείμενα που αλληλεπιδρούν ενώνονται με τη χρήση γραμμών που ονομάζονται **δεσμοί**(links).
- * Η ύπαρξη ενός δεσμού ανάμεσα σε δύο αντικείμενα φανερώνει ότι το ένα γνωρίζει ή χρησιμοποιεί το άλλο.
- * Πάνω στους δεσμούς απεικονίζονται τα αντίστοιχα μηνύματα.

Παράδειγμα:

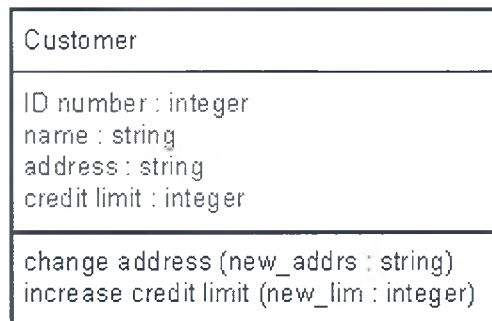
- * Υπάρχει σχέση ανάμεσα στην κατάσταση ενός αντικειμένου και στη συμπεριφορά του.
- * Μια συγκεκριμένη χρονική στιγμή η συμπεριφορά εξαρτάται από την τρέχουσα κατάσταση και η κατάσταση μπορεί να αλλάξει από τη συμπεριφορά.

Παράδειγμα:+ Ταυτότητα Αντικειμένου

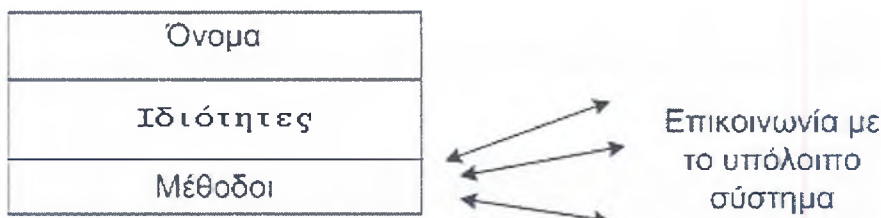
- * Χαρακτηρίζει την ύπαρξη ενός αντικειμένου.
- * Επιτρέπει τη διάκριση των αντικειμένων ανεξάρτητα από την κατάσταση τους. π.χ. Διάκριση δύο αντικειμένων που όλα οι ιδιότητες τους έχουν την ίδια τιμή.
- * Δεν παριστάνεται με τρόπο σαφή κατά τη μοντελοποίηση ενός αντικειμένου.
- * Κατά την υλοποίηση του αντικειμένου χρησιμοποιείται ένα αναγνωριστικό για την απεικόνιση της ταυτότητας του αντικειμένου.

Κλάσεις

- Κάθε κλάση παριστάνεται με ένα ορθογώνιο και είναι ένα περίγραμμα/ μία φόρμα(template) για τον προσδιορισμό των ιδιοτήτων και των μεθόδων ενός συνόλου από αντικείμενα.
- Μία κλάση προσδιορίζει:
 - ◆ Το όνομα της κλάσης
 - ◆ Το όνομα(και τον τύπο) κάθε ιδιότητας
 - ◆ Το όνομα(και τις παραμέτρους) κάθε μεθόδου



- Βασικά χαρακτηριστικά μίας κλάσης:
 - ◆ Προσδιορίζει τη γενική μορφή ενός συνόλου αντικειμένων - Διαφορετικά στιγμιότυπα(instances) της κλάσης είναι δυνατό να κατασκευασθούν από το γενικό περίγραμμα.
 - ◆ Οι μέθοδοι χαρακτηρίζουν την κλάση και αποτελούν τη διεπαφή(interface) ανάμεσα στην κλάση και στο υπόλοιπο σύστημα.



- Τα ονόματα των ιδιοτήτων και μεθόδων της κλάσης μπορούν να έχουν ένα πρόθεμα ανάλογα με την ορατότητά τους:
 - ◆ + Δημόσιο(public)
 - ◆ - Ιδιωτικό(private)
 - ◆ # Προστατευόμενο(protected)

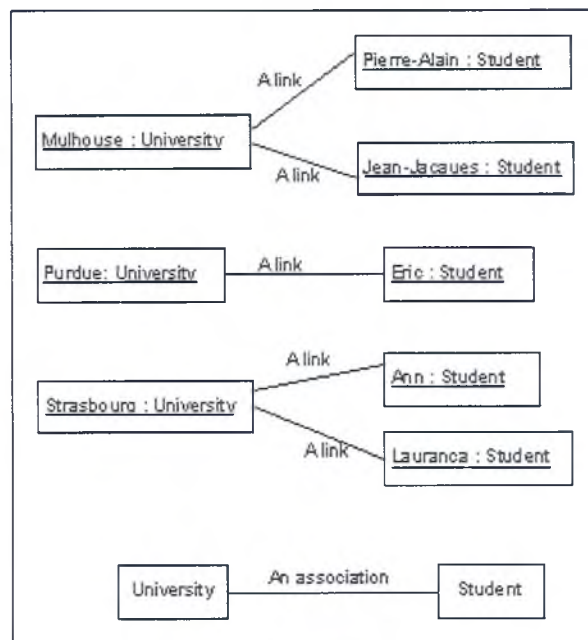
Σχέσεις Ανάμεσα σε Κλάσεις

- ➔ Οι δεσμοί που συνδέουν αντικείμενα εμφανίζονται και κατά την απεικόνιση των αντίστοιχων κλάσεων.
- ➔ Για κάθε οικογένεια δεσμών ανάμεσα σε αντικείμενα υπάρχει μια σχέση(relationship) ανάμεσα στις αντίστοιχες κλάσεις στις οποίες ανήκουν τα αντικείμενα.
- ➔ Όπως κάθε αντικείμενο είναι στιγμιότυπο μιας κλάσης, έτσι και κάθε δεσμός ανάμεσα σε αντικείμενα είναι στιγμιότυπο της σχέσης ανάμεσα στις αντίστοιχες κλάσεις.
- ➔ Βασικές σχέσεις ανάμεσα σε κλάσεις:
 - ▶ Συσχέτιση(association).
 - ▶ Συσσωμάτωση(aggregation).

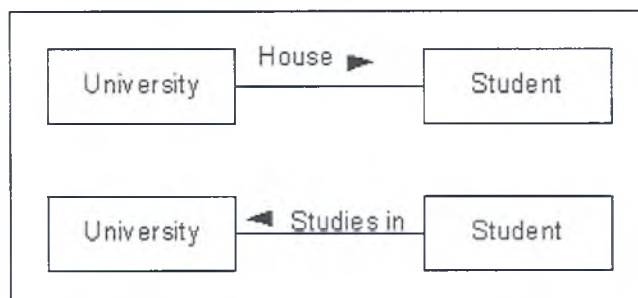
Συσχέτιση

- Η *συσχέτιση* εκφράζει μια σημασιολογική σύνδεση δύο κατευθύνσεων ανάμεσα σε κλάσεις.
- Οι συσχετίσεις απεικονίζονται όπως και οι δεσμοί ανάμεσα σε αντικείμενα.
- Συσχετίσεις και δεσμοί διακρίνονται από τα υπόλοιπα στοιχεία του διαγράμματος που τα περιέχει.

Παράδειγμα : Οι δεσμοί ανάμεσα σε πανεπιστήμια και φοιτητές είναι στιγμιότυπα της συσχέτισης ανάμεσα στις κλάσεις "Πανεπιστήμιο" και "Φοιτητής".

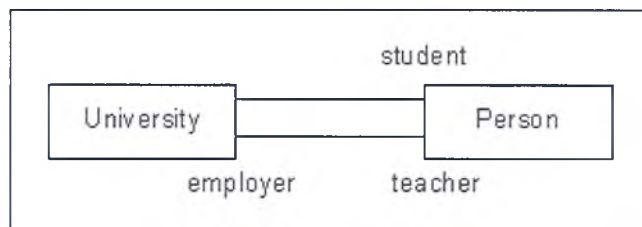


- Για να αυξηθεί η αναγνωσιμότητα των διαγραμμάτων η συσχέτιση χαρακτηρίζεται με κάποια κατάλληλη ετικέτα.
- Η φορά ανάγνωσης προσδιορίζεται πάνω στο διάγραμμα χρησιμοποιώντας κατάλληλα σύμβολα.



- Πολλές φορές προσδιορίζεται ο ρόλος μιας κλάσης σε μια συσχέτιση.
- Το όνομα ενός ρόλου μπορεί να εμφανίζεται και στα δύο μέρη μιας συσχέτισης.

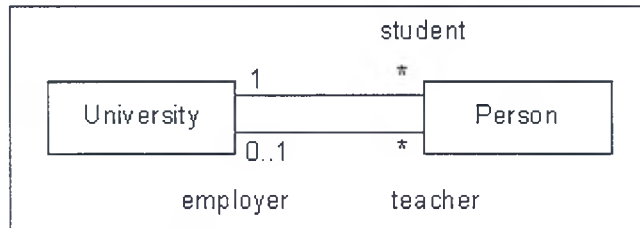
Παράδειγμα:



- Δύο συσχετίσεις ανάμεσα στις κλάσεις "University" και "Person".
 - Μερικοί άνθρωποι ενεργούν ως φοιτητές και άλλοι ως καθηγητές.
 - Το πανεπιστήμιο ενεργεί ως εργοδότης για τους καθηγητές του.
- Ένας ρόλος πολλές φορές προσδιορίζει τον αριθμό των στιγμιότυπων της κλάσης που συμμετέχουν στη σχέση (πολλαπλότητα της κλάσης):

1	Ένα και μόνο ένα
0..1	Κανένα ή ένα
M..N	Από M έως N (M,N φυσικοί αριθμοί)
*	Από 0 μέχρι οποιοδήποτε θετικό ακέραιο αριθμό
0..*	Από 0 μέχρι οποιοδήποτε θετικό ακέραιο αριθμό
1..*	Από ένα μέχρι οποιοδήποτε θετικό ακέραιο αριθμό

- Είναι δυνατό να χρησιμοποιηθούν και πολύπλοκες παραστάσεις.

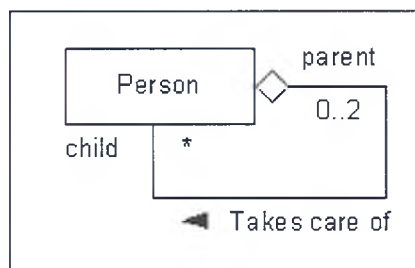
Παράδειγμα: Συσχετίσεις με Ρόλους

- Το πιο πάνω διάγραμμα μπορεί να ερμηνευθεί ως εξής:
- Ένα συγκεκριμένο πανεπιστήμιο περιέχει πολλούς ανθρώπους, άλλοι είναι φοιτητές, άλλοι καθηγητές.
 - Ένας συγκεκριμένος φοιτητής φοιτά σε ένα μόνο πανεπιστήμιο.
 - Ένας συγκεκριμένος καθηγητής μπορεί να δουλεύει στο πανεπιστήμιο μόνο κάποια συγκεκριμένη χρονική περίοδο ή διαρκώς.

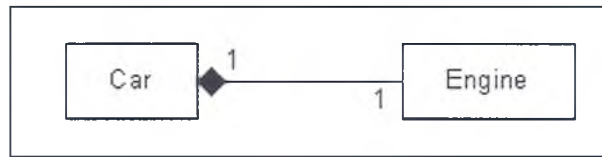
Συσσωμάτωση

- ➔ Η *συσσωμάτωση* είναι ένα είδος συσχέτισης που εκφράζει μια ισχυρότερη σύζευξη ανάμεσα στις κλάσεις=> μία από τις κλάσεις έχει έναν πιο σημαντικό ρόλο.
- ➔ Μέσω της συσσωμάτωσης μπορούν να απεικονιστούν σχέσεις της μορφής:
 - ➔ Αφέντη και σκλάβου (master and slave).
 - ➔ Μέρος ενός όλου (part of).
 - ➔ Αποτελείται από συνιστώσες (composed of).
- ➔ Η συσσωμάτωση εκφράζει μια ασύμμετρη σχέση δύο κατευθύνσεων ανάμεσα σε κλάσεις.

Παράδειγμα: Ένα άτομο που μπορεί να προσέχει πολλά παιδιά.



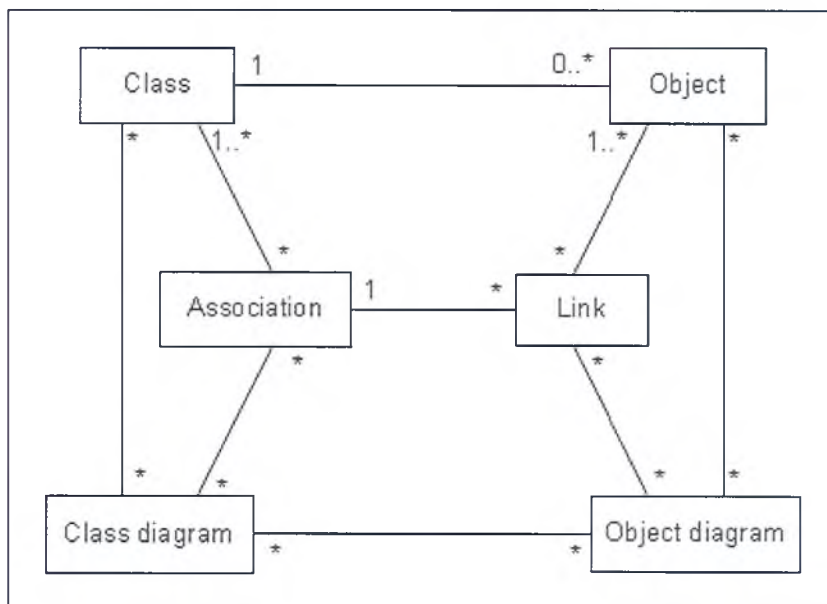
- ➔ Κάποια άτομα ενεργούν ως γονείς και κάποια σαν παιδιά.
- ➔ Η σχέση είναι εννοιολογικά ασύμμετρη- ο ενήλικας προσέχει τα παιδιά.
- ➔ Η συσσωμάτωση παριστάνεται όπως η συσχέτιση με την προσθήκη ενός μικρού διαμαντιού στην πλευρά της σύνθετης κλάσης.
- ➔ Η συσσωμάτωση επιτρέπει τη διάδοση των τιμών των ιδιοτήτων και των μεθόδων της σύνθετης κλάσης στις συνιστώσες της.
- ➔ Η UML χρησιμοποιεί μια ισχυρότερη μορφή συσσωμάτωσης που ονομάζεται σύνθεση (composition) όταν:
 - ➔ Η πολλαπλότητα της σύνθετης κλάσης είναι 1.
 - ➔ Η διαγραφή της σύνθετης κλάσης έχει ως αποτέλεσμα και τη διαγραφή των συνιστωσών της.
- ➔ Η *σύνθεση* απεικονίζεται όπως η συσσώρευση με τη διαφορά ότι το διαμάντι είναι γεμάτο.

Παράδειγμα:

- Κάθε αυτοκίνητο έχει μια μηχανή που δεν μπορεί να μοιραστεί με άλλα αυτοκίνητα.
- Η διαγραφή του αυτοκινήτου έχει ως αποτέλεσμα και τη διαγραφή της μηχανής.

Σύγκριση Διαγραμμάτων Κλάσης & Αντικειμένου

- Τα διαγράμματα κλάσης και τα διαγράμματα αντικειμένου αποτελούν δύο συμπληρωματικές όψεις ενός μοντέλου.
- Ένα **διάγραμμα κλάσης**:
 - Παριστάνει με αφαιρετικό τρόπο την πραγματικότητα.
 - Εστιάζει στην περιγραφή της στατικής δομής.
- Ένα **διάγραμμα αντικειμένου**:
 - Παριστάνει μια συγκεκριμένη περίπτωση - μια κατάσταση που εμφανίζεται μία συγκεκριμένη χρονική στιγμή.
 - Εκφράζει τόσο τη στατική δομή, όσο και τη συμπεριφορά.



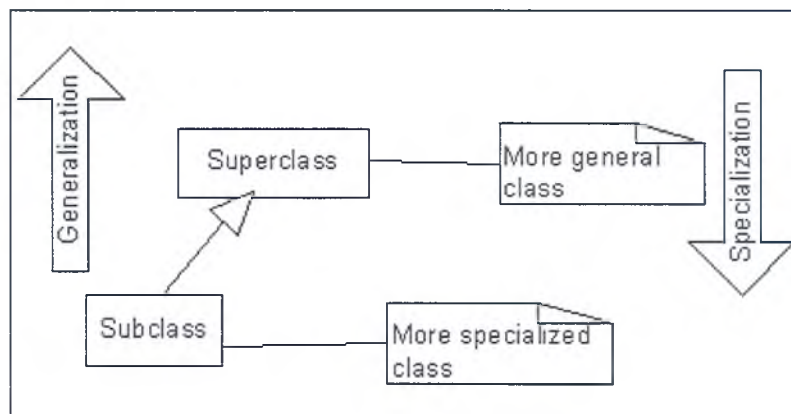
- Η **μετάβαση** από τον ένα τύπο διαγράμματος στον άλλο καθορίζεται από τους εξής κανόνες:
 - Κάθε αντικείμενο είναι στιγμιότυπο μιας κλάσης και η κλάση αυτή δεν αλλάζει σε όλη τη διάρκεια ζωής του αντικειμένου.
 - Μερικές κλάσεις (αφηρημένες κλάσεις) δεν μπορούν να έχουν στιγμιότυπα.
 - Κάθε δεσμός είναι το στιγμιότυπο μιας συσχέτισης.
 - Οι δεσμοί συνδέουν αντικείμενα, οι συσχετίσεις συνδέουν κλάσεις.
 - Ένας δεσμός ανάμεσα σε δύο αντικείμενα υποδηλώνει:

- Μια συσχέτιση ανάμεσα στις κλάσεις των δύο αντικειμένων.
- Ότι τα δύο αντικείμενα μπορούν να ανταλλάσσουν μηνύματα.
- Τα διαγράμματα αντικειμένου που περιέχουν αντικείμενα και δεσμούς είναι στιγμιότυπα των διαγραμμάτων κλάσης που περιέχουν κλάσης και συσχετίσεις.
- Η αντικειμενοστραφής ανάπτυξη λογισμικού δεν είναι μια γραμμική διαδικασία.
- Στην πράξη ποτέ δε φτιάχνεται αρχικά ο ένας τύπος διαγράμματος και στη συνέχεια παράγεται ο άλλος από τον πρώτο.
- Τα διαγράμματα κλάσης και αντικειμένου συνήθως φτιάχνονται παράλληλα:
 - Κάποιες φορές τα αντικείμενα προσδιορίζονται πρώτα και στη συνέχεια δημιουργούνται οι κλάσεις- Ο πραγματικός κόσμος περιέχει αντικείμενα και όχι κλάσεις.
 - Κάποιες φορές η δομή των κλάσεων είναι προφανής.
 - Δεν υπάρχει γενικός κανόνας.

Γενίκευση και Ειδίκευση

- Είναι δύο διαφορετικές οπτικές γωνίες της έννοιας της ταξινόμησης.
- Εκφράζουν την κατεύθυνση κατά την οποία επεκτείνεται μια ιεραρχία κλάσεων.
- Συνήθως υλοποιούνται συγχρόνως:
 - ⊕ Η γενίκευση εφαρμόζεται όταν έχουν βρεθεί τα στοιχεία της λύσης, για να αποκτηθεί μια γενική / αφηρημένη περιγραφή της λύσης.
 - ⊕ Η ειδίκευση αποτελεί τη βάση της επαναχρησιμοποίησης - οι νέες απαιτήσεις ενσωματώνονται σε υποκλάσεις που επεκτείνουν την ήδη υπάρχουσα λειτουργικότητα με τρόπο ομαλό.
- Η δημιουργία υπερκλάσεων απαιτεί αφαιρετική ικανότητα, ενώ η δημιουργία υποκλάσεων απαιτεί βαθιά γνώση και πείρα σε έναν τομέα.
- **Παράδοξο:**
 - ✗ Είναι δύσκολη η εύρεση υπερκλάσεων, αλλά τα προγράμματα που τις χρησιμοποιούν είναι εύκολο να γραφούν.
 - ✗ Είναι εύκολη η εύρεση υποκλάσεων, αλλά δύσκολη η υλοποίησή τους.

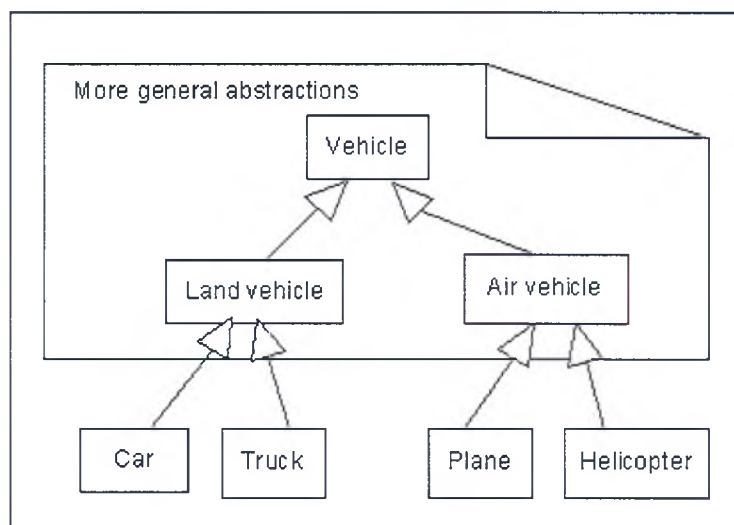
Παράδειγμα:



Γενίκευση

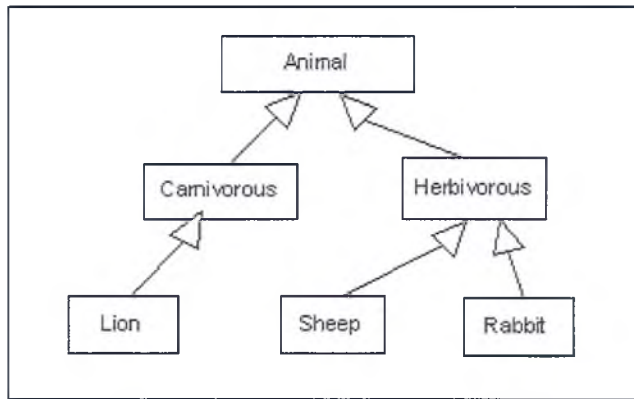
- Η γενίκευση πραγματοποιείται με τον εντοπισμό κοινών στοιχείων (ιδιότητες, μέθοδοι) σε ένα σύνολο από κλάσεις και την τοποθέτησή τους σε μια πιο γενική κλάση (**υπερκλάση**).
- Δημιουργείται μια ιεραρχία κλάσεων.
- Μια υπερκλάση αποτελεί μια αφηρημένη εικόνα των υποκλάσεών της.
- Η γενίκευση είναι μια δύσκολη διαδικασία που πραγματοποιείται με τρόπο **επαναληπτικό**.
- Τα δέντρα κλάσεων δεν αναπτύσσονται από τις ρίζες τους, αλλά από τα φύλλα τους, αφού αυτά συναντώνται στον πραγματικό κόσμο.
- Η γενίκευση παριστάνεται χρησιμοποιώντας βέλη που δείχνουν στην πιο γενική κλάση.

Παράδειγμα: Η ιεραρχία των μέσων μεταφοράς.

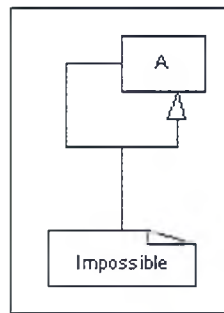


Χαρακτηριστικά:

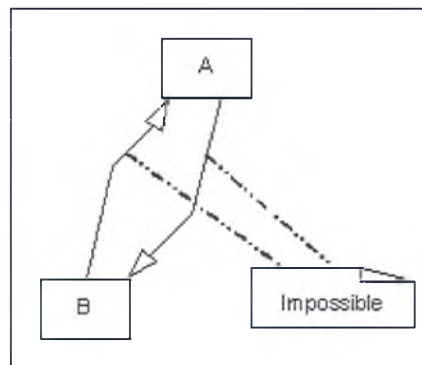
- Η γενίκευση έχει τη σημασία "είναι ένα από" (is one of) ή "είναι ένα είδος" (is a kind of).
- Αφορά μόνο κλάσεις - δεν υπάρχει η έννοια της πολλαπλότητας.



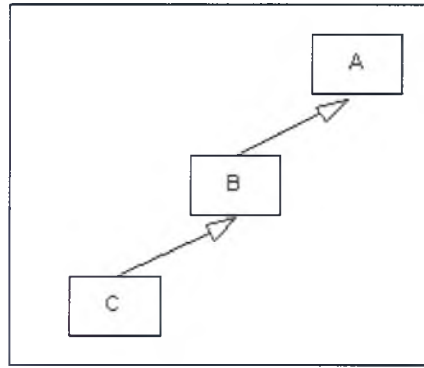
- Η γενίκευση είναι μια μη ανακλαστική σχέση - μια κλάση δεν μπορεί να παραχθεί από τον εαυτό της.



- Η γενίκευση είναι μια ασύμμετρη σχέση - αν η κλάση B παράγεται από την κλάση A, τότε η κλάση A δεν μπορεί να παραχθεί από την κλάση B.



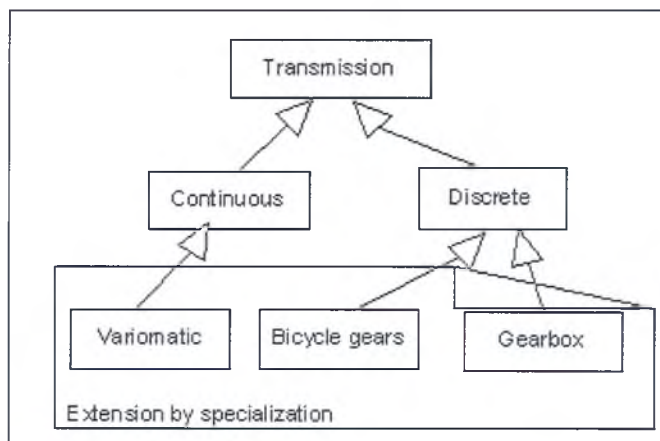
- Η γενίκευση είναι μια μεταβατική σχέση - αν η κλάση Γ παράγεται από την κλάση B, η οποία παράγεται από την A, τότε η κλάση Γ παράγεται από την κλάση A.



Ειδίκευση

- Η ειδίκευση επιτρέπει τη συγκέντρωση ειδικών χαρακτηριστικών από ένα σύνολο αντικειμένων που δεν εμφανίζονται στις κλάσεις που έχουν ήδη οριστεί.
- Τα νέα χαρακτηριστικά παριστάνονται από μια νέα κλάση, που είναι υποκλάση των ήδη υπάρχοντων κλάσεων.
- Η ειδίκευση επιτρέπει την επέκταση ενός συνόλου από κλάσεις με τρόπο λογικά συνεπή.

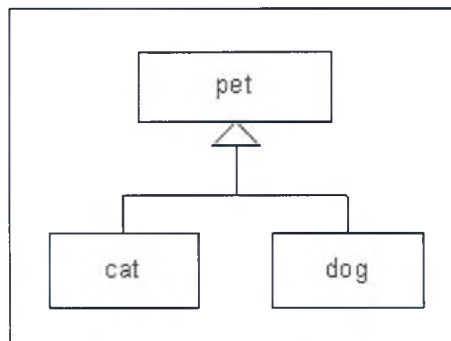
Παράδειγμα: Ταξινόμηση εξοπλισμού μετάδοσης της κίνησης σε οχήματα



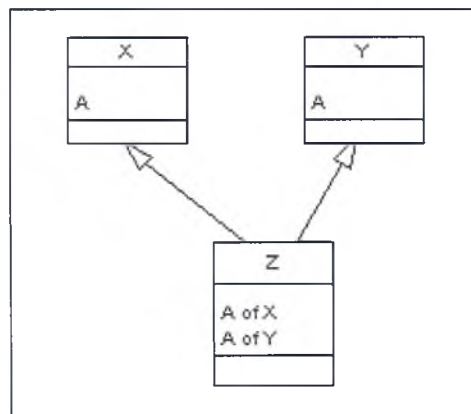
- Συγκεκριμένοι μηχανισμοί προστίθενται στην ιεραρχία με επέκταση της πιο κοντινής κλάσης πατέρα.

Κληρονομικότητα

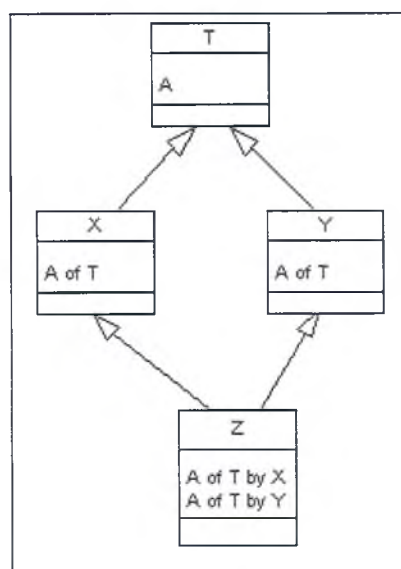
- Απεικονίζεται με ένα *βέλος τριγωνικής μορφής*.
- Χρησιμοποιούνται οι ακόλουθοι όροι:
 - *Κλάση βάσης*(Base class) και *παραγόμενη κλάση*(Derived class).
 - *Υπερκλάση*(super class) και *υποκλάση*(sub class).
 - *Πατρική κλάση*(parent class) και *κλάση παιδί*(child class).



- Η κλάση B μπορεί να είναι υποκλάση της κλάσης A, αλλά υπερκλάση της κλάσης Γ.
- Το τριγωνικό βέλος που χρησιμοποιείται για την απεικόνιση της κληρονομικότητας δείχνει προς την υπερκλάση.
- Μέσω της κληρονομικότητας οι γλώσσες προγραμματισμού υλοποιούν δομές ταξινόμησης.
- Βασικά χαρακτηριστικά:
 - Υπάρχει μια ιεραρχία κλάσεων.
 - Οι κλάσεις παιδιά κληρονομούν τις ιδιότητες και τις μεθόδους της κλάσης πατέρας.
 - Υπάρχουν διάφορα επίπεδα ορατότητας (απεικονίζονται στη UML).
- Πολλές φορές η κληρονομικότητα και η αυτόματη διάδοση των χαρακτηριστικών μιας κλάσης σε μια άλλη χρησιμοποιούνται καταχρηστικά για την κατασκευή σύνθετων αντικειμένων.
- Θα πρέπει να δηλώνει ο προγραμματιστής όταν χρησιμοποιεί κληρονομικότητα για πιο κόπο τη χρησιμοποιεί.
- Πιθανή εμφάνιση προβλημάτων σύγκρουσης ονομάτων.

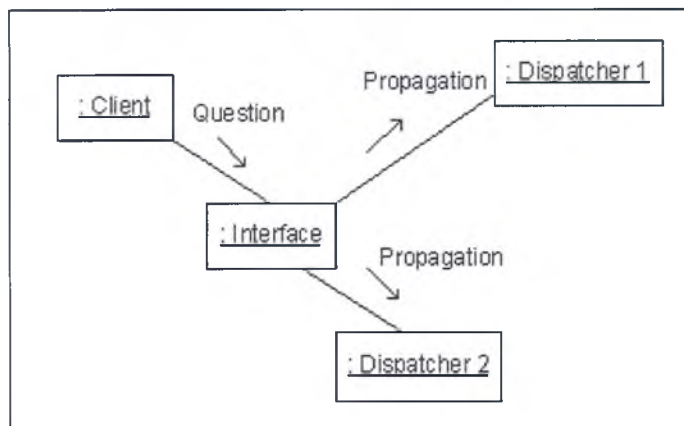
Παράδειγμα 1:

- Και οι δύο υπερκλάσεις (X και Y) έχουν μια ιδιότητα A.
- Η κλάση Z έχει δύο ιδιότητες A.
- Αν το A παριστάνει την ίδια έννοια στις κλάσεις X και Y δε χρειάζεται η Z να έχει δύο ιδιότητες A.
- Σε αντίθετη περίπτωση σε μία από τις X ή Y το A θα πρέπει να αλλάξει όνομα.
- Αντιμετωπίζεται με διαφορετικό τρόπο από κάθε γλώσσα προγραμματισμού.

Παράδειγμα 2:

- Η Ζ δεν χρειάζεται να έχει δύο ιδιότητες.
- Απαιτείται ιδιαίτερη προσοχή όταν χρησιμοποιείται πολλαπλή κληρονομικότητα για τη συνένωση δύο κλάσεων που έχουν δημιουργηθεί ανεξάρτητα ή μία από την άλλη.
- Η Java δεν υποστηρίζει πολλαπλή κληρονομικότητα.
- Από την άλλη η *Αντιπροσωπεία*:
 - ▶ Αντικαθιστά πολλές φορές την κληρονομικότητα.
 - ▶ Έχει το πλεονέκτημα ότι ελαττώνει τη σύζευξη σε ένα μοντέλο.
 - ▶ Ένα αντικείμενο πελάτης δε γνωρίζει ποιο αντικείμενο τον εξυπηρετεί.
 - ▶ Το αντικείμενο εξυπηρετητής μπορεί να αλλάζει ανάλογα με την περίπτωση.
 - ▶ Επιτρέπει την υλοποίηση πολλαπλής κληρονομικότητας από γλώσσες προγραμματισμού που υποστηρίζουν μόνο απλή κληρονομικότητα.

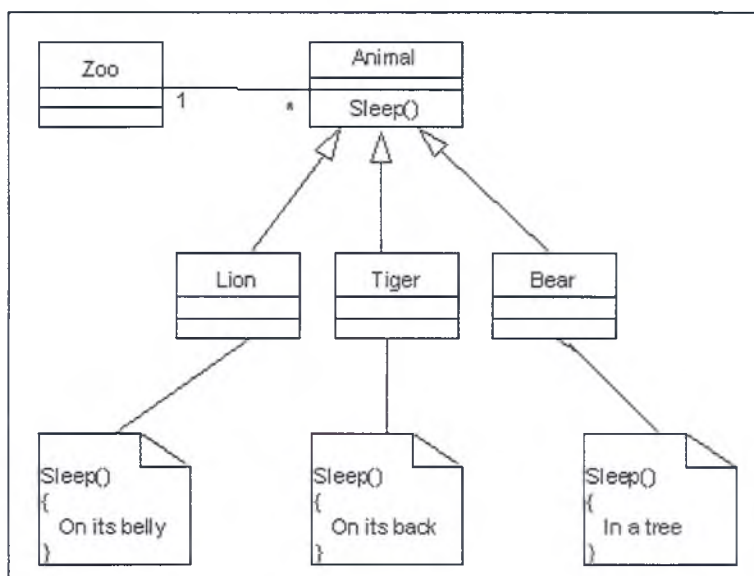
Παράδειγμα: Ο πελάτης επικοινωνεί με μια διεπαφή που προωθεί τις ερωτήσεις του στον κατάλληλο κάθε φορά εξυπηρετητή.



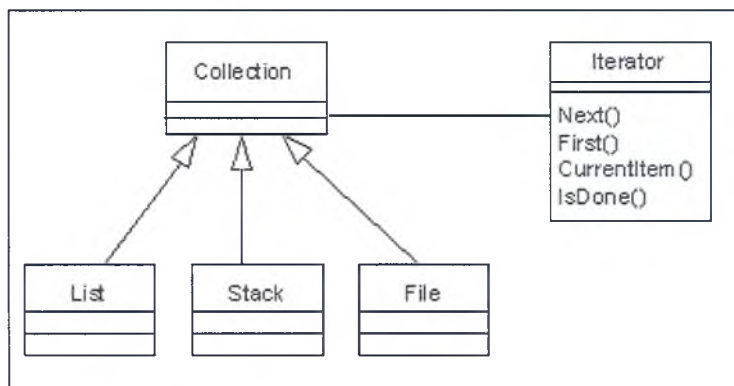
Πολυμορφισμός

- Πολυμορφισμός (polymorphism) ή υπερφόρτωση (overloading) είναι η ιδιότητα που επιτρέπει το ίδιο μήνυμα να σταλεί σε διαφορετικά αντικείμενα και το κάθε αντικείμενο να εκτελέσει μια λειτουργία που αντιστοιχεί στην κλάση. Η μόνη διαφορά τους έγκειται στο γεγονός ότι ο πολυμορφισμός χρησιμοποιεί δυναμική διασύνδεση ενώ η υπερφόρτωση χρησιμοποιεί στατική διασύνδεση.
- Το αντικείμενο που στέλνει το μήνυμα δε χρειάζεται να γνωρίζει την κλάση του αντικειμένου παραλήπτη ή πως το τελευταίο θα αποκριθεί στο μήνυμα
- Ένα όνομα αντικειμένου μπορεί να προσδιορίζει στιγμιότυπα από διαφορετικές κλάσεις.
- **Πολυμορφισμός μεθόδου:** Η ικανότητα να εκτελούνται διαφορετικές μέθοδοι με τη λήψη του ίδιου μηνύματος.
- Κάθε υποκλάση κληρονομεί τις μεθόδους της υπερκλάσης της, αλλά μπορεί να αλλάξει τον τρόπο που αυτές υλοποιούνται.
- Το κύριο όφελος του πολυμορφισμού είναι η μεγαλύτερη αναγνωσιμότητα και η ευκολότερη συντήρηση του κώδικα.
- Ο πολυμορφισμός εμφανίζεται ως έμμεσο αποτέλεσμα της διαδικασίας της αφαίρεσης.

Παράδειγμα: Ένας ζωολογικός κήπος που περιέχει πολλά διαφορετικά ζώα.



- ◆ Οι υποκλάσεις υλοποιούν τη μέθοδο **Sleep()** ανάλογα με το κάθε ζώο.
 - ◆ Στην πράξη γίνεται χρήση της κλάσης **Zoo** και με την βοήθεια ενός επαναλήπτη(iterator) στέλνεται το μήνυμα **Sleep** σε κάθε ζώο.
- Ένας επαναλήπτης έχει τις εξής μεθόδους:
- ◆ **First()**: Μετακινεί τον επαναλήπτη στο πρώτο από τα στοιχεία της συλλογής.
 - ◆ **Next()**: Επιτρέπει τη μετακίνηση στο επόμενο στοιχείο.
 - ◆ **CurrentItem()**: Επιστρέφει το τρέχων στοιχείο.
 - ◆ **IsDone()**: Γίνεται **true** όλα τα στοιχεία έχουν επισκεφθεί.



Visit: Iterator;

AnAnimal: Animal; --polymorphic variable

...

Visit.First(theZoo);

While not Visit.IsDone()

loop

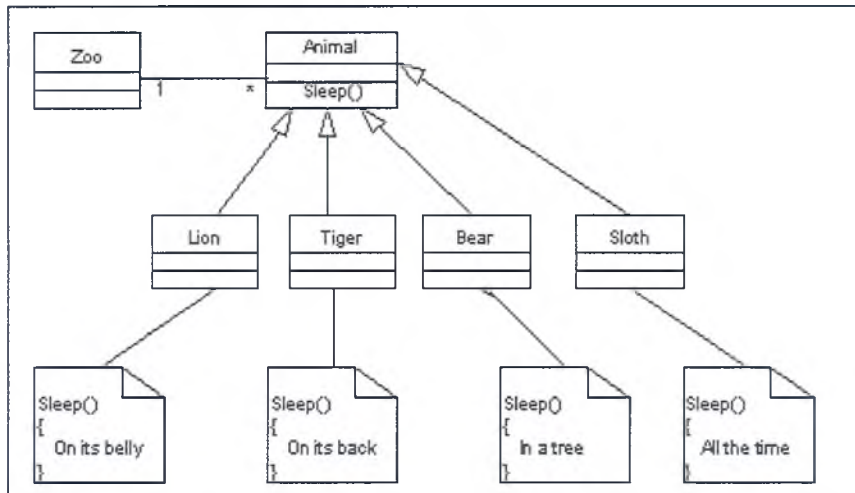
 AnAnimal := Visit.CurrentItem();

 AnAnimal.Sleep();

 Visit.Next();

end loop;

- Η μεταβλητή **AnAnimal** είναι πολυμορφική- μπορεί να περιέχει οποιοδήποτε από τα ζώα που επιστρέφονται από τη **Visit.CurrentItem()**.
- Αν έρθουν νέα ζώα δε χρειάζεται να αλλάξει ο κώδικας των ήδη υπαρχόντων ζώων.
- Προστίθεται μια νέα υποκλάση της **Animal** που υλοποιεί με το δικό της τρόπο τη **Sleep()**.



- ✱ Η αυτόματη επιλογή του σωστού κώδικα είναι αποτέλεσμα της τεχνικής της **δυναμικής δέσμευσης**.
- ✱ Η κλήση μιας μεθόδου πραγματοποιείται κατά το χρόνο εκτέλεσης του προγράμματος.
- ✱ Υλοποίηση του παραδείγματος χωρίς δυναμική δέσμευση:
 - ◆ Εισάγονται πολλά σημεία συντήρησης στον κώδικα.
 - ◆ Γίνεται αναγκαία η χρήση εντολών πολλαπλής διακλάδωσης.
 - ◆ Αυξάνεται η πολυπλοκότητα του κώδικα.

Visit: Iterator;

AnAnimal: AnAnimal; --polymorphic variable

...

Visit.First(theZoo);

while not Visit.IsDone()

loop

AnAnimal := Visit.CurrentItem();

case AnAnimal.Class()

When Lion

-- Sleep on its belly

When Tiger

-- Sleep on its back

When Bear

-- Sleep in a tree

When Sloth

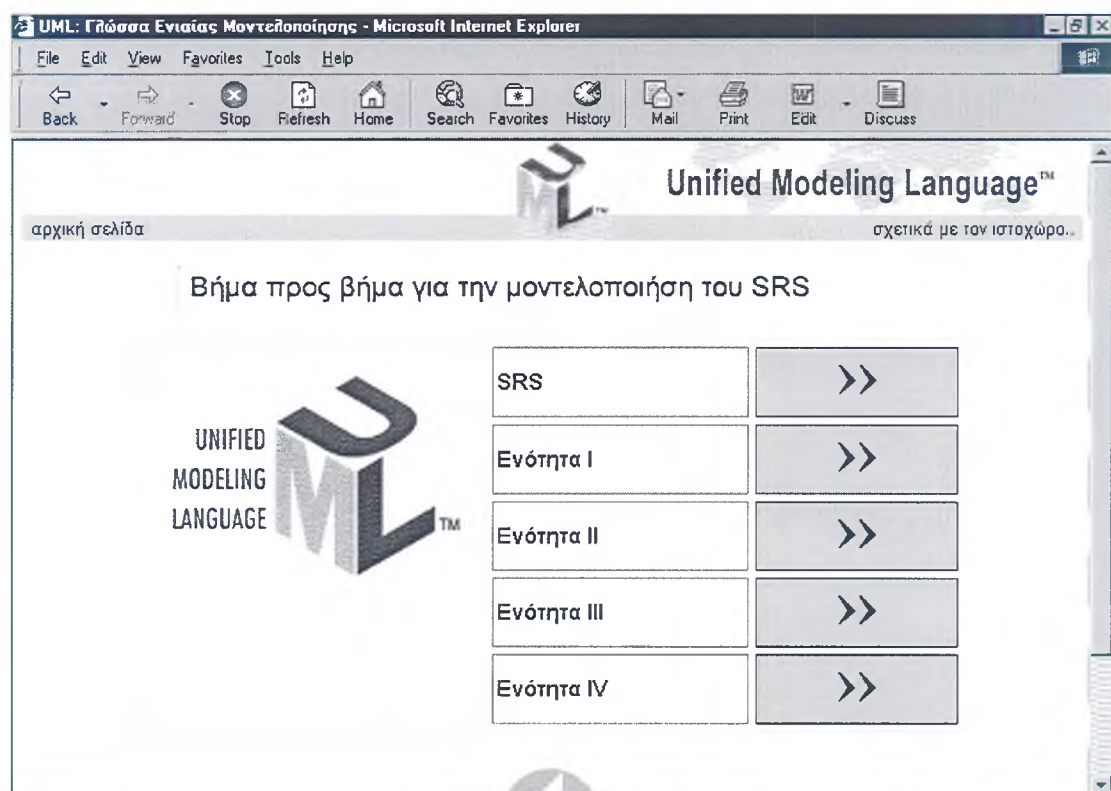
-- Sleep all the time

end case;

Visit.Next();

end loop;

Βήμα προς βήμα για την μοντελοποίηση του SRS



Σύστημα Εγγραφής Φοιτητών(SRS) - Απαιτούμενα Χαρακτηριστικά

Μας έχει ζητηθεί να αναπτύξουμε ένα αυτοματοποιημένο Σύστημα Εγγραφής Φοιτητών(SRS) για το πανεπιστήμιο(University) . Αυτό το σύστημα θα επιτρέπει στους φοιτητές(Students) να εγγράφονται (on-line) όταν είναι συνδεδεμένοι με το σύστημα σε διάφορους κλάδους(Course) κάθε διδακτικής περιόδου(semester) και συνάμα να προσδιορίζει την πρόοδο τους μέχρι την συμπλήρωση του διπλώματος τους.

Όταν ένας φοιτητής(Student) εγγράφεται αρχικά στο πανεπιστήμιο(University), αυτός/ αυτή χρησιμοποιεί το SRS για να σχεδιάσει το πρόγραμμα σπουδών(Plan of Study) του/ της σχετικά με τα μαθήματα(Course) που προγραμματίζει να ακολουθήσει για να ικανοποιήσει ένα συγκεκριμένο πρόγραμμα σπουδών προς εξασφάλιση διπλώματος, επιλέγοντας παράλληλα και ένα σύμβουλο καθηγητή (Faculty advisor). Το SRS θα επιβεβαιώσει κατά πόσο το προτεινόμενο σχέδιο σπουδών(Plan of Study) ικανοποιεί τις απαιτήσεις του διπλώματος(requirement of the degree) που προσδοκεί ο φοιτητής(Student).

Εφόσον επιλεχθεί ένα πρόγραμμα σπουδών(Plan of Study), τότε, κατά την περίοδο εγγραφών(registration period) που προηγείται κάθε διδακτικής περιόδου(semester), οι φοιτητές(Students) έχουν την δυνατότητα να βλέπουν το σχεδιάγραμμα των υπαρχόντων τμημάτων (on line) όταν είναι συνδεδεμένοι με το σύστημα και έτσι θα μπορούν να επιλέγουν οποιαδήποτε τμήματα επιθυμούν να παρακολουθήσουν, δείχνοντας το προτεινόμενο τμήμα (ημέρα εβδομάδας και ώρα της ημέρας) εάν το μάθημα αυτό προσφέρεται από περισσότερους από ένα καθηγητές(Professor). Το SRS θα επιβεβαιώσει κατά πόσον ο φοιτητής(Student) έχει ικανοποιήσει τις αναγκαίες προαπαιτήσεις για κάθε επιλεγμένο μάθημα(Course) με απλή αναφορά στο αντίγραφο(transcript) μαθημάτων (on line) του φοιτητή, καθώς και την αντίστοιχη βαθμολογία που πέτυχε (ο φοιτητής μπορεί να διαβάσει το αντίγραφο σπουδών του on line οποιαδήποτε στιγμή).

Θεωρώντας ότι :

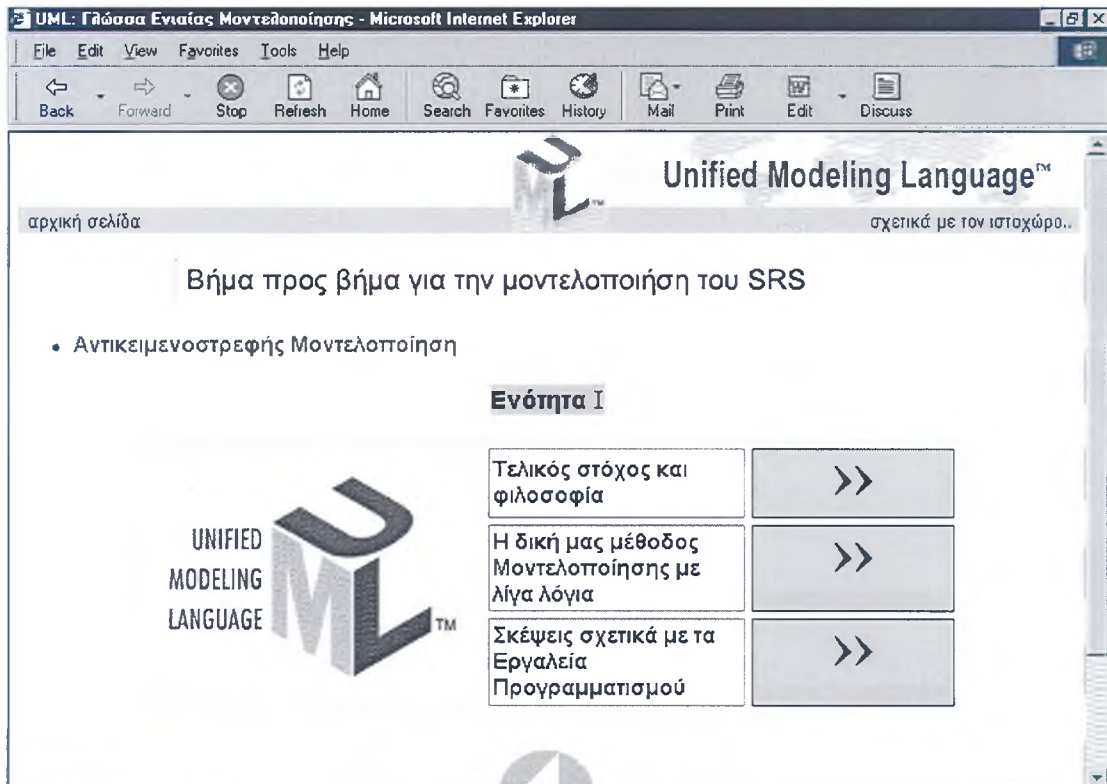
- (α) οι προαπαιτήσεις για το κάθε μάθημα ικανοποιούνται,
- (β) τα μαθήματα ανταποκρίνονται στις απαιτήσεις του σχεδίου σπουδών του φοιτητή και

- (γ) υπάρχει διαθέσιμη θέση για τα μαθήματα αυτά, τότε ο μαθητής γίνεται δεκτός και εγγράφεται στο μάθημα αυτό που επιθυμεί.

Εάν ικανοποιούνται τα (α) και (β) και δεν υπάρχει χώρος στα μαθήματα(Course) αυτά, τότε ο φοιτητής(Student) τοποθετείται σε κατάλογο αναμονής με προτεραιότητα αυτός που εγγράφηκε πρώτος, εξυπηρετείται πρώτος(first-come, first_served wait list). Εάν το συγκεκριμένο μάθημα(Section) γίνει διαθέσιμο - είτε επειδή κάποιος άλλος φοιτητής αποχώρησε, είτε επειδή η χωρητικότητα του τμήματος μεγάλωσε - ο φοιτητής ο οποίος είχε σειρά προτεραιότητας στο κατάλογο αναμονής εγγράφεται αυτόματα στο μάθημα για το οποίο περίμενε και ένα μήνυμα αποστέλλεται στον συγκεκριμένο ενδιαφερόμενο φοιτητή σχετικά με την εγγραφή του. Είναι ευθύνη του μαθητή εάν θα δεχθεί ή θα αφήσει το μάθημα αυτό, εάν δεν ενδιαφέρεται πλέον. Διαφορετικά θα χρεωθεί με το μάθημα.

Οι φοιτητές μπορούν να αφήσουν ένα μάθημα στο οποίο έχουν επιλεγεί με τον πιο πάνω τρόπο εντός της πρώτης εβδομάδας κάθε διδακτικής περιόδου που θα διδαχθεί το μάθημα αυτό.

Ενότητα Ι



Τελικός στόχος και φιλοσοφία

Ο τελικός μας στόχος στη Αντικειμενοστραφή Μοντελοποίηση είναι να αποδοθεί ένα μοντέλο, το οποίο θα είναι προσανατολισμένο προς το αντικείμενο για το σύστημα που θα αυτοματοποιηθεί.

- ✱ Για τους μελλοντικούς χρήστες το σύστημα το οποίο προτιθέμεθα να υλοποιήσουμε είναι ένα σύστημα μοντέλων το οποίο επικοινωνεί με τις ανάγκες του συστήματος.
- ✱ Ένα αντικειμενοστραφές μοντέλο επικοινωνεί με την δομή και την σχέση/ λειτουργία του προγράμματος που απαιτείται για να υλοποιηθεί ώστε να ικανοποιεί αυτές τις απαιτήσεις. Αυτό δεν ωφελεί μόνο τους προγραμματιστές αλλά ακόμα και πλήθος ατόμων που είναι υπεύθυνοι για την διαπίστωση ποιότητας, έλεγχο και αρχειοθέτηση.
- ✱ Ένα αντικειμενοστραφές μοντέλο υλοποιείται σαν ένα σχηματικό διάγραμμα που βοηθά τα εκατομμύρια πλήθη ατόμων που είναι υπεύθυνα για την στήριξη και συντήρηση μίας εφαρμογής να κατανοήσουν την δομή και την λειτουργία του.

Φυσικά, αυτό το τελευταίο σημείο είναι αληθές μόνο εάν το αντικειμενοστραφές μοντέλο αντικατοπτρίζει με ακρίβεια το σύστημα όπως πραγματικά κτίστηκε και όχι όπως αρχικά επινοήθηκε. Ο σχεδιασμός πολύπλοκων συστημάτων κατά κανόνα αλλάζει κατά την διάρκεια της κατασκευής του και έτσι θα πρέπει να λαμβάνεται πρόνοια για βελτίωση ούτως ώστε να κρατιέται το αντικειμενοστραφές μοντέλο σύγχρονο ενόσω το σύστημα κτίζεται.

Η μεθοδολογία Μοντελοποίησης περιλαμβάνει τρία στοιχεία/ συστατικά:

- Την διαδικασία: " Πώς να σχεδιαστούν οι απαιτήσεις και να καθοριστεί το θέμα προς Μοντελοποίηση "
- Τον Συμβολισμό: " Μια γραφική μέθοδος/ γλώσσα για επικοινωνία με το μοντέλο "
- Τα Μέσα/ Εργαλεία: " Ένα αυτοματοποιημένο τρόπο για την εκπλήρωση και εφαρμογή του συμβολισμού "

Μεθοδολογία Μοντελοποίησης = Διαδικασία + Συμβολισμός + Μέσα

Modeling Methodology = Process + Notation + Tool

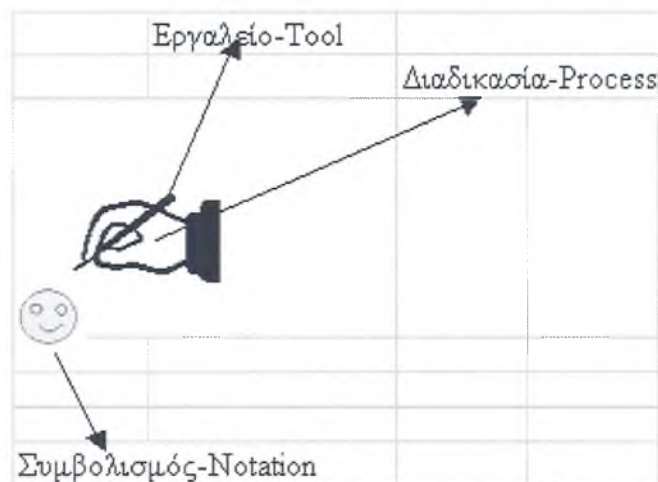
Παρά το γεγονός ότι αυτά τα τρία στοιχεία αποτελούν τα ιδανικά υλικά για την μέθοδο της Μοντελοποίησης δεν είναι όλα ίσης σημασίας. Μία καλή

διαδικασία είναι στα σίγουρα κρίσιμης σημασίας, όμως θα μπορούσαμε να βολευτούμε με ένα ωραίο κείμενο περιγραφής πολλών θεμάτων. Και όταν επιλέξουμε να χρησιμοποιήσουμε μια γραφική μέθοδο και συμβολισμό, δεν είναι υποχρεωτικό να χρησιμοποιήσουμε ένα ειδικό εργαλείο ή τρόπο ή μέσο για να την υλοποιήσουμε. Με άλλα λόγια, η διαδικασία είναι το πιο σημαντικό από όλα, ακολουθούμενο στενά από τον συμβολισμό και τα μέσα που είναι της μικρότερης σημασίας από τα τρία.

Πολλές σημαντικές προσφορές στην μορφή των νέων μεθόδων συμβολισμού και εργαλείων έχουν γίνει στην μέθοδο μοντελοποίησης ΟΟ τα τελευταία χρόνια από πολλούς γνωστούς μεθοδιστές. Κατά τέτοιο τρόπο, εάν έρχεσαι σε επαφή με το θέμα αυτό για πρώτη φορά, είσαι ευτυχής, επειδή καταφέρνει να αποφύγει τους πολέμους της μεθοδολογίας που ξέσπασαν για πολλά χρόνια, καθώς οι μεθοδιστές και οι ακόλουθοι τους διαφωνούσαν μεταξύ τους στο κατά πόσο ποίες ήταν σε κάποιες περιπτώσεις εσωτερικές λεπτομέρειες. Τα τελευταία χρόνια, υπήρξε μια μεγάλη ώθηση στην βιομηχανία στο να συγκαίρουν τις καλύτερες ιδέες από τους ανταγωνιζομένους μεθοδολογιστές, με μία προσέγγιση με ιδιαίτερη έμφαση να τοποθετείται στην προσπάθεια να δοθεί ένας παγκόσμιος συμβολισμός. Ο συμβολισμός στον οποίο κατέληξαν είναι γνωστός σαν Ενοποιημένη Γλώσσα Σχεδιασμού- Unified Modeling Language (UML) , αντιπροσωπεύει τις επίπονες προσπάθειες τριών από τους ηγέτες στον τομέα μεθοδολογισμού ΟΟ- τον James Rumba, Grady Booch και Ivar Jacobsen- και έχει γίνει το σύνηθες αντικείμενο μοντελοποίησης στη βιομηχανία.

Είναι σημαντικό να έχουμε κατά νου ότι η μεθοδολογία είναι ο τρόπος για να φτάσουμε στο τέλος - είναι το τέλος χρήσιμο και ευέλικτο έχοντας εμπιστοσύνη και διαδικαστικά ορθό πρόγραμμα μαζί με μια εξονυχιστικά καθαρή παρουσίαση υποστηρικτικών κειμένων - για το οποίο νοιαζόμαστε περισσότερο όταν όλα λέγεται ότι έχουν γίνει.

Για να βοηθήσουμε σε αυτό το σημείο ας χρησιμοποιήσουμε ένα απλό παράδειγμα. Ας υποθέσουμε ότι ο στόχος μας είναι να ευθυμήσουμε τους ανθρώπους. Αποφασίσαμε να ζωγραφίσουμε μία μέθοδος-process με ένα γελαστό πρόσωπο(μια περιγραφή της ποθούμενης συμπεριφοράς, που δόθηκε με ένα γραφικό συμβολισμό) χρησιμοποιώντας μια βούρτσα ως εργαλείο-tool.



Ακολουθώντας, έχοντας φτιάξει την εικόνα αυτή/ το έργο, θα φυλάξουμε το πινέλο μας και θα κρεμάσουμε στο τοίχο το γελαστό πρόσωπο που φτιάξαμε. Αφού περάσουν μερικές μέρες και αφού διαπιστώσουμε ότι πραγματικά οι άνθρωποι είναι ευτυχισμένοι με την εικόνα μας θα διαπιστώσουμε ότι ο αρχικός μας στόχος έχει επιτευχθεί. Επίσης θα μπορούσαμε να ολοκληρώσουμε αυτόν τον στόχο χρησιμοποιώντας:

- ✱ Μία ποικιλία από διάφορες μεθόδους-σχέδια στο χέρι, πλαστικές στάμπες, εικόνες από περιοδικά.
- ✱ Μία ποικιλία από διάφορους συμβολισμούς-την γραφική απεικόνιση ενός γελαστού προσώπου ή ενός κινούμενου σχεδίου ή ακόμα το διηγηματικό κείμενο ενός ανέκδοτου ή πινακίδας.
- ✱ Μία ποικιλία από διάφορα εργαλεία όπως ένα στυλό, ένα μολύβι, ένα πινέλο ή ένα κραγιόν.

Τώρα ας συγκρίνουμε το πιο πάνω έργο με ένα ανάλογο έργο της κατασκευής ενός σπιτιού. Ο αρχιτέκτονας και οι εργάτες που θα κτίσουν ένα σπίτι, φεύγοντας από την οικοδομή θα πάρουν τις συσκευές και τα εργαλεία τους μαζί. Το σπίτι το οποίο είχαν κτίσει θα συνεχίσει να υπάρχει εκεί σαν μαρτυρία προς τα υλικά που χρησιμοποίησαν δηλαδή κατά πόσο στερεή είναι η μέθοδος κατασκευής και ο τρόπος προσέγγισης και πόσο κομψό ήταν το σχέδιο που είχαν πριν ξεκινήσουν. Αυτό θα είναι χρήσιμο αργότερα όταν θα χρειάζεται να συντηρήσουμε ή να ανακαινίσουμε το σπίτι. Η άνεση του και η δυνατότητα συντήρησης του θα είναι το κύριο μέτρο επιτυχίας.

Το ίδιο είναι αληθές και για την ανάπτυξη προγραμμάτων: το πραγματικό κληροδότημα της ανάπτυξης προγραμμάτων είναι το τελικό σύστημα προγράμματος, το οποίο είναι σε τελική ανάλυση ο λόγος για τον οποίο χρησιμοποιούμε την μεθοδολογία αυτή για να κατασκευάσουμε ένα μοντέλο. Θα πρέπει να φροντίσουμε να αποφύγουμε τον εγκλωβισμό μας σε συζητήσεις αμφισβήτησης των σχετικών προσόντων μιας μεθοδολογίας σε σύγκριση με άλλη με την οποία αποτύχαμε να κατασκευάσουμε ωφέλημα

προγράμματα... υπάρχουν πολλά μονοπάτια που οδηγούν στον ίδιο προορισμό.



Η δική μας μέθοδος Μοντελοποίησης με Λίγα Λόγια

Παρουσιάζουμε εδώ μια βασική ανασκόπηση της μεθόδου μοντελοποίησης, την οποία εισηγούμαστε και την οποία θα επιδείξουμε σε βάθος διαμέσων του υπόλοιπου ηλεκτρονικού υλικού που θα βρείτε στη ιστοσελίδα αυτή.

- ✱ Ξεκινάς με την περιγραφή για το συγκεκριμένο πρόβλημα προς επίλυση, παρόμοια με το Σύστημα Εγγραφής Φοιτητών(SRS) που περιγράφεται. Σκέψου τις διάφορες κατηγορίες χρηστών οι οποίοι θα έρχονται σε επαφή με το σύστημα και τις διάφορες καταστάσεις κάτω από τις οποίες ο καθένας θα το χρησιμοποιεί, για να είσαι σίγουρος ότι έχεις ανακαλύψει όποιες απαιτήσεις δεν είναι και τόσο εμφανές αρχικά και τις οποίες πιθανό να είχες ξεχάσει.
- ✱ Χρησιμοποίησε τα δεδομένα της εφαρμογής αυτής με την ταυτοποίηση και ανάδειξη των διαφόρων τάξεων/ βαθμίδων των αντικειμένων του πραγματικού κόσμου, τα οποία θα χρειαστούν στην εφαρμογή αυτή και καθόρισε/ προσδιόρισε πώς αυτά αλληλοσυνδέονται[Ενότητα ΙΙΙ]
- ✱ Χρησιμοποίησε την λειτουργική πλευρά της εφαρμογής με την μελέτη του «πως» τα διάφορα αντικείμενα θα πρέπει να συνεργάζονται για να φέρουν εις πέρας την αποστολή του συστήματος ,καθορίζοντας ποιές συμπεριφορές/ ευθύνες θα χρειαστούν από κάθε βαθμίδα και επίπεδο[Ενότητα ΙV]
- ✱ Έλεγξε το μοντέλο για να διασφαλίσεις ότι πραγματικά ανταποκρίνεται σε όλες τις αρχικές πραγματικές απαιτήσεις του συστήματος.

Θα δεις μία πληθώρα παραδειγμάτων για κάθε μια από αυτές τις τεχνικές στις ενότητες που ακολουθούν και θα έχουμε την ευκαιρία να εξασκήσουμε αυτές τις τεχνικές/ τεχνάσματα που βασίζονται στις ασκήσεις που εισάγονται στο τέλος. Οπλισμένοι με ένα στερεό μοντέλο SRS, θα είσαι έτοιμος να αποδώσεις το μοντέλο αυτό σε κώδικα JAVA ,που είναι το θέμα της Ενότητας ΙV του παρόντος εγχειριδίου.

Σημειώστε ότι αυτές οι μέθοδοι και βήματα δεν είναι κατά ανάγκη σημαντικό να εκτελούνται με την σειρά που παρατίθενται. Στην ουσία, καθώς εξομοιώνεσαι με κάθε ένα από αυτά τα βήματα, θα δεις ότι θα μπορείς να τα εκτελέσεις παράλληλα ή με ανακατεμένη σειρά. Για παράδειγμα, σκεφτόμενοι τους τομείς συμπεριφοράς ενός μοντέλου θα μπορούμε να φέρουμε στο φως νέα δεδομένα απαιτήσεων. Στην πραγματικότητα, για όλα σχεδόν τα μοντέλα

είναι συνηθισμένο να περνάς μέσα από αυτά τα βήματα πολλές φορές, κάθε φορά μπαίνοντας σε αυξημένα επίπεδα κατανόησης και έτσι να δίνεται περισσότερη λεπτομέρεια στη κατασκευή του μοντέλου και της στήριξης του κάθε φορά.

Είναι επίσης σημαντικό να σημειώσουμε ότι η επισημότητα της μεθόδου θα πρέπει να ρυθμίζεται ανάλογα με το μέγεθος της ομάδας του έργου και την πολυπλοκότητα των απαιτήσεων. Εάν ξεχωρίσουμε τον τρόπο(form) της χρήσης μίας μεθοδολογίας από την ουσία(substance) από του τι παράγει αυτή η μεθοδολογία με τον τρόπο της μοντέλο, βιβλιογραφία, κωδικοποίηση και ούτω καθεξής- τότε ένας καλός κανόνας είναι ότι η ομάδα εργασίας δεν θα έπρεπε να σπαταλά περισσότερο από 10-20% του χρόνου τους στη φόρμα(form) και 80-90% στην ουσία(substance). Εάν η ομάδα βρει ότι ξοδεύει πολύ χρόνο στην φόρμα και ότι λίγη ή καθόλου πρόοδος έχει γίνει στην ουσία, είναι καιρός να επαναξιολογήσουν την μεθοδολογία και τα διάφορα κομμάτια της, για να δούμε που μπορούν να γίνουν ρυθμίσεις απλοποίησης ή βελτιώσεις στην απόδοση.

Σκέψεις σχετικά με τα Εργαλεία Προγραμματισμού της Αντικειμενοστραφούς Μοντελοποίησης

Αξίζει τον κόπο να ξοδέψουμε λίγο χρόνο για να μιλήσουμε για τα πλεονεκτήματα και τα μειονεκτήματα της χρήσης εργαλείων του προγραμματισμού της Αντικειμενοστραφούς Μοντελοποίησης .

Τα εργαλεία για την μοντελοποίηση αντικειμένου, εμπίπτουν κάτω από την γενική κατηγορία των τίτλων "Computer Aided Software Engineering" ή αλλιώς CASE.

Τα πλεονεκτήματα του CASE σε γενικές γραμμές είναι:

- ✳ Τα εργαλεία CASE παρέχουν ένα γρήγορο τρόπο για την δημιουργία ενός μοντέλου. Παρά να προσπαθούμε να αποδώσουμε ένα δεδομένο συμβολισμό με ένα γενικό εργαλείο σχεδιασμού, όπου τα βασικά του στοιχεία ζωγραφίσματος είναι απλές γραμμές, βέλη, κείμενο, κουτιά και άλλα γεωμετρικά σχήματα, τα εργαλεία CASE παρέχουν ένα ή περισσότερες παλέτες από προκατασκευασμένα γραφικά στοιχεία, συγκεκριμένα για τον στηριζόμενο συμβολισμό. Έτσι μπορείς να διαλέξεις ή να αφήσεις τις γραφικές απεικονίσεις και συμβολισμούς για ένα επίπεδο ή παράδειγμα μοντέλου παρά να έχεις να το συναρμολογήσεις με επίπονο μόχθο από την αρχή χρησιμοποιώντας απλούστερα στοιχεία σχεδιασμού.
- ✳ Τα εργαλεία CASE παράγουν έξυπνα σχέδια τα οποία ενδυναμώνουν τους συντακτικούς κανόνες ενός συγκεκριμένου συμβολισμού. Αυτό είναι σε αντίθεση με το γενικό πακέτο σχεδιασμού, το οποίο θα σε άφηνε να σχεδιάσεις ότι σου αρέσει είτε αυτό απευθύνετε στη σύνταξη συμβολισμού ή όχι. Φυσικά, εδώ πρέπει να σημειώσουμε πώς ο έλεγχος που επιβάλλεται από τα εργαλεία CASE από την μία μπορεί να σε αποτρέψει από το να κάνεις συντακτικά λάθη αλλά από την άλλη μπορεί επίσης να σε εμποδίσει από το να κάνεις τις απαιτούμενες επιθυμητές ρυθμίσεις στο συμβολισμό σου.
- ✳ Πληροφορίες για τις ταξινομήσεις αντικατοπτρίζονται στο διάγραμμα - τα ονόματα τους, ιδιότητες, εμβλήματα και σχέσεις - είναι αποθηκευμένα σε μια αποθήκη που καθορίζει το διάγραμμα. Τα πλείστα εργαλεία CASE παρέχουν την

δυνατότητα δημιουργίας κειμένου χαρακτηριστικών βασισμένα σε αυτή την αποθήκη, καθιστώντας σε ικανό να δημιουργήσεις αυτόματα ένα κείμενο για το έργο όπως για παράδειγμα ένα λεξικό δεδομένων, ένα τύπο αναφοράς που θα συζητήσουμε στην Ενότητα ΙΙΙ. Κάποια εργαλεία ακόμα, σου επιτρέπουν να μπεις σε αυτή την αποθήκη και με κατάλληλο προγραμματισμό να φτιάξεις αυτό που επιθυμείς αν αυτό χρειαστεί.

- ✱ Τα περισσότερα εργαλεία CASE παρέχουν την δυνατότητα δημιουργίας κώδικα, καθιστώντας σε ικανό να μεταπήδησης από ένα διάγραμμα κώδικα JAVA ή C++ με το πάτημα ενός κουμπιού. Θα μπορούσες ή όχι να ωφεληθείς με αυτό το χαρακτηριστικό για τους εξής λόγους:

- ✚ Ανάλογα με το πόσο έλεγχο μπορείς να έχεις από τα εργαλεία CASE σε σχέση με τη δομή που παίρνει ο δημιουργημένος κώδικας, ο κώδικας που δημιουργείται δυναμικά δεν θα ανταποκρίνεται στα επίπεδα της ομάδας εργασίας.

- ✚ Με τα πλείστα εργαλεία, δεν μπορείς να γράψεις/διορθώσεις τον δημιουργηθέντα κώδικα έξω από τα εργαλεία, επειδή το εργαλείο τότε δεν θα είναι ενήμερο για τις αλλαγές που έκανες, που σημαίνει ότι την επόμενη φορά που θα δημιουργηθεί ο κώδικας οι αλλαγές που έκανες θα διαγραφούν.

- ✚ Επίσης βεβαιώσου ότι τα εργαλεία που επέλεξες σου επιτρέπουν την εισαγωγή και χρήση άλλων στοιχείων προγραμμάτων, τα οποία δεν δημιουργήθηκαν μέσα σε αυτό. Αυτό έχει επιπτώσεις στην περίπτωση χρήσης κωδικών για άλλα έργα.

- ✱ Είναι κάποτε καλύτερα να γράφουμε τον κώδικα μας στο τέλος αντί στην αρχή, παρά το γεγονός ότι ίσως είναι πιο χρονοβόρο, είναι πολύ πιο εύκολο να διαχειρίζεσαι τον κώδικα μακροπρόθεσμα για το έργο σου και να αποφεύγεις έτσι την πιθανότητα να γίνεσαι σκλάβος σε ένα συγκεκριμένο εργαλείο μοντελοποίησης για μια μακρά συντήρηση κώδικα.

- ✱ Πολλά εργαλεία CASE επιτρέπουν ένα είδος ελέγχου του τύπου version control επιτρέποντας έτσι να διατηρείς γενεές από το ίδιο μοντέλο. Εάν κάνεις μια αλλαγή στο μοντέλο σου και όταν το ξαναδείς με τον χειριστή/χρήστη αποφασίσετε να επανέλθετε στην αρχική μορφή των πραγμάτων, είναι ασήμαντος κόπος να γίνει αν υπάρχει ο έλεγχος του τύπου των εργαλείων.

- ✖ Τα εργαλεία CASE παρέχουν την δυνατότητα διαχείρισης ελέγχου και δυνατοτήτων επιτρέποντας σε μια ομάδα μοντελιστών εύκολα να μπορούν να μοιραστούν τα καθήκοντα στην δημιουργία ενός μοντέλου.
- ✖ Κάποια εργαλεία CASE στηρίζουν πολλαπλούς γραφικούς συμβολισμούς, επιτρέποντας σου αρχικά να δημιουργήσεις ένα διάγραμμα σε ένα συμβολισμό(για παράδειγμα OMT) αλλά μετά να μετατρέψεις το διάγραμμα σε άλλο συμβολισμό (όπως π.χ. UML) πολύ γρήγορα, χωρίς προσπάθεια ή κόπο.

Τα εργαλεία CASE έχουν και τα μειονεκτήματα τους:

- ✖ Τα εργαλεία αυτά μπορεί να είναι ακριβά. Δεν είναι ασυνήθιστο για ένα τέτοιο εργαλείο να κοστίζει μερικές εκατοντάδες αν όχι και χιλιάδες ευρώ για κάθε χρήστη.
- ✖ Κάποτε μπορεί να μην είναι ευέλικτα. Μιλάμε για την υιοθέτηση μεθόδων, συμβολισμών και εργαλείων αλλά τα εργαλεία δεν συνεργάζονται πάντα. Θα πάρουμε μερικά συγκεκριμένα παραδείγματα αργότερα εάν το εργαλείο CASE παρέχει αυτή τη δυνατότητα.
- ✖ Ένα Εργαλείο CASE δεν μπορεί να χρησιμοποιηθεί όταν είμαστε στα μισά της μελέτης, επειδή το υπάρχον πρόγραμμα δεν εισάγει πάντα το ζητούμενο εργαλείο.

Είναι εύκολο να εγκλωβισθούμε με μια φόρμα ουσίας. Αυτό είναι αληθές για κάθε αυτοματοποιημένο εργαλείο, ακόμα και ένα πρόγραμμα δακτυλογραφίας αναγκάζει τους ανθρώπους να ξοδεύουν περισσότερο χρόνο στα διακοσμητικά ενός κειμένου παρά στην ουσία.

Ενότητα II

UML: Γλώσσα Ενιαίας Μοντελοποίησης - Microsoft Internet Explorer

File Edit View Favorites Tools Help

Back Forward Stop Refresh Home Search Favorites History Mail Print Edit Discuss

Unified Modeling Language™

αρχική σελίδα σχετικά με τον ιστοχώρο..

Βήμα προς βήμα για την μοντελοποίηση του SRS

- Τακτοποίηση των Απαιτήσεων χρησιμοποιώντας Ανάλυση Περιπτώσεων Χρήσης

Ενότητα II

	Εισαγωγή	>>
	Τι είναι τα "Use Case"	>>
	Χρήστες(actors)	>>
	Καθορίζοντας τα "Use Case"	>>
	Μοντελοποιώντας ή	>>

Εισαγωγή

Όταν ετοιμαζόμαστε να πάμε διακοπές, περνάμε από το μυαλό μας πολλές σκέψεις. "Πακετάραμε όλα όσα χρειαζόμαστε;" "Πακετάραμε περισσότερα;" "Έχουμε κανονίσει να έχουμε τις κατάλληλες υπηρεσίες (εφημερίδες, ταχυδρομείο κλπ) τερματισμένες;" "Φροντίσαμε να ποτίζει κάποιος τα λουλούδια και να ταΐζει τα ζώα;" Μόλις αναχωρήσουμε για το ταξίδι θέλουμε να γνωρίζουμε ότι θα διασκεδάσουμε και όταν φθάσουμε σπίτι ξανά θέλουμε να βρούμε τα πράγματα όπως τα αφήσαμε χωρίς καταστροφές.

Αυτό δεν διαφέρει και πολύ από μία μελέτη ανάπτυξης και ανάλυσης ενός προγράμματος. Θα πρέπει να οργανώσουμε ένα κατάλογο των εφαρμογών που θα πρέπει το πρόγραμμα να παρέχει πριν ξεκινήσουμε την δημιουργία του, έτσι ώστε η μελέτη να κυλά ομαλά και να μην δημιουργήσουμε μία καταστροφή(στην περίπτωση που δεν εκπληρώνονται οι απαιτήσεις μας και δυσχερατούμε τον πελάτη μας/ χρήστη του συστήματος).

Η τέχνη και η επιστήμη της ανάλυσης απαιτήσεων, είναι το πολύ εκτεταμένο σαν θέμα που θα μπορούσαμε να αφιερώσουμε ολόκληρο τόμο μονάχα για αυτό. Υπάρχει μία τεχνική συγκεκριμένα για την ανακάλυψη και ομαδοποίηση των απαιτήσεων που ονομάζεται "Use Case" (Περίπτωση Χρήσης) και είναι ο θεμέλιος λίθος του "Rational Unified Process" στη μοντελοποίηση και το οποίο εγγυάται τις σκέψεις και την μελέτη σας. Τα «Use Cases» δεν είναι αυστηρά ένα χειροποίητο αντικείμενο της μεθοδολογίας ΟΟ . Μπορούν να ετοιμαστούν για κάθε αναπτυσσόμενο σύστημα ανεξάρτητα από την μεθοδολογία ανάπτυξης που χρησιμοποιείται.

Σε αυτή την ενότητα θα μάθεις:

- ✦ Πώς θα πρέπει να αντισταθμίσουμε όλους τους διάφορους ρόλους που θα έχουν οι χρήστες όταν θα έρθουν σε επαφή με το μελλοντικό μας σύστημα.
- ✦ Θα πρέπει να υποθέσουμε κάθε μία από τις απόψεις, στο να περιγράψουμε τις υπηρεσίες που ένα πρόγραμμα θα παρέχει σαν σύνολο.
- ✦ Πώς να ετοιμάσουμε ένα "Use Case" σαν μέσο που να περιγράψουμε και να στοιχειοθετούμε τα πιο πάνω.

Τι είναι τα "Use Cases"

Στον προσδιορισμό της επιθυμητής λειτουργικότητας του συστήματος μας, θα πρέπει να αναζητήσουμε τις απαιτήσεις στα ακόλουθα:

- Ποιος θα θέλει να χρησιμοποιήσει το σύστημα?
- Τι είδους υπηρεσίες θα πρέπει να παρέχει το σύστημα μας για να είναι κάποιος αξίας για αυτό?
- Όταν ο χρήστης έρχεται σε επαφή με το σύστημα για μία συγκεκριμένη εφαρμογή, ποια είναι η προσδοκία του σχετικά με το επιθυμητό αποτέλεσμα?

Τα "Use Case" είναι ο φυσικός τρόπος έκφρασης των απαιτήσεων στα πιο πάνω ερωτήματα. Κάθε "Use Case" είναι μία απλή φράση, περιγραφικό κείμενο ή γραφική απεικόνιση, που περιγράφει συγκεκριμένο στόχο ή αποτέλεσμα του συστήματος. Για παράδειγμα ένας στόχος του SRS είναι η εγγραφή ενός φοιτητή σε συγκεκριμένο "Course". Αυτό είναι το πρώτο μας "Use Case"! (Ναι, πραγματικά τα "Use Case" είναι τόσο απλά). Στην ουσία, πρέπει να είναι τόσο απλά, για να είναι κατανοητά στον χρήστη του συστήματος, όπως θα συζητήσουμε πιο κάτω.

Χρήστες

Οι χρήστες παριστάνουν οποιονδήποτε και οτιδήποτε που θα αλληλεπιδρούσε με το σύστημα αφότου κατασκευαστεί. Οι χρήστες εξάλλου είναι αυτοί που χρησιμοποιούν τα "Use Cases". Οι χρήστες γενικά εμπίπτουν σε δύο ευρείες κατηγορίες:

- Ανθρώπους χρήστες
- Άλλα συστήματα υπολογιστών

"Αλληλεπίδραση" έχουμε από 1)παροχή/ συνεισφορά πληροφοριών του συστήματος ή και 2)λαμβάνοντας/ καταναλώνοντας πληροφορίες από το σύστημα.

- ◆ Παρέχοντας πληροφορίες εννοούμε κατά πόσο ή όχι οι πληροφορίες που δόθηκαν από τον χρήστη, προσθέτονται στα βασικά δεδομένα του συστήματος: για παράδειγμα, μία θέση διδασκαλίας στο τμήμα ή εγγράφοντας ένα φοιτητή στο κατάλληλο τμήμα σπουδών.
- ◆ Καταναλώνουμε πληροφορίες εννοούμε ότι ο χρήστης χρησιμοποιεί το σύστημα για να εξασφαλίσει πληροφορίες/ γνώσεις: για παράδειγμα, ένας χρήστης στο τμήμα της σχολής εκτυπώνει ένα κατάλογο για ένα κλάδο σπουδών τον οποίο διδάσκει.

Πρέπει να δημιουργήσουμε ένα χειριστή για κάθε διαφορετικό ρόλο που θα θεωρηθεί από τις διάφορες κατηγορίες χειριστών σχετικά με το σύστημα. Για να προσδιορίσουμε τέτοιους ρόλους, τυπικά σκεφτόμαστε τις περιγραφικές απαιτήσεις χαρακτηριστικών, εάν υπάρχει κάτι τέτοιο: δηλαδή, μία δήλωση λειτουργικών απαιτήσεων, όπως τα χαρακτηριστικά του συστήματος εγγραφής φοιτητών. Η μόνη κατηγορία χρηστών που αναφέρθηκε διεξοδικά από τα χαρακτηριστικά είναι οι φοιτητές. Έτσι θα θεωρούσαμε στα σίγουρα τον φοιτητή να είναι ένας τύπος χειριστών για το SRS. Εάν σκεφτούμε πέραν των χαρακτηριστικών, όμως, είναι δύσκολο να έλθουμε σε επαφή με άλλες κατηγορίες χρηστών που επίσης ωφελούνται από την χρήση του SRS:

- ◆ Ο εκπρόσωπος του τμήματος μπορεί να επιθυμεί να διαβάσει/ αριθμήσει πόσοι φοιτητές είναι εγγεγραμμένοι σε ένα τμήμα των επικείμενων μαθημάτων που θα διδακτού, ή μπορεί να χρησιμοποιήσουν το σύστημα για να δώσουν τελική βαθμολογία που με την σειρά τους ανακυκλώνονται στο "Transcript" (πιστοποιητικό σπουδών) του φοιτητή.
- ◆ Οι διευθυντές των τμημάτων μπορεί να επιθυμούν να δουν πόσο δημοφιλές είναι κάποια μαθήματα. Εδώ πρέπει να σημειωθεί πως ένα

μάθημα δεν οφείλει να ακυρωθεί λόγω έλλειψης ενδιαφέροντος στο τμήμα από τους φοιτητές.

- ◆ Το προσωπικό στο γραφείο του διευθυντή μπορεί να επιθυμεί να χρησιμοποιήσει το SRS για να αποδείξει ότι ένας συγκεκριμένος φοιτητής προβάλλεται να έχει ανταποκριθεί στις απαιτήσεις για να αποφοιτήσει σε ένα συγκεκριμένο εξάμηνο.
- ◆ Μπορεί το Ακαδημαϊκό Συμβούλιο να επιθυμεί να χρησιμοποιήσει το SRS και να ζητήσει αντίγραφα των "Transcript".
- ◆ Πιθανοί φοιτητές- αυτοί που σκέφτονται να επιβάλλουν αίτηση αλλά δεν το έχουν κάνει- μπορεί να επιθυμούν να ελέγξουν τα μαθήματα και τους κλάδους σπουδών που προσφέρονται σε ένα επερχόμενο εξάμηνο για να ελέγξουν κατά πόσο το πανεπιστήμιο διαθέτει κάποιες πληροφορίες που να ανταποκρίνονται στα ενδιαφέροντα τους.

Ομοίως, εφόσον είπαμε ότι άλλα συστήματα υπολογιστών μπορεί να είναι ενεργοί χρήστες, μπορεί να έχουμε να κτίσουμε ενδιάμεσους κρίκους ανάμεσα στο SRS και άλλα υπάρχοντα αυτοματοποιημένα συστήματα στο πανεπιστήμιο, όπως:

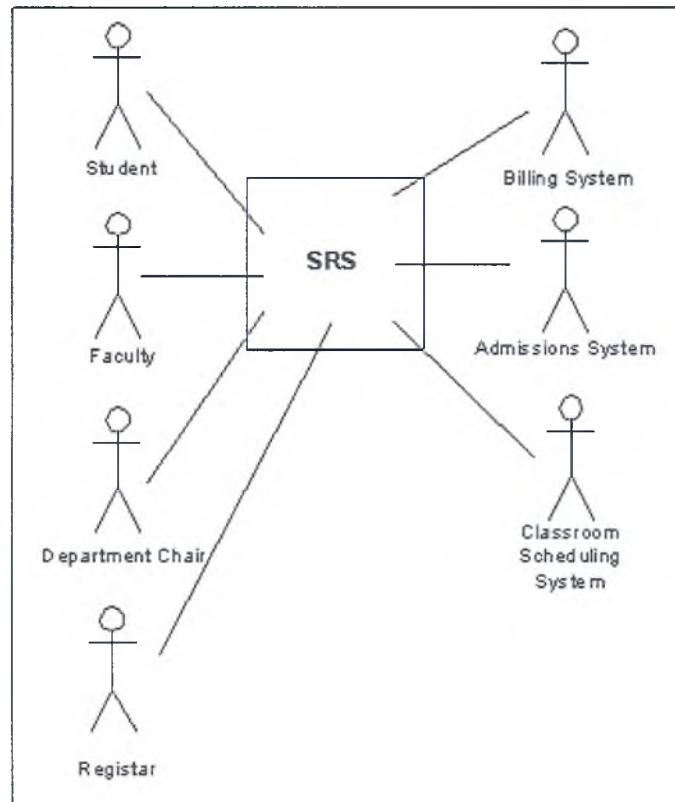
- ◆ Το σύστημα τιμολόγησης -"Billing System", έτσι ώστε να μπορούν οι φοιτητές να χρεώνονται ακριβώς βασισμένοι στο παρόν φορτίο σπουδών.
- ◆ Το σύστημα σχεδιασμού τάξεων -"Classroom Scheduling", για να διασφαλίσει ότι οι τάξεις που θα διδάσκονται κατανέμονται σε αμφιθέατρα που έχουν την κατάλληλη χωρητικότητα.
- ◆ Το σύστημα εγγραφών -"Admissions System", έτσι ώστε το SRS μπορεί να ενημερωθεί όταν ένας νέος φοιτητής έχει εγγραφεί για να παρακολουθεί κλάδους σπουδών.

Φυσικά, πρέπει να πάρουμε μία απόφαση ούτως ώστε ο σκοπός του συστήματος που θα κτίσουμε να είναι τέτοιος ώστε να αποφύγουμε τις παραφουσκωμένες απαιτήσεις. Το να προσπαθήσεις να δώσεις χώρο σε όλους τους πιθανούς χρήστες που αναφέρθηκαν πιο πάνω θα είχε σαν αποτέλεσμα μία μαζική υποχρέωση που μπορεί απλά να είναι πολύ δαπανηρό για τους χορηγούς του συστήματος. Για παράδειγμα, θα έχει νόημα να προνοήσεις για πιθανούς μελλοντικούς φοιτητές, χρήστες του SRS, να ανασκοπήσουν στο τι το πανεπιστήμιο προσφέρει και τον τρόπο που τα διάφορα μαθήματα προσφέρονται, ή μήπως υπάρχει διαφορετικό σύστημα-ας πούμε ένας κατάλογος on-line- που αρμόζει καλύτερα στην παρούσα περίπτωση? Διαμέσου συνεντεύξεων με όλες τις ομάδες προτιθέμενων χρηστών, ο σκοπός του συστήματος μπορεί να περιβληθεί όπως πρέπει και κάποιοι από τους υποτιθέμενους πιθανούς χρήστες να απαλειφθούν.

Στην δική μας συγκεκριμένη περίπτωση θα υποθέσουμε ότι οι χορηγοί του SRS έχουν αποφασίσει ότι δεν χρειάζεται να περιλάβουν τις ανάγκες του Ακαδημαϊκού Συμβουλίου ή τους μελλοντικούς φοιτητές σαν χρήστες. Ένα σημείο κλειδί εδώ είναι ότι οι χορηγοί αποφασίζουν τέτοια πράγματα και όχι οι προγραμματιστές. Ουσιαστικά οι χορηγοί του συστήματος έχουν δικαιωματικά τον τελευταίο λόγο στο τι θα κατασκευαστεί.

Σημείωσε ότι ο ίδιος χρήστης μπορεί να επιδρά με το σύστημα σε διάφορες περιπτώσεις και διαφορετικούς ρόλους των χρηστών. Δηλαδή, ένας καθηγητής που διευθύνει ένα τμήμα, μπορεί να θεωρήσει τον ρόλο του χειριστή σαν διευθυντής του τμήματος όταν αυτός/ αυτή προσπαθεί να καθορίσει κατά πόσο ένα μάθημα θα πρέπει να ακυρωθεί. Εναλλακτικά, ο ίδιος καθηγητής μπορεί να θεωρήσει και τον ρόλο του χρήστη "Faculty" όταν αυτός/ αυτή επιθυμεί να ερωτήσει το SRS για την κατανομή των φοιτητών για το συγκεκριμένο μάθημα που διδάσκει.

Εφόσον έχουμε αποκατασταθεί στο όσον αφορά τους χρήστες του σύστημα μας, το μόνο που μας μένει είναι να σκιαγραφήσουμε γραφικά το σύστημα με τους χρήστες σε συμβολισμό UML:

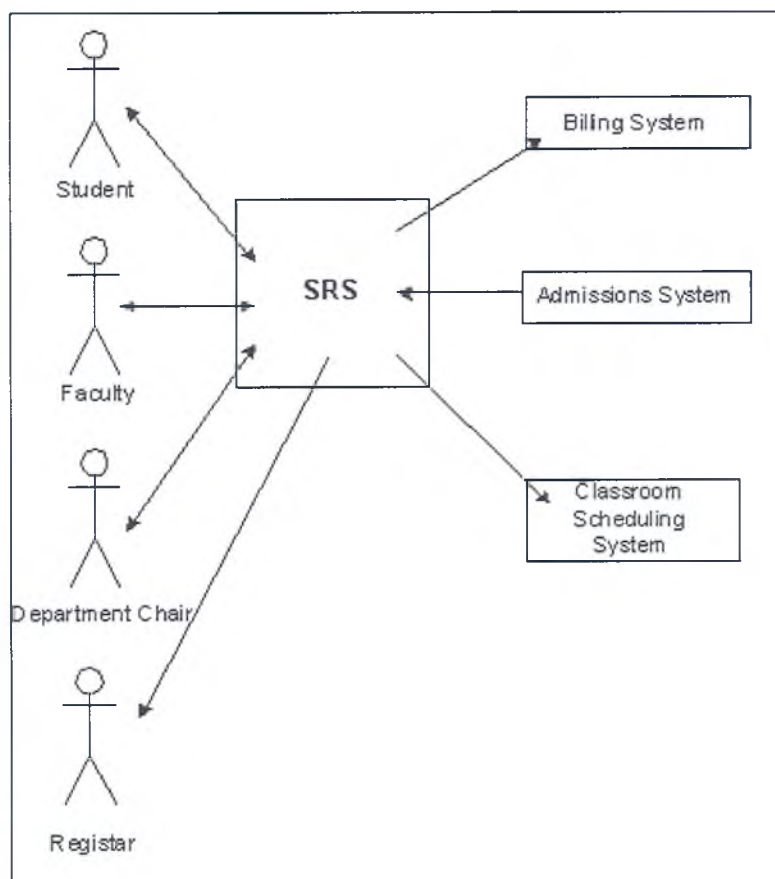


Προτιμούμε να χρησιμοποιήσουμε ελαφρώς τροποποιημένη μορφή του UML, όπως ακολούθως:

- ◆ Επιμηκύνουμε την χρήση του κουτιού/ ορθογωνίου για να παριστάνει όχι μόνο το βασικό σύστημα αλλά επίσης όλα τα εξωτερικά συστήματα των χρηστών , παρά να παριστάνει το τελευταίο σαν ανθρώπινες γραμμικές φιγούρες.
- ◆ Βρίσκουμε ότι χρησιμοποιώντας κεφαλές με βέλη για να παραστήσουμε κατευθυνόμενες ροές πληροφοριών- δηλαδή κατά πόσο ένας χρήστης καταναλίσκει ή παρέχει πληροφορίες- είναι κάπως πιο επικοινωνιακό. Για παράδειγμα, στην αναθεωρημένη έκδοση του συμβολισμού που ακολουθεί, αναπαριστά τον φοιτητή που ταυτόχρονα παρέχει και καταναλίσκει πληροφορίες, ενώ ο διευθυντής καταναλίσκει μόνο πληροφορίες.

Ο "Registrar" πραγματικά παρέχει πληροφορίες αλλά όχι προς το SRS: « παρέχει πληροφορίες στο Σύστημα Εισαγωγής Φοιτητών-"Admissions System" για το ποιοι φοιτητές εγγράφονται στο πανεπιστήμιο. Αυτή η πληροφορία τροφοδοτείται στο σύστημα SRS από το Σύστημα Εισαγωγής Φοιτητών-"Admissions System". Έτσι το Σύστημα Εισαγωγής Φοιτητών φαίνεται σαν ο τροφοδότης πληροφοριών στο SRS και παρίσταται σαν ένας ενεργός χρήστης του SRS. Από την πλευρά του SRS ο "Registrar" δεν είναι τίποτα περισσότερο παρά ένας καταναλωτής»

Με αυτές τις λίγες αλλαγές στον συμβολισμό, το διάγραμμα UML γίνεται ένα πολύ πιο επικοινωνιακό εργαλείο:



Καθορίζοντας τα "Use Cases"

Κάνοντας την πρώτη περικοπή στο ποιοι είναι οι χρήστες του SRS, θα αριθμήσουμε με ποιους τρόπους το σύστημα θα χρησιμοποιηθεί από τους χρήστες, με άλλα λόγια τι "Use Case" θα χρησιμοποιήσουμε.

Ένα "Use Case" παριστάνει ένα «λογικό νήμα» ή μία σειρά από αιτίες και επιδρόντα γεγονότα, αρχίζοντας με την πρώτη επαφή του χρήστη με το σύστημα και τελειώνοντας με την επίτευξη των στόχων του χρήστη που αρχικά χρησιμοποίησε το σύστημα.

Τα "Use Cases" δίνουν έμφαση στο τι κάνει το σύστημα- λειτουργικές απαιτήσεις- χωρίς να νοιάζεται για το πώς θα επιτευχθούν εσωτερικά. Στην ουσία μπορείς να σκεφτείς μία "Use Case" σαν υποβολή συμπεριφοράς για το σύστημα σαν ολότητα.

Κάποια παραδείγματα ψηλού επιπέδου "Use Case" για το SRS μπορούσε να είναι τα εξής:

- Εγγραφή για μάθημα
- Εγκατάλειψη μαθήματος
- Υπολογισμός του φορτίου μαθημάτων για τον φοιτητή
- Επιλογή συμβουλου
- Εγκαθίδρυση πλάνου σπουδών
- Ορατή λήψη σχεδιαγράμματος τάξεων/ κλάσεων
- Παράκληση για ένα κατάλογο των μαθημάτων του φοιτητή
- Παράκληση για ένα πιστοποιητικό σπουδών φοιτητή
- Διατήρηση των πληροφοριών για τα διάφορα μαθήματα(για παράδειγμα, αλλαγή της περιγραφής του μαθήματος, ανάκλαση διαφορετικού λέκτορα για το μάθημα κτλ)
- Ταχυδρόμηση της τελικής βαθμολογίας του εξαμήνου για το συγκεκριμένο μάθημα

Να θυμάστε, ότι το "Use Case" ενεργοποιήθηκε από τον χρήστη που είναι και ο λόγος για τον οποίο δεν καταγράψαμε σε κατάλογο άλλες λειτουργίες και απαιτήσεις χαρακτηριστικών, όπως για παράδειγμα "ειδοποίησε τον φοιτητή με email", που καλούνται από το SRS.

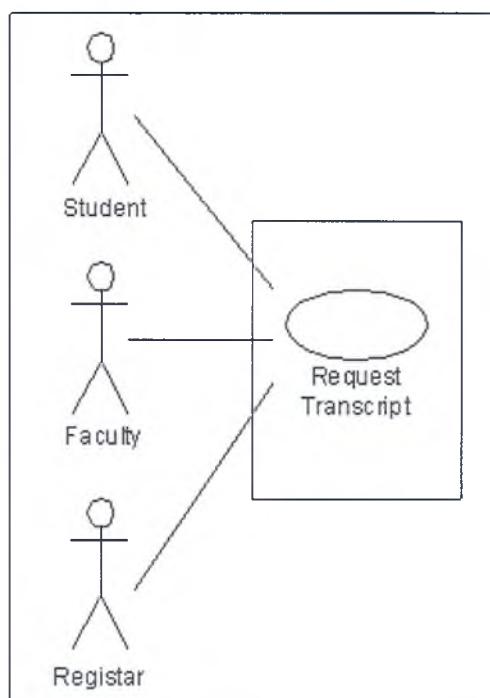
Μπορούμε να αποσυνθέσουμε τα "Use Case" σε στάδια, καθένα εκ των οποίων παριστούσε το υπό "Use Case". Για παράδειγμα, "Εγγραφή σε τμήμα-Course" μπορεί να αποσυντεθεί σε:

- Εξέτασε ότι ο φοιτητής πλήρη τις προαπαιτήσεις
- Εξέτασε το πλάνο σπουδών του φοιτητή για να βεβαιώσεις ότι το εν λόγω μάθημα χρειάζεται
- Έλεγε την ύπαρξη θέσεων για το εν λόγω μάθημα
- Τοποθέτησε τον φοιτητή σε λίστα αναμονής αν το επιθυμεί

Δυστυχώς, όπως αληθεύει για όλες τις απαιτήσεις ανάλυσης, δεν υπάρχει η μαγική φόρμουλα για να εφαρμοστεί έτσι ώστε να δούμε κατά πόσον έχουν προσδιοριστεί όλες οι σημαντικές "Use Case" ή όλοι οι χρήστες.

Μοντελοποιώντας ή Όχι?

Το θέμα των "Use Case" είναι κάπως ευθύ και απλώς περιγραφικό κείμενο όπως έχουμε δει μέχρι τώρα στη παρόν ενότητα. Αυτό είναι συχνά αρκετό για να εκφραστεί το "Use Case". Η UML παρέχει ένα επίσημο τρόπο για να σχεδιάζουμε τα "Use Case" και τις αλληλεπιδράσεις τους με τους χρήστες. Όπως αναφερθήκαμε προηγουμένως, οι χρήστες (άνθρωποι ή συστήματα) παρίστανται σαν γραφικές φιγούρες, ενώ τα "Use Case" παρίστανται σαν οβάλ με μία σύντομη περιγραφή από κάτω που περιγράφει το "Use Case". Το κουτί που περικλείει το οβάλ παριστάνει τα όρια του συστήματος. Μία απλή περίπτωση στο UML είναι η παρακάτω. Εδώ παριστούμε τρεις χρήστες: "Student", "Faculty" και "Registrar".



Εφόσον καταφέρεις να στοιχειοθετήσεις τους χρήστες και τα "Use Case" του συστήματος, είτε μόνο με γραπτό περιγραφικό κείμενο ή και με διαγράμματα, αυτό γίνεται μέρος του κορμού της βιβλιογραφίας που θα αυτοματοποιηθεί. Στην επόμενη Ενότητα θα μάθουμε πως θα χρησιμοποιούμε τέτοιου είδους βιβλιογραφία σαν σημείο αναφοράς για να καθορίσουμε τι κλάσεις θα χρειαστεί να δημιουργήσουμε και θα αρχίσουμε να κτίζουμε τα δομικά στοιχεία του συστήματος μας.

Το UML προλέγει κάποια επιπρόσθετη επιστημότητα με αναφορά στη μοντελοποίηση των Use Case.

Ενότητα III

UML: Γλώσσα Ενιαίας Μοντελοποίησης - Microsoft Internet Explorer

File Edit View Favorites Tools Help

Back Forward Stop Refresh Home Search Favorites History Mail Print Edit Discuss



Unified Modeling Language™

αρχική σελίδα

σχετικά με τον ιστοχώρο..

Βήμα προς βήμα για την μοντελοποίηση του SRS

- Μοντελοποίηση των Στατικών Χαρακτηριστικών Συμπεριφοράς του Συστήματος

Ενότητα III

UNIFIED MODELING LANGUAGE 	Εισαγωγή	>>
	Προσδιορίζοντας τις σωστές Κλάσεις	>>
	Ανάλυση Ουσιαστικών Προτάσεων	>>
	Έλεγχος των "Use Case"	>>
	Λεξικό Δεδομένων για	>>

82

Εισαγωγή

Έχοντας υιοθετήσει την ανάλυση με την τεχνική των "Use Case" στην ενότητα II για να ανταποκριθούμε στις απαιτήσεις του συστήματος SRS , είμαστε έτοιμοι να χειριστούμε το επόμενο στάδιο του μοντέλου μας, το οποίο θα προσδιορίσει τον τρόπο με τον οποίο θα μπορούσαμε να ανταποκριθούμε στις απαιτήσεις σε μια περίπτωση "Object-Oriented"

Ένα μοντέλο ΟΟ πρέπει να ορίζει:

- ✱ **Τι κλάσεις αντικειμένων πρόκειται να δημιουργήσουμε έτσι ώστε να γίνει η κατάλληλη δήλωση/ περιγραφή:** συγκεκριμένα, τα χαρακτηριστικά, οι μέθοδοι και οι δομικές σχέσεις μεταξύ τους. Επειδή αυτά τα στοιχεία σε ένα σύστημα "Object -Oriented" έχουν εγκαθιδρυθεί είναι σχετικά στατικό με τον ίδιο τρόπο που ένα σπίτι, εφόσον έχει ήδη κτιστεί έχει ένα συγκεκριμένο πλάνο, ένα δεδομένο αριθμό δωματίων, δεδομένη και συγκεκριμένη γραμμή οροφών κτλ. Αναφερόμαστε συχνά σε αυτή την διαδικασία σαν την διαδικασία ετοιμασίας του στατικού μοντέλου. Μπορούμε σίγουρα με το χρόνο να αλλάξουμε την στατική δομή του σπιτιού αυτού με διάφορες μελέτες επαναμοντελοποίησης , όπως μπορούμε να αλλάξουμε την στατική δομή ενός συστήματος προγραμματισμού ΟΟ καθώς και να ξεπροβάλλουν νέες υπό-κλάσεις εφευρίσκοντας νέες μεθόδους για ήδη υπάρχουσες κλάσεις και ούτω καθεξής. Όμως, εάν μια δομή - είτε ένα σπίτι είτε ένα πρόγραμμα- είναι σχεδιασμένη σωστά από την αρχή, τότε η ανάγκη για τέτοιες αλλαγές θα αναδύονται σχετικά πολύ σπάνια κατά την διάρκεια της ωφέλιμης ζωής του και οι οποίες θα ήταν δύσκολο να τις χειριστούμε.
- ✱ **Πώς θα πρέπει να συνεργάζονται αυτά τα αντικείμενα έτσι ώστε να υλοποιούνται οι συνολικές ανάγκες και η αποστολή του συστήματος:** Οι τρόποι με τις οποίες τα αντικείμενα(object) αλληλεπιδρούν μπορεί να αλλάξει από τη μια στιγμή στην άλλη βασιζόμενο στις περιστάσεις που υπάρχουν. Σε μία στιγμή ένα αντικείμενο "Course" εγγράφει ένα αντικείμενο "Student" και από την άλλη θα ανταποκρίνεται σε μια ερώτηση από ένα αντικείμενο "Professor" σχετικά με την τρέχουσα κατάσταση του φοιτητή -"Student". Αναφερόμαστε στη διαδικασία για συνεργασίες περί λεπτομερειών στην ετοιμασία της δυναμικής μοντελοποίησης. Σκεφτόμενοι αυτό σαν όλες τις διαφορετικές καθημερινές δραστηριότητες που

συμβαίνουν σε ένα σπίτι: ίδια δομή, διαφορετικές λειτουργίες. Η στατική και η δυναμική μοντελοποίηση είναι απλά δύο διαφορετικές όψεις του ίδιου νομίσματος. Και τα δύο μαζί εργάζονται για να εφαρμόσουν ένα σύστημα και στη δική μας περίπτωση το σύστημα SRS.

Σε αυτή την Ενότητα, θα επικεντρωθούμε στο κτίσιμο του στατικού μοντέλου για το σύστημα SRS, αφήνοντας την συζήτηση της δυναμικής μοντελοποίησης για την Ενότητα IV. Θα μάθεις:

- ✚ Τεχνάσματα για να εντοπίζεις την κατάλληλη κλάση και τα χαρακτηριστικά της.
- ✚ Πως θα προσδιορίζεις τις δομικές σχέσεις που υπάρχουν ανάμεσα σε αυτές τις κλάσεις.
- ✚ Πώς να απεικονίζεις/ παρουσιάζεις γραφικά αυτές τις πληροφορίες σαν ένα διάγραμμα κλάσης, χρησιμοποιώντας τον συμβολισμό UML.

Προσδιορίζοντας τις σωστές Κλάσεις

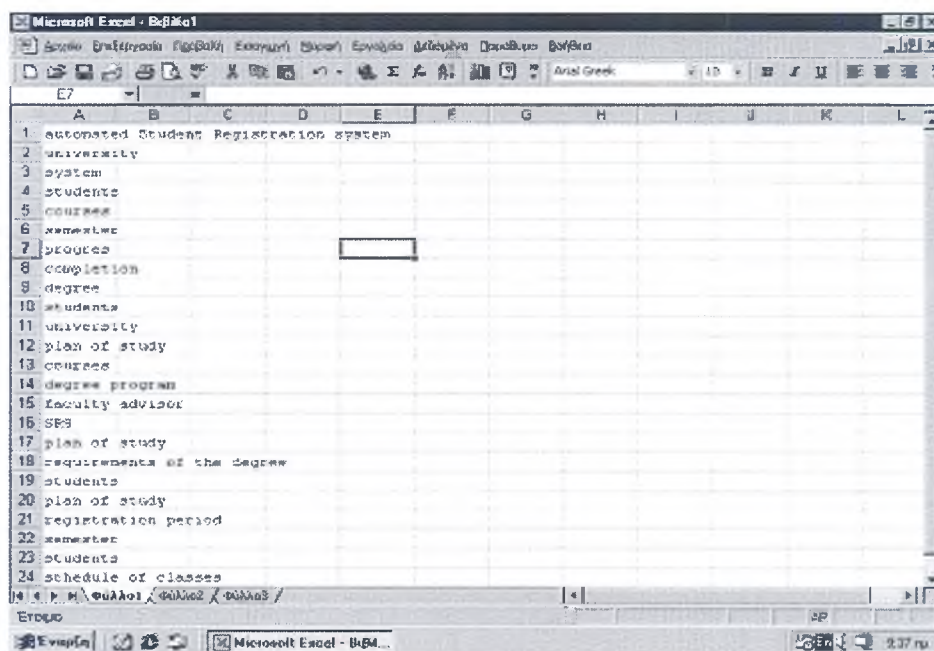
Η πρώτη μας πρόκληση στη μοντελοποίηση είναι ο προσδιορισμός των κλάσεων που θα χρειαστούμε σαν δομικά κομμάτια(blocks). Δυστυχώς, η διαδικασία προσδιορισμού των κλάσεων είναι κάπως "θαμπή". Βασίζεται κυρίως σε διαίσθηση, πείρα και στην οικειότητα που έχει ο χρήστης με το θέμα ή όπως είναι αλλιώς γνωστό "domain" του συστήματος που θα αναπτυχθεί. Έτσι πως θα ξεκινήσει η μοντελοποίηση? Κάποιος δοκίμασε μία αληθινή αλλά πολύπλοκη διαδικασία για τον προσδιορισμό των κλάσεων των υποψηφίων, η οποία είναι να χρησιμοποιήσουμε την μέθοδο "hunt and gather" που είναι "κυνηγώ και μαζεύω" ένα κατάλογο με όλα τα ουσιαστικά από την βιβλιογραφία της μελέτης και στη συνέχεια χρησιμοποιώντας την μέθοδο της απαλοιφής μικραίνουμε τον κατάλογο σε μια δέσμη κατάλληλων κλάσεων.

Στην περίπτωση του SRS , η βιβλιογραφία μας περιλαμβάνει:

- Τα χαρακτηριστικά των απαιτήσεων
- Το μοντέλο "Use Case" που ετοιμάστηκε στην Ενότητα II.

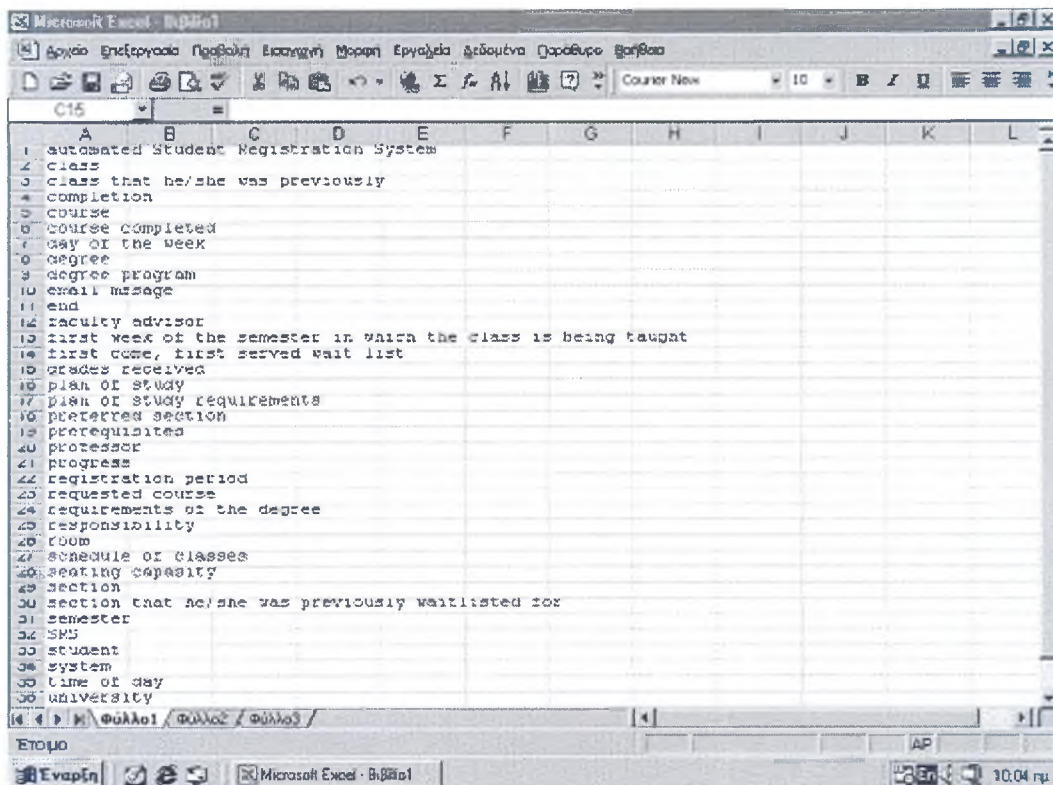
Ανάλυση Ουσιαστικών Προτάσεων

Ένα απλό spreadsheet εξυπηρετεί σαν ένα ιδανικό εργαλείο για την καταγραφή των αρχικών μας ευρημάτων. Απλά εισήγαγε τις ουσιαστικές προτάσεις σαν μία στήλη του κατάλογου με την σειρά που πραγματοποιούνται τα χαρακτηριστικά στο SRS. Μην ανησυχείς για την τυχόν παρουσία διπλών εγγραφών ή συνώνυμων. Θα τα δούμε σε λίγο. Το συνιστάμενο spreadsheet φαίνεται πιο κάτω:



Όπως βλέπουμε και πιο πάνω θα ήταν αδύνατο να διεξάγουμε μια εξονυχιστική ανάλυση των ουσιαστικών προτάσεων(περίπου 350 λέξεις σε μήκος). Εάν είσαι αντιμέτωπος με κάτι τέτοιο, καλό θα ήταν να γράψεις μια περίληψη που θα σε βοηθήσει και να κατανοήσεις καλύτερα τις απαιτήσεις του συστήματος και θα την χρησιμοποιήσεις αργότερα στο ξεκίνημα της μελέτης σου.

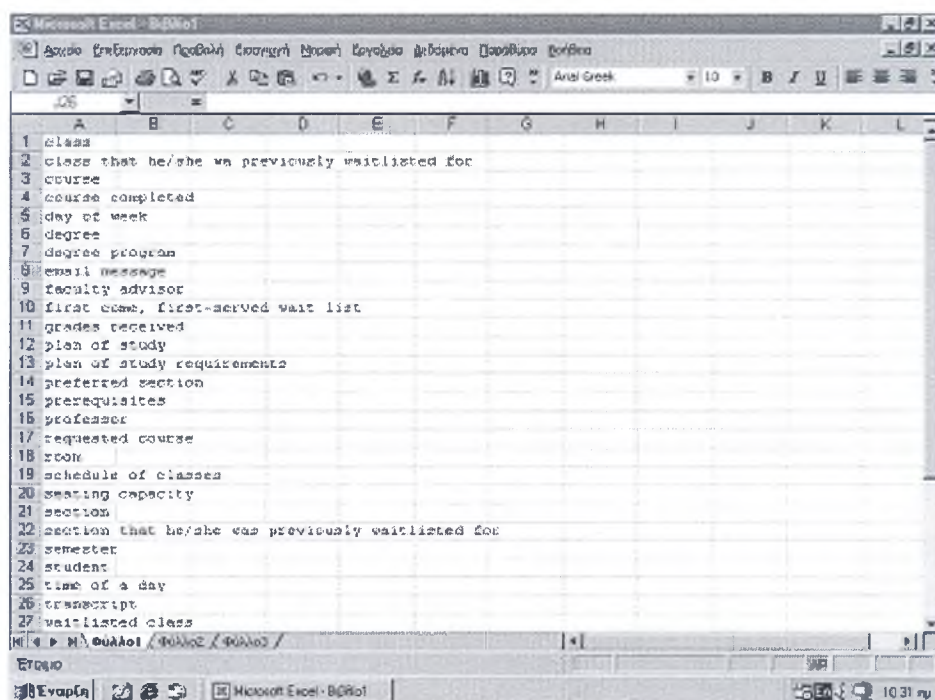
Μετά που θα έχεις δακτυλογραφήσει όλες τις προτάσεις κλειδιά στο spreadsheet, ταξινόμησε και κάνε απαλοιφή των διπλών εγγραφών. Στην τελική ανάλυση θέλουμε όλα τα ονόματα των κλάσεων μας στον ενικό αριθμό. Κάνοντας αυτό το SRS μας μικραίνει σε 38 αντικείμενα όπως πιο κάτω:



Να θυμάσαι προσπαθούμε να προσδιορίσουμε φυσικά ή νοητικά αντικείμενα που να αναφέρονται :

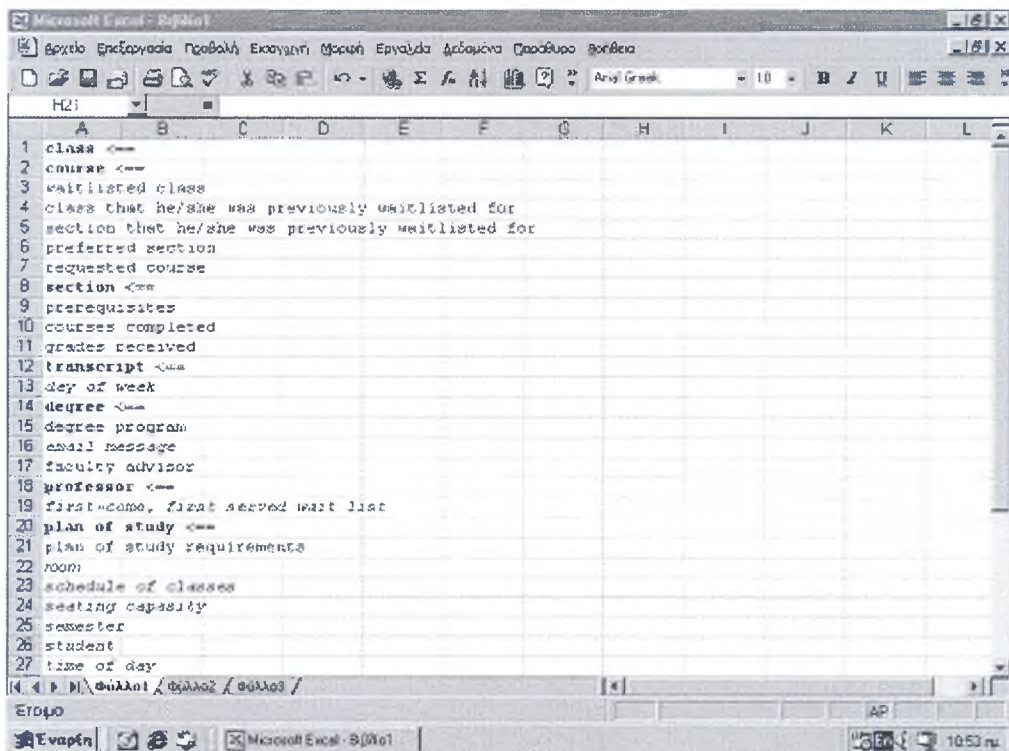
- Στο ίδιο το σύστημα SRS
- Στο πανεπιστήμιο. Επειδή ξεκινήσαμε να κτίζουμε το SRS μέσα στα πλαίσια ενός πανεπιστημίου, το πανεπιστήμιο ουσιαστικά βρίσκεται απέξω και περιλαμβάνει το SRS. Δεν χρειάζεται να χειριζόμαστε πληροφορίες για το πανεπιστήμιο εντός του SRS και έτσι μπορούμε να απαλείψουμε τον όρο πανεπιστήμιο "University" από τον κατάλογο. Σημειώστε όμως ότι αν κτίζαμε ένα σύστημα το οποίο χρειάζεται να ερευνά διάφορα πανεπιστήμια- ένα σύστημα που να ελέγχει τα προγράμματα σπουδών αποφοίτων στον τομέα της πληροφορικής στα 100 πρώτα καλύτερα πανεπιστήμια της χώρας- τότε θα χρειαζόταν να μοντελοποιήσουμε κάθε πανεπιστήμιο σαν ξεχωριστό αντικείμενο(object). Σε τέτοια περίπτωση θα πρέπει να κρατήσουμε το "University" στον κατάλογο τάξεων των υποψηφίων μας.
- Επίσης υπάρχουν κάποιοι συμπληρωματικοί κανόνες που δεν φαίνεται να ταιριάζουν με τον ορισμό ενός αντικειμένου όπως είναι: η συμπλήρωση -"completion", τέλος -"end", πρόοδος- "progress", υπευθυνότητα- "responsibility", περίοδος εγγραφής- "registration period" κ.α.

Ο κατάλογος μικραίνει περισσότερο σε 27 αντικείμενα, όπως φαίνεται πιο κάτω και αρχίζει τώρα να γίνεται ελεγχόμενος.



Η επόμενη εξέταση του πίνακα απαιτεί περισσότερη τέχνη. Χρειάζεται να ομαδοποιήσουμε εμφανές συνώνυμα, για να επιλέξουμε ένα κατευθυνόμενο συνώνυμο από κάθε ομάδα που να ταιριάζει περισσότερο στο να είναι όνομα της κλάσης.

Έχουμε ομαδοποιήσει όρους που φαίνεται να είναι συνώνυμα στον κατάλογο πιο κάτω. Με έντονο χρώμα είναι η όροι που τείνουμε να διαλέξουμε από κάθε ομάδα. Οι όροι με πλαγιαστά γράμματα αντιπροσωπεύουν αυτούς τους όρους για τους οποίους δεν έχουν εντοπιστεί συνώνυμα.



Ας επανεξετάσουμε τις επιλογές μας:

- Διαλέξαμε τον συντομότερο τύπο ισοδύναμων εκφράσεων που ήταν δυνατό - "degree" αντί του "degree program" και "plan of study" αντί του "plan of study requirements" - για να κάνουμε το σύστημα μας πιο σύντομο.
- Παρά το γεγονός ότι δεν υπάρχουν τέτοια συνώνυμα, ο συμβολισμός του "transcript" υπονοεί ένα κατάλογο μαθημάτων που έχουν συμπληρωθεί με τους αντίστοιχους βαθμούς
- Όταν διαλέγουμε ονόματα υποψηφίων για την κλάση θα πρέπει να αποφεύγουμε να διαλέγουμε ουσιαστικά που υπονοούν ρόλους ανάμεσα στα αντικείμενα(object). Ο ρόλος είναι κάτι που κατέχει ένα αντικείμενο που ανήκει σε μια κλάση A προς τη σχέση του με ένα άλλο αντικείμενο που ανήκει σε μια κλάση B. Για παράδειγμα, ο καθηγητής -"professor" έχει τον ρόλο του σύμβουλου του τμήματος, όταν ο "professor" συσχετιστεί με τον μαθητή - "student" μέσω της σχέσης σύμβουλου "advisee". Έστω και αν ο "professor" θα έχανε όλους τους συμβουλευόμενους του, δηλαδή χάνει τον ρόλο του σύμβουλου του τμήματος, θα είναι ακόμα καθηγητής "professor" για τον λόγο ότι εργοδοτείται από το πανεπιστήμιο.
- Μέσα στις διάφορες ομαδοποιήσεις που έχουμε υπονοούνται αρκετοί ρόλοι

- ◆ "waitlisted" υπονοεί ένα ρόλο στην σχέση μεταξύ του "Student" και του "course"
- ◆ "prerequisite" υπονοεί το ρόλο σε μια σχέση μεταξύ δύο μαθητών (student)
- ◆ "requested course" υπονοεί σχέση ένα ρόλο στη σχέση μεταξύ του "Student" και του "course"
- ◆ "preferred section" υπονοεί σχέση ένα ρόλο στη σχέση μεταξύ του "Student" και του "course"

Εξαλείφοντας όλους αυτούς τους ρόλους υπόδειξης, μένουμε μόνο με τρεις όρους : "class", "course" και "section".

Πριν απαλείψουμε τους πιο πάνω όρους και αφήσουμε μόνο ένα για συνώνυμο, ας σκεφτούμε τι θα θέλαμε να παραστήσουμε στο σύστημα μας:

- Ο συμβολισμός που τυπικά συνταυτίζεται με τον όρο "course" είναι η σειρά διαλέξεων κατά την διάρκεια του εξαμήνου, καθώς επίσης και οι εργασίες, εξετάσεις κ.τ.λ. που όλα σχετίζονται με κάθε μάθημα ξεχωριστά και τα οποία είναι μια μονάδα μόρφωσης προς την απόκτηση του πτυχίου.
- Οι όροι "class" και "section" από την άλλη, αναφέρονται γενικά στην προσφορά συγκεκριμένου κύκλου σπουδών σε ένα δεδομένο εξάμηνο, σε δεδομένη μέρα της εβδομάδας και σε δεδομένη ώρα της ημέρας.
- Υπάρχει μια σχέση ένα προς πολλά μεταξύ του "course" και "class/section". Ο ίδιος κύκλος σπουδών προσφέρεται δυναμικά πολλές φορές σε ένα δεδομένο εξάμηνο και για πολλά εξάμηνα, κατά την διάρκεια της ζωής αυτού του κύκλου σπουδών.

Επομένως, εφόσον το "class" και "Section" φαίνεται να είναι συνώνυμα, επιλέγουμε το "Section" για να αποφύγουμε αργότερα και την σύγχυση όταν αναφερόμαστε σε μια κλάση(class).

Έτσι μετά από όλα αυτά, ένας κατάλογος κλάσεων έχει αρχίσει να προβάλλεται:

§	Course
§	Plan of Study
§	Professor
§	Section
§	Student
§	Transcript

Έλεγχος των "Use Cases"

Ένα ακόμα πράγμα που χρειάζεται να κάνουμε πριν εμβαθύνουμε στην λίστα κλάσεων μας είναι να επισκεφθούμε την περίπτωση χρήσης με την οποία ασχοληθήκαμε στην ενότητα II και συγκεκριμένα τις κατηγορίες χρηστών-ηθοποιών που ορίσαμε για το SRS ούτως ώστε να ελέγξουμε την καταλληλότητά τους.

- ▶ Student
- ▶ Faculty
- ▶ Department Chair
- ▶ Registrar
- ▶ Billing System
- ▶ Admissions System
- ▶ Classroom Scheduling System

Αρμόζει σε κάποιο από αυτά να μοντελοποιηθεί σαν κλάση στο SRS? ΝΑΙ εάν κάποιος χρήστης συσχετίζεται με οποιονδήποτε ηθοποιό-χρήστη του τύπου 'A' και θα χρειαστεί να τον χειριστούμε(πρόσβαση και τροποποίηση) στις πληροφορίες που αφορούν ένα ηθοποιό του τύπου 'B'. Επίσης όταν ο 'A' εμφανίζεται δεσμευμένος στο SRS, τότε ο 'B' χρειάζεται να περιληφθεί σαν μια κλάση στο μοντέλο μας. Αυτό επιδεικνύεται με μερικά παραδείγματα που ακολουθούν:

- ✱ Όταν ο φοιτητής συνδεθεί με το SRS, μπορεί να χρειαστεί να επεξεργαστεί πληροφορίες σχετικές με το τμήμα? Η απάντηση είναι "ναι". Όταν ο φοιτητής επιλέγει ένα σύμβουλο για παράδειγμα, θα χρειαστεί να δει τις πληροφορίες που σχετίζονται με τα διάφορα μέλη του τμήματος για να διαλέξει τον κατάλληλο σύμβουλο. Έτσι ο ρόλος του Faculty χρήστη θα πρέπει να παριστάνει μια κλάση στο SRS και πραγματικά έχουμε να ορίσουμε την κλάση "Professor". Επίσης οι φοιτητές δεν ενδιαφέρονται για τις θέσεις των καρεκλών - Department Chair.
- ✱ Ακολουθώντας την ίδια λογική, θα χρειαστούμε να απεικονίσουμε τη δράση του φοιτητή "Student" (χρήστης-ηθοποιός) σαν κλάση επειδή όταν οι καθηγητές- "Professor" συνδεθούν με το SRS θα χρειάζονται τα αντικείμενα "Students" όταν εκτυπώνουν ένα κατάλογο του "Course" ή όταν κατανέμουν για παράδειγμα βαθμούς στους φοιτητές- "Students".

Επειδή ο φοιτητής -"Student" εμφανίζεται στη λίστα των υποψηφίων μας είμαστε και εδώ καλυμμένοι.

- ✱ Όταν οποιοσδήποτε από τους χρήστες "Faculty", "Students", "Registrar", "Billing System", "Admissions System" ή "Classroom Scheduling System" ενωθούν με το SRS, θα υπάρχει ανάγκη να επεξεργαστούν πληροφορίες για τον "Registrar"? Όχι, τουλάχιστον σύμφωνα με τις ανάγκες του SRS που έχουμε δι μέχρι τώρα. Επομένως δεν χρειάζεται να μοντελοποιήσουμε την δράση του "Registrar" σαν κλάση.
- ✱ Το ίδιο ισχύει και για το "Billing", "Admissions" και "Classroom Scheduling System": χρειάζονται παρασκηνιακή πρόσβαση σε πληροφορίες που διευθύνονται από το SRS, αλλά κανείς που συνδέεται με το SRS αναμένει να είναι ικανός να επεξεργάζεται κάποιο από τα τρία αυτά συστήματα απευθείας. Έτσι δεν χρειάζεται να παριστάνονται από τις κλάσεις ορισμού στο SRS.

Επομένως, η προτεινόμενη λίστα υποψηφίων παραμένει ως έχει:

- ◆ Course
- ◆ Plan of study
- ◆ Professor
- ◆ Section
- ◆ Student
- ◆ Transcript

Λεξικό Δεδομένων για το SRS

Αρχικά στις προσπάθειες ανάλυσης, είναι σημαντικό να ξεκαθαρίσουμε και να αρχίσουμε να συγγράφουμε την δική μας ορολογία. Ένα λεξικό δεδομένων είναι ιδανικό για αυτό το λόγο. Για κάθε κλάση υποψηφίου, το λεξικό δεδομένων θα πρέπει να περιλαμβάνει ένα απλό ορισμό για του τι σημαίνει αυτό το αντικείμενο στη χρήση του στο σύστημα σαν ολότητα.

Πιο κάτω είναι ο κατάλογος του λεξικού δεδομένων για το SRS:

Μάθημα-"Course" : μία σειρά διαλέξεων(ανατιθέντων έργων και εξετάσεων), διάρκειας ενός εξαμήνου, που σχετίζεται με μία συγκεκριμένη θεματική ενότητα και που τυπικά συνδέεται με ένα συγκεκριμένο αριθμό διδακτικών μονάδων. Για παράδειγμα, το "Εισαγωγή στα Αντικείμενα" είναι ένα μάθημα που απαιτείται για το master στην Τεχνολογία των Πληροφοριακών Συστημάτων.

Πρόγραμμα Σπουδών-"Plan of Study" : μία λίστα μαθημάτων που ένας φοιτητής πρόκειται να πάρει για να εκπληρώσει τις απαιτήσεις μαθημάτων ενός συγκεκριμένου πτυχίου.

Καθηγητής-"Professor" : ένα μέλος ενός σώματος καθηγητών που διδάσκει και/ ή συμβουλεύει τους φοιτητές.

Διδακτική τομή(Μάθημα)-"Section" : η προσφορά ενός συγκεκριμένου μαθήματος ενός συγκεκριμένου εξαμήνου μίας συγκεκριμένης μέρας της εβδομάδας και σε μία συγκεκριμένη ώρα της ημέρας(π.χ. ένα μάθημα "εισαγωγή στα Αντικείμενα" ,όπου διδάχτηκε στο Εαρινό Εξάμηνο του 2001, τις δευτέρες, μεταξύ 1:00-3:00 μ.μ.

Φοιτητής-"Student" : ένα άτομο που είναι γραμμένο στο πανεπιστήμιο και που είναι νόμιμος να εγγραφεί σε ένα ή περισσότερα μαθήματα(Sections).

Αντίγραφο αναλυτικής βαθμολογίας-"Transcript" : μία εγγραφή όλων των μαθημάτων ανά ημερομηνία ενός συγκεκριμένου φοιτητή σε αυτό το πανεπιστήμιο, που περιλαμβάνει σε ποιο εξάμηνο επιλέχτηκε κάθε μάθημα, το βαθμό που πήρε, τις μονάδες που αντιστοιχούν σε αυτό το μάθημα. Επίσης, περιλαμβάνει το συνολικό αριθμό διδακτικών μονάδων που έχουν αποκτηθεί και το μέσο όρο βαθμολογίας του φοιτητή.

Καθορισμός των Συσχετίσεων μεταξύ Κλάσεων

Όταν κατασταλάξουμε σε μία αρχική λίστα κλάσεων των υποψηφίων, το επόμενο βήμα είναι να καθορίσουμε πως αυτές οι κλάσεις συσχετίζονται. Για να το κάνουμε αυτό πηγαίνουμε πίσω στο περιγραφικό μας κείμενο (το οποίο αποτελείται από τα χαρακτηριστικά απαιτήσεων του SRS, Use Case και το λεξικό δεδομένων) για να μελετήσουμε ρηματικές φράσεις αυτή τη φορά.

Ο στόχος μας κοιτάζοντας τις ρηματικές φράσεις είναι να επιλέξουμε αυτές που εισηγούνται τις δομικές σχέσεις - associations, aggregations και inheritance - και να απαλείψουμε ή να αγνοήσουμε αυτές που παριστάνουν μεταβατικές πράξεις και συμπεριφορά.

- ✱ Για παράδειγμα, όταν ο φοιτητής επιλέγει ένα σύμβουλο καθηγητή, αυτή είναι μία μεταβατική πράξη αλλά το αποτέλεσμα αυτής της πράξης είναι μια διαρκή δομική σχέση μεταξύ "Professor" και "Student", το οποίο μπορεί να μοντελοποιηθεί δια μέσου της συσχέτισης "ο Professor" συμβουλεύει τον "Student"
- ✱ Επίσης ο καθηγητής σαν σύμβουλος του φοιτητή, συναντά το φοιτητή, απαντά σε τυχόν ερωτήσεις του, εισηγείται κάποια μαθήματα, εγκρίνει το πλάνο σπουδών του κτλ. Αυτές είναι συμπεριφορές από πλευράς "Professor" δρώντας στο ρόλο του συμβούλου, αλλά δεν οδηγεί απευθείας σε μία νέα σχέση που να δημιουργείται ανάμεσα στα αντικείμενα.

Μία συμπληρωματική τεχνική για τον προσδιορισμό και την καταγραφή του ποιες θα πρέπει να είναι οι σχέσεις μεταξύ κλάσεων είναι να δημιουργήσουμε ένα πίνακα $n \times n$ όπου $n =$ ο αριθμός των υποψήφιων κλάσεων που ορίσαμε. Ονομάστε τις σειρές και τις στήλες με τα ονόματα των κλάσεων όπως φαίνεται στον άδειο πίνακα πιο κάτω:

	Section	Course	Plan of Study	Professor	Student	Transcript
Section						
Course						
Plan of Study						
Professor						
Student						
Transcript						

Μετά συμπληρώστε τον πίνακα όπως πιο κάτω:

✱ Σε κάθε τετραγωνάκι του πίνακα, παραθέτουμε όλες τις συσχετίσεις που μπορούν να οριστούν μεταξύ των κλάσεων. Για παράδειγμα στο τετραγωνάκι με σειρά "Student" και στήλη "Section" θα έχουμε τρεις πιθανές/ δυνατές συσχετίσεις:

- Ένας φοιτητής είναι στη λίστα αναμονής για ένα "Section"
- Ένας φοιτητής εγγράφεται για ένα "Section" ή αλλιώς αυτό θα μπορούσε να εκφραστεί σαν «Ο φοιτητής παρακολουθεί επί του παρόντος ένα "Section"»
- Ένας φοιτητής έχει προηγουμένως πάρει ένα "Section" : Αυτή η τρίτη συσχέτιση είναι σημαντική εάν σχεδιάζουμε το ιστορικό όλων των κλάσεων που έχουν παρθεί από τους φοιτητές κατά την διάρκεια της φοιτητικής ζωής, το οποίο θα πρέπει να κάνουμε εάν προτιθέμεθα να κάνουμε transcript on line.

✱ Σημειώνουμε ένα στοιχείο με «x» εάν δεν υπάρχουν γνωστές σχέσεις μεταξύ των κλάσεων υπό ερώτηση, ή αν οι δυναμικές σχέσεις μεταξύ των κλάσεων είναι άσχετες. Για παράδειγμα, έχουμε μαρκάρει τα στοιχεία που παριστάνουν την τομή μεταξύ "Professor" και "Course" με ένα x, αν και υπάρχει μια δυνατή σχέση- ο "Professor" είναι ικανός να διδάξει "Course"- επειδή δεν είναι δυνατό προς την αποστολή του SRS.

- Ξέρουμε ότι όλες οι συσχετίσεις είναι κατά κληρονομικότητα αμφίδρομες. Αυτό υπονοεί ότι αν ένα στοιχείο στη σειρά j, στήλη k δείχνει μία ή

περισσότερες συσχετίσεις, τότε το στοιχείο σειρά k , στήλη j θα πρέπει να αντανakλά αμοιβαία σχέση αυτών. Σε τέτοιες περιπτώσεις όπου η αβεβαιότητα συσχέτισης θα είναι αδέξιο να εκφραστεί, απλά θα βάζουμε ένα $\sqrt$.

	Section	Course	Plan of Study	Professor	Student	Transcript
Section	X			is taught by	$\sqrt$	included in
Course	$\sqrt$	prerequisite for	is called for by	X	$\sqrt$	X
Plan of Study	X	calls for	X	X	observed by	X
Professor	teaches	X	X	X	advises; teaches	X
Student	registered for; waitelisted for; has previously taken	plans to take	observes	is advised by; studies under	X	owns
Transcript	includes	X	X	X	belongs to	X

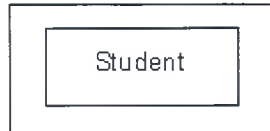
Σύντομα, θα σκιαγραφήσουμε αυτές τις συσχετίσεις σε γραφική μορφή. Επί του παρόντος θα θέλαμε να πάμε πίσω και να επιμικρύνουμε τα δεδομένα του λεξικού μας για να επεξηγήσουμε τι σημαίνει η κάθε μία από αυτές. Εδώ είναι ένα παράδειγμα:

Calls for (Ένα πλάνο σπουδών καλεί ένα μάθημα):

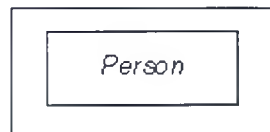
Για να δείξουμε ότι ο φοιτητής θα ικανοποιήσει τις προαπαιτήσεις για το επιλεγμένο πρόγραμμα σπουδών, ο φοιτητής θα πρέπει να τυποποιήσει το πλάνο σπουδών. Αυτό το πλάνο σπουδών απλώνει όλα τα προγράμματα σπουδών που προτίθεται να πάρει και πιθανόν ορίζει σε ποιο εξάμηνο ο φοιτητής επιθυμεί να συμπληρώσει τον κάθε κύκλο σπουδών.

Κλάσεις, Ιδιότητες και Μέθοδοι

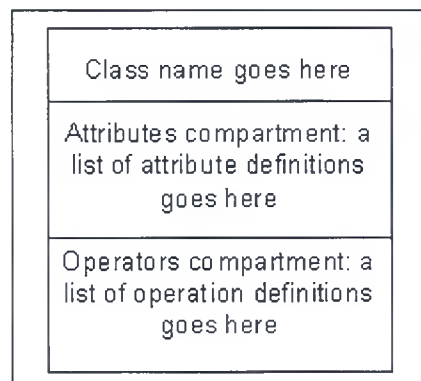
Παριστάνουμε τις κλάσεις με ορθογώνια. Όταν αρχικά αντιληφθούμε μια κλάση- πριν ακόμα αντιληφθούμε ποια από τα γνωρίσματα ή μέθοδοι θα την χαρακτηρίζουν- τοποθετούμε απλά το όνομα της κλάσης στο ορθογώνιο, όπως για παράδειγμα:



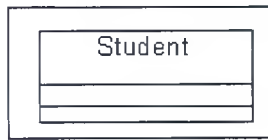
Μία abstract class δηλώνεται με πλάγια γράμματα, για παράδειγμα:



Όταν είμαστε έτοιμοι να συλλογιστούμε τις ιδιότητες(attributes) ή/ και τις μεθόδους(methods) μιας κλάσης, διαιρούμε το ορθογώνιο της κλάσης σε τρία τμήματα- το τμήμα με το όνομα της κλάσης, το τμήμα με τα γνωρίσματα και το τμήμα με τις λειτουργίες- όπως φαίνεται πιο κάτω. Σημειώστε ότι προτιμούμε την ονοματολογία "operations" αντί "methods" για να ενισχύσουμε την άποψη ότι το παραγόμενο διάγραμμα προτίθεται να είναι ανεξάρτητο από την γλώσσα προγραμματισμού.

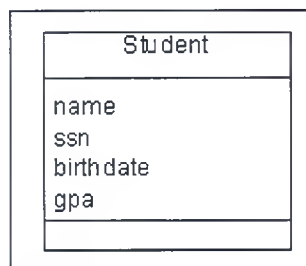


Κάποια Εργαλεία CASE αυτόματα επιδεικνύουν τρία τμήματα όταν αρχικά δημιουργείται μία κλάση, έστω και αν δεν έχουμε ορίσει ακόμα κάποια γνωρίσματα ή μεθόδους.

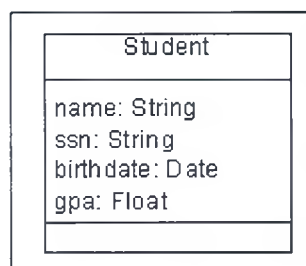


Ενώσω αρχίζουμε να προσδιορίζουμε τις ιδιότητες(attributes) και τις μεθόδους(operations) που χρειάζονται για την συγκεκριμένη κλάση, μπορούμε να τα προσθέσουμε αυτά στο διάγραμμα με πολλές ή λιγότερες λεπτομέρειες από αυτές που μας ενδιαφέρουν,

- Μπορούμε να διαλέξουμε απλά να παραθέσουμε σε κατάλογο τα ονόματα των ιδιοτήτων:



ή μπορούμε να ορίσουμε τα ονόματα παράλληλα με τους τύπους τους:

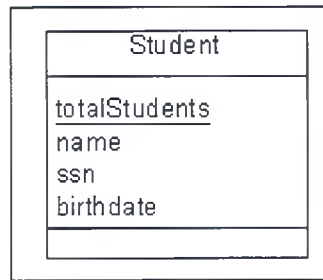


Θα μπορούσαμε ακόμα εάν επιθυμούμε να ορίσουμε μία αρχική τιμή έναρξης για μία ιδιότητα όπως είναι:

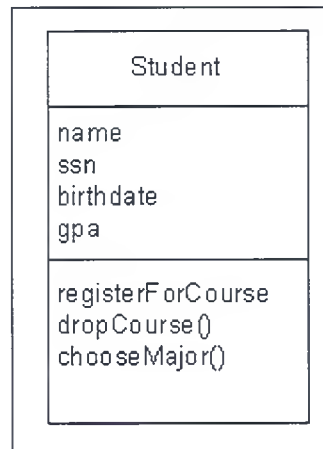
`gpa : Float = 0.0`

αν και αυτό είναι λιγότερο συχνό.

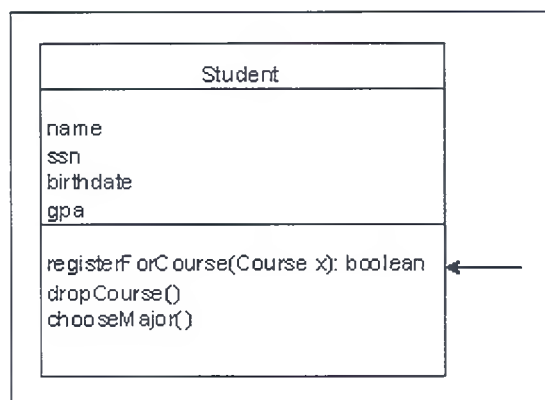
- Στατικές ιδιότητες (Static attributes) ορίζονται υπογραμμισμένες:



- Θα μπορούσαμε να επιλέξουμε να παραθέσουμε τα ονόματα σε κατάλογο (τυπικά με το πρόθεμα «()» για να υπονοεί τη φύση τους σαν μέθοδοι) στο τμήμα λειτουργιών της κλάσης.



- Ή μπορούμε προαιρετικά να διαλέξουμε να χρησιμοποιήσουμε ένα διευρυμένο τύπο ορισμού λειτουργιών όπως έχουμε για το registerForCourse() παρακάτω:



Σημειώστε ότι η επίσημη σύνταξη για τα λειτουργικά χαρακτηριστικά σε ένα UML διάγραμμα κλάσης

methodName(optional argument List): returnType

διαφέρει από την σύνταξη που συνηθίσαμε να βλέπουμε για τις μεθόδους υπογραφής της "JAVA"

```
returnType methodName(optional argument list);
```

Είναι συχνά όχι και τόσο πρακτικό να δείξουμε όλες τις ιδιότητες και μεθόδους κάθε κλάσης σε ένα διάγραμμα κλάσης, επειδή το διάγραμμα θα γίνει τόσο γεμάτο που θα χάσει την δύναμη του σαν εργαλείο επικοινωνίας. Επίσης, επειδή το τμήμα των χαρακτηριστικών ή λειτουργιών της κλάσης είναι άδειο δεν σημαίνει ότι δεν υπάρχουν τέτοιου είδους γνωρίσματα που να σχετίζονται με αυτή τη κλάση. Απλά μπορεί να σημαίνει ότι το μοντέλο ακόμα ανελίσσεται.

Σχέσεις μεταξύ Κλάσεων

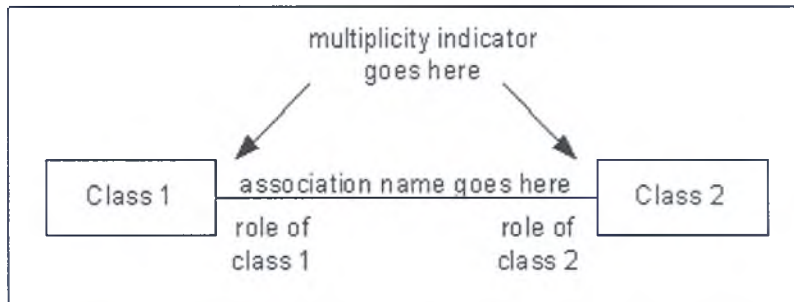
Οι σχέσεις μεταξύ των κλάσεων ενός συστήματος μπορεί να είναι συσχετίσεις, γενικεύσεις και διαφόρων ειδών εξαρτήσεις.

Ας εξερευνήσουμε πως κάθε μία από αυτές τις σχέσεις παριστάνεται γραφικά.

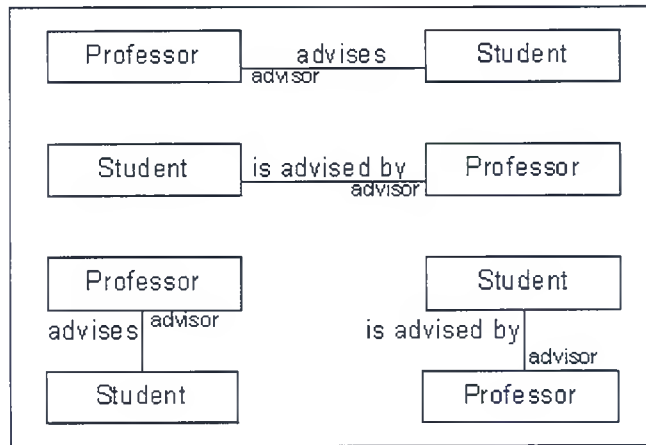
□ Δυαδικές Συσχετίσεις(Binary associations)

Με άλλα λόγια σχέσεις μεταξύ δύο διαφορετικών κλάσεων. Επιδεικνύονται σχεδιάζοντας μία γραμμή ανάμεσα στα δύο ορθογώνια που παριστάνουν τις κλάσεις και ονομάζοντας τη γραμμή με το όνομα της συσχέτισης. Τα ονόματα των ρόλων μπορούν να δηλωθούν στα δύο άκρα της γραμμής συσχέτισης εάν προσθέτουν αξία στο μοντέλο, διαφορετικά θα πρέπει να παραλείπονται.

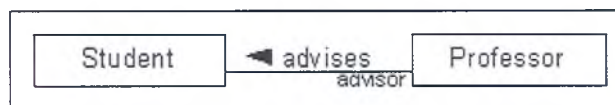
Επίσης σημειώνουμε σε κάθε άκρο της γραμμής τον κατάλληλο δείκτη πολλαπλότητας για να υποδηλώσουμε τον αριθμό των στιγμιότυπων της κλάσης που μπορούν να συσχετιστούν με το στιγμιότυπο της άλλης κλάσης. Όπως για παράδειγμα μπορεί να είναι ένα προς πολλά ή πολλά προς πολλά κ.τ.λ. Θα μιλήσουμε αναλυτικότερα σχετικά με αυτό αργότερα.



Όλες οι συσχετίσεις μπορούν να είναι αμφίδρομες(ο συμβολισμός είναι μία γραμμή χωρίς βέλη) στη προσπάθεια μοντελοποίησης στο παρόν στάδιο και δεν πειράζει με ποια σειρά οι συμμετέχουσες κλάσεις τακτοποιούνται στο διάγραμμα κλάσεων. Έτσι για να ορίσουμε την σχέση "a Professor advises a student" οι ακόλουθοι γραφικοί συμβολισμοί τίθενται υπόψη εξίσου:



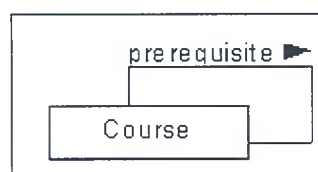
Με το OMT, τον προγενέστερο συμβολισμό της UML, είχαμε την ευκαιρία να περιγράψουμε τις συσχετίσεις έτσι ώστε τα ονόματα τους να έχουν νόημα όταν διαβάζουμε ένα διάγραμμα από αριστερά προς τα δεξιά ή από πάνω προς τα κάτω. Υπήρχε επομένως το κίνητρο να διαρρυθμίσουμε τις κλάσεις στο διάγραμμα μας με οποιοδήποτε τρόπο θα έκανε την ονομασία σχέσεων πιο ήπια. Επιτυγχάνοντας μία ιδανική τοποθέτηση των κλάσεων για λόγους απλούστευσης του ονόματος των συσχετίσεων σε ένα διάγραμμα δεν είναι συχνά δυνατό σε ένα πολύπλοκο διάγραμμα. Εν τούτοις το UML έχει εστιάσει τον απλό συμβολισμό της χρήσης μικρών τόξων για να αντανakλά την κατεύθυνση στην οποία μία συσχέτιση ερμηνεύεται, παρέχοντας μας πολύ περισσότερη ελευθερία για τον τρόπο τοποθέτησης των κλάσεων πάνω στο διάγραμμα:



Με το UML, ανεξάρτητα από το πώς τοποθετούνται οι δύο κλάσεις πιο πάνω, μπορούμε πάντοτε να ονομάσουμε την σχέση "advises".

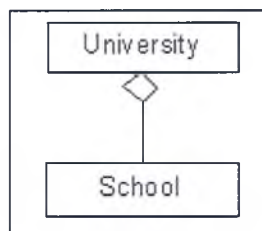
□ Αντανakλαστικές Συσχετίσεις(Reflexive associations)

Είναι σχέσεις μεταξύ δύο διαφορετικών αντικειμένων που ανήκουν στην ίδια κλάση. Σχεδιάζονται με μία γραμμή η οποία επανέρχεται κλείνοντας το κύκλωμα στο ίδιο τρίγωνο το κύκλωμα στην ίδια κλάση από όπου ξεκίνησε. Για παράδειγμα, για να παραστήσουμε την σχέση "a Course is a prerequisite for a(different) Course", θα χρησιμοποιήσουμε το πιο κάτω συμβολισμό:

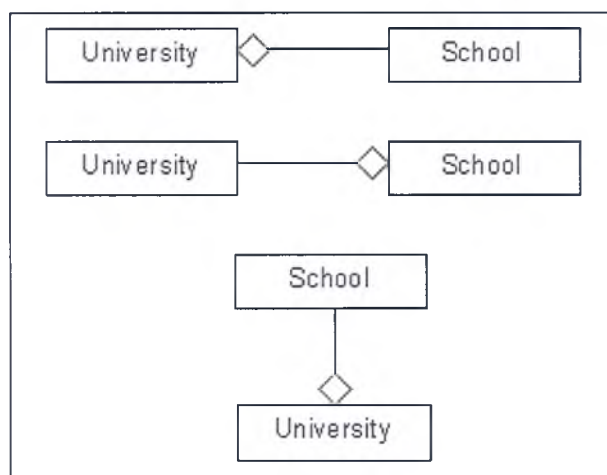


□ Συσσωμάτωση

Είναι μία συσχέτιση που τυγχάνει να υπονοεί περιεκτικότητα. Διαφοροποιείται από την συνηθισμένη συσχέτιση με την τοποθέτηση ενός ρομβοειδούς στο τέλος της γραμμής συσχέτισης που αγγίζει την κλάση που το περιέχει. Για παράδειγμα, για να σκιαγραφήσουμε ότι το "University" αποτελείται από διάφορες σχολές(School)- Νομική Σχολή, Μαθηματική Σχολή κ.α.- θα χρησιμοποιήσουμε τον ακόλουθο συμβολισμό:

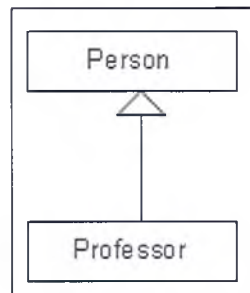


Μία σχέση συσσωμάτωσης μπορεί να κατευθύνεται σε πολλές κατευθύνσεις, ενόσω το ρομβοειδές είναι κατάλληλα τοποθετημένο στην περιέχουσα(containing) κλάση.

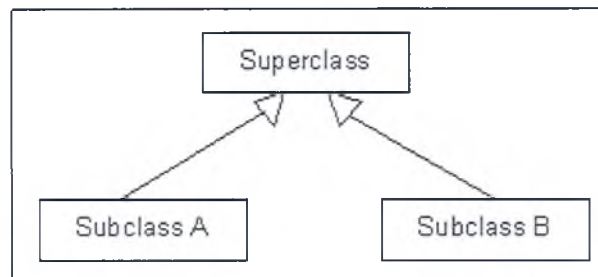


□ Κληρονομικότητα(γενίκευση/ εξειδίκευση)

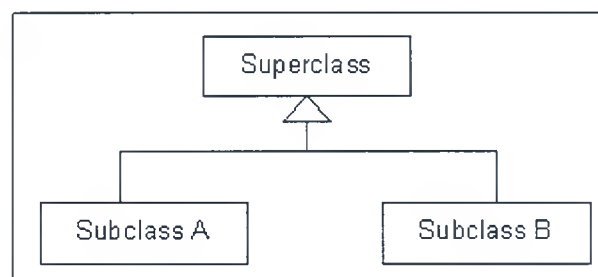
Συμβολίζεται με ένα τρίγωνο-βέλος στο ένα άκρο που αντιστοιχεί στη πατρική κλάση της σχέσης.



Όπως με την συσσωμάτωση, όταν δύο ή περισσότερες κλάσεις παριστάνουν υποκλάσεις της ίδιας γονικής κλάσης, έτσι και κάθε υποκλάση εμπλέκεται σε μία ξεχωριστή κληρονομική σχέση με την γονική.



αλλά συχνά ενώνουμε τη γραμμή κληρονομικότητας σε ένα μόνο δομικό κατασκεύασμα όπως φαίνεται.

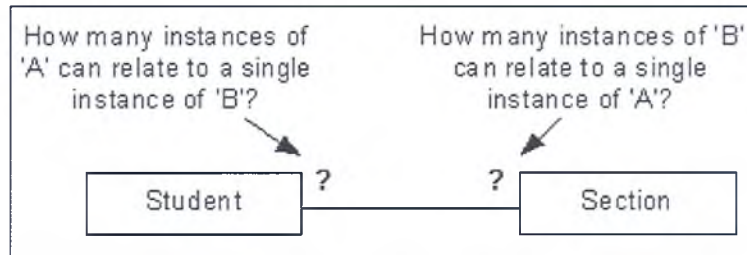


Βλέποντας τη δεύτερη περίπτωση δεν εννοείται να υπονοηθεί κάτι διαφορετικό από ότι στη πρώτη περίπτωση. Και στις δύο περιπτώσεις οι κλάσεις "Subclass A" και "Subclass B" συνδέονται με την ίδια γονική κλάση. Η δεύτερη περίπτωση είναι απλά ένας τρόπος για να καθαρίσουμε το διάγραμμα.

Πολλαπλότητα

Όταν σχεδιάζουμε ένα διάγραμμα κλάσης:

- Σημειώνουμε τον αριθμό των στιγμιότυπων της κλάσης "B" που μπορούν να συσχετιστούν με ένα στιγμιότυπο της κλάσης "A".
- Σημειώνουμε τον αριθμό των στιγμιότυπων της κλάσης "A" που μπορούν να συσχετιστούν με ένα στιγμιότυπο της κλάσης "B".

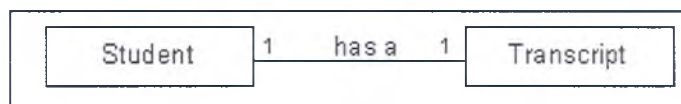


Οι διάφορες δυνατές πολλαπλότητες και οι ερμηνείες του είναι:

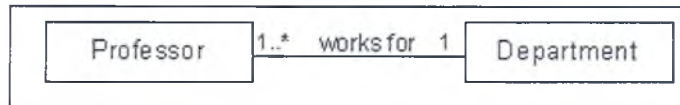
*	Οποιοσδήποτε αριθμός αντικειμένων
1	Ακριβώς ένα αντικείμενο
n	Ακριβώς n αντικείμενα (όπου n ακέραιος)
0..1	Μηδέν ή ένα αντικείμενα (υποδηλώνει ότι η συσχέτιση είναι προαιρετική)
n..m	Εύρος τιμών με μικρότερη δυνατή τιμή n και μέγιστη m
n,m	Διακριτοί συνδυασμοί

Εδώ βλέπουμε μερικά παραδείγματα UML:

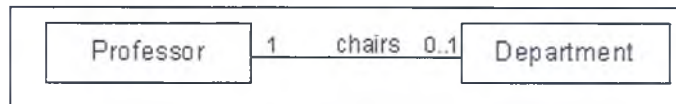
- ◆ Ένας φοιτητής "Student" έχει ένα μόνο αντίγραφο "Transcript" και το συγκεκριμένο αντίγραφο "Transcript" ανήκει σε ένα μόνο φοιτητή.



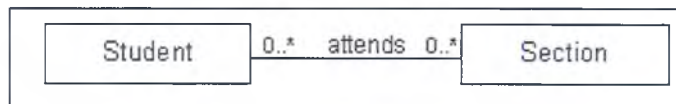
- ◆ Ένας καθηγητής "Professor" απασχολείται σε ένα μόνο τμήμα "Department" αλλά το τμήμα "Department" απασχολεί περισσότερους από ένα καθηγητές "Professor".



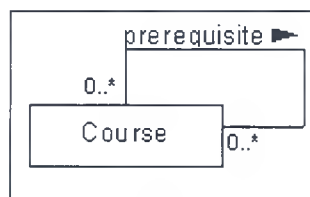
- ◆ Ένας καθηγητής "Professor" μπορεί κατά επιλογή δική του να διευθύνει σε περισσότερα από ένα τμήματα "Department" αλλά κάθε τμήμα "Department" έχει ένα ή καθόλου διευθυντή.



- ◆ Ένας φοιτητής "Student" παρακολουθεί πολλά(ένα ή περισσότερα) μαθήματα "Section" και τα μαθήματα "Section" παρακολουθούνται από πολλούς(ένα ή περισσότερους) φοιτητές "Student".



- ◆ Ένα μάθημα "Course" είναι προϋπόθεση για πολλά στη συνέχεια μαθήματα "Course" που θα επακολουθήσουν και μπορεί επίσης ένα μάθημα "Course" να έχει πολλά προϋποτιθέμενα μαθήματα "Course".



- ◆ Ένα φοιτητικό πλάνο σπουδών "PlanOfStudy" αποτελείται από πολλά μαθήματα "Course". Κάθε δοσμένο μάθημα "Course" μπορεί να επάγεται σε πολλά διαφορετικά φοιτητικά πλάνα "PlanOfStudy".



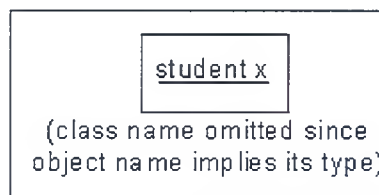
Διάγραμμα Αντικειμένων

Όταν περιγράφουμε πώς τα αντικείμενα(objects) αλληλεπιδρούν, κάποτε βρίσκουμε βοηθητικό να σκιαγραφήσουμε ένα σενάριο (scenario) για συγκεκριμένο αντικείμενο. Ένα στιγμιότυπο ή αντικείμενο μοιάζει περισσότερο με μία κλάση συμβολισμού UML, με τις κύριες διαφορές να είναι:

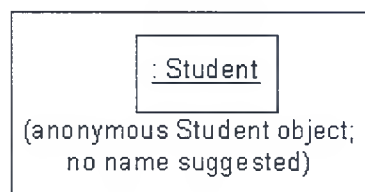
- Παρέχουμε ταυτόχρονα και το όνομα και τον τύπο του αντικειμένου να διαχωρίζονται από μία στήλη. Υπογραμμίζουμε το κείμενο για να δώσουμε έμφαση στο ότι αυτό είναι ένα αντικείμενο και όχι μία κλάση.



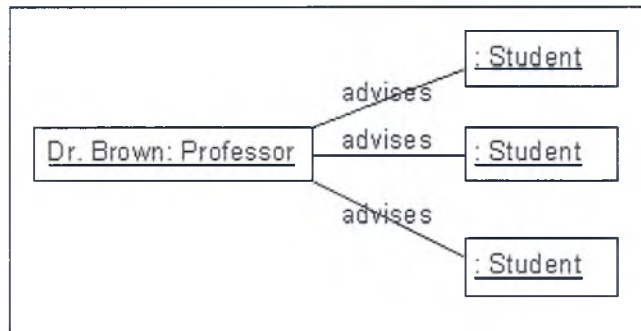
- Ο τύπος των αντικειμένων θα μπορούσε να παραληφθεί εάν είναι εμφανές από το όνομα του αντικειμένου. Για παράδειγμα, το όνομα "student x" υπονοεί ότι το αντικείμενο υπό εξέταση ανήκει στη κλάση "Student".



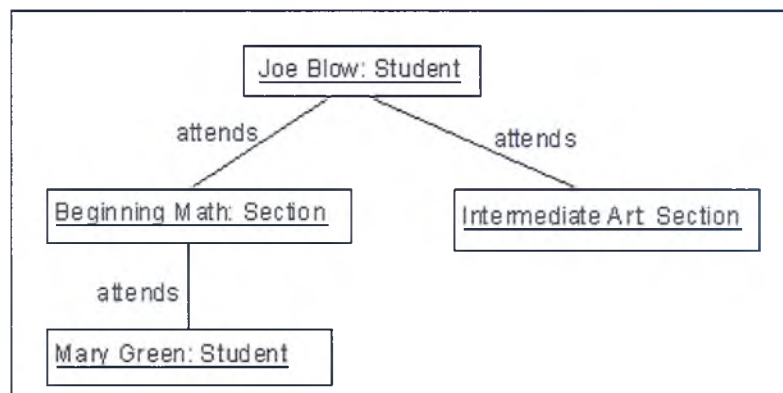
- Εναλλακτικά το όνομα του αντικείμενου θα μπορούσε να παραληφθεί εάν θέλαμε να αναφερθούμε σε μία γενική μορφή αντικειμένου ενός δεδομένου τύπου. Τέτοιου είδους αντικείμενο είναι γνωστό σαν "anonymous object". Σημειώστε ότι πρέπει να περιβάλλουμε το όνομα της κλάσης με άνω και κάτω τελεία (:) σε τέτοια περίπτωση.



- Επομένως, εάν θέλουμε να δείξουμε ότι ένας καθηγητής, ο Dr.Brown, είναι σύμβουλος για τρεις φοιτητές, θα μπορούσαμε να δημιουργήσουμε το ακόλουθο διάγραμμα αντικειμένου:

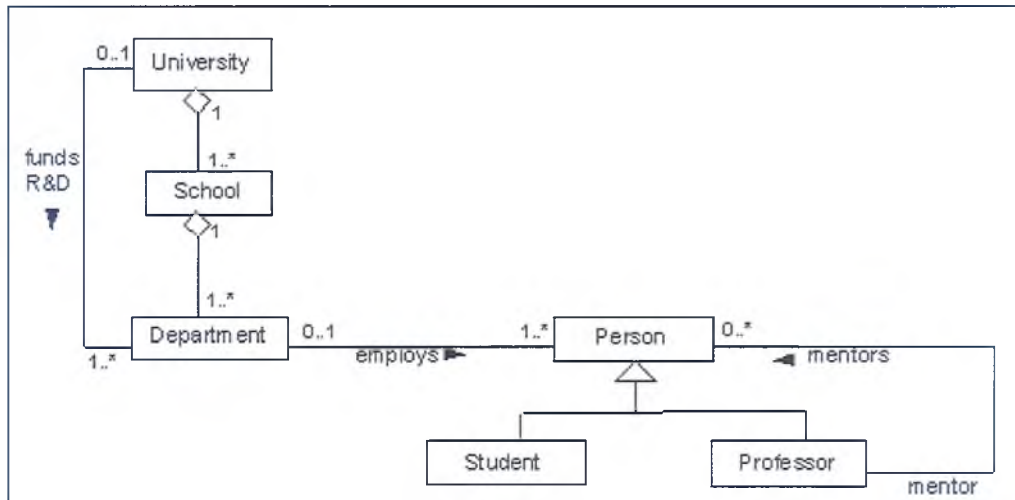


ή να παραστήσουμε ότι ένας φοιτητής "Student" με όνομα "Joe Blow" αυτό το εξάμηνο παρακολουθεί δύο μαθήματα "Section", ένα εκ των οποίων παρακολουθείται και από την φοιτήτρια "Mary Green".



"Ανακατεύοντας και Ταιριάζοντας" τους Συμβολισμούς Σχέσεων

Είναι δυνατό να περιδιαβούμε τους διάφορους τύπους σχέσεων με ένα κάπως πολύπλοκο τρόπο. Για να εκτιμήσουμε αυτό το γεγονός ας μελετήσουμε το ακόλουθο μοντέλο για να δούμε τι μας λέει:



- Πρώτα από όλα βλέπουμε κάποιες γνωστές σχέσεις συσσωμάτωσης και κληρονομικότητας.

∅ Χρήση συσσωμάτωσης-πάνω αριστερή γωνία του διαγράμματος: Δείχνει ότι ένα πανεπιστήμιο "University" αποτελείται από ένα ή περισσότερες σχολές "School" και ότι κάθε σχολή αποτελείται από ένα ή περισσότερα τμήματα "Department", αλλά κάθε τμήμα συνδέεται μόνο με μια σχολή και κάθε σχολή συνδέεται μόνο με ένα πανεπιστήμιο.

∅ Χρήση κληρονομικότητας- κάτω δεξιά γωνία του διαγράμματος: Δείχνει ότι ένα άτομο είναι η κοινή υπερκλάση για το Φοιτητή "Student" και το καθηγητή "Professor". Εναλλακτικά, εκφραζόμενο με άλλο τρόπο: "Ο Φοιτητής είναι άτομο όπως και ο καθηγητής"

- Η πρώτη ενδιαφέρον χρήση συσχέτισης που παρατηρούμε είναι ότι μία σχέση θα μπορεί να χρησιμοποιηθεί για να συσχετίσει κλάσεις σε διαφορετικά επίπεδα σε μία συσσωμάτωση όπως στη χρήση της σχέσης funds R&D (Έρευνα και ανάπτυξη) που συσχετίζει τις κλάσεις "University" και "Department". Αυτό δείχνει ότι το πανεπιστήμιο

χρηματοδοτεί ένα ή περισσότερα τμήματα για σκοπό έρευνας & ανάπτυξης και ότι ένα τμήμα μπορεί να χρηματοδοτηθεί ή όχι.

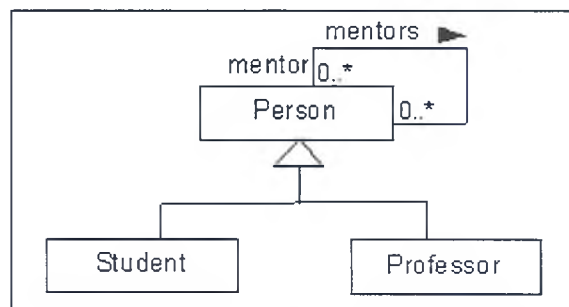
- Στη συνέχεια, σημειώνουμε τη χρήση της συσχέτισης "employs" για να συσχετίσουμε τις κλάσεις "Department" και "Person", δείχνοντας ότι σε ένα τμήμα δουλεύουν ένα ή περισσότερα άτομα "Person" αλλά ένα συγκεκριμένο άτομο μπορεί να εργάζεται ή όχι για μόνο ένα τμήμα.

Επειδή "Person" είναι μια υπερκλάση ταυτόχρονα για "Student" και "Professor", οτιδήποτε μπορούμε να πούμε για το "Person" θα πρέπει να είναι αληθές και για τις δύο υπερκλάσεις.

Επομένως, ένας συγκεκριμένος μαθητής θα μπορεί προαιρετικά να εργάζεται για ένα τμήμα, ίσως σαν διδακτικός βοηθός και ένας συγκεκριμένος καθηγητής θα μπορεί προαιρετικά να εργαστεί για ένα τμήμα.

- Τελικά, σημειώνουμε ότι μία συσχέτιση (association) μπορεί να συσχετίσει κλάσεις σε διαφορετικά επίπεδα σε μία κληρονομική ιεραρχία όπως στη περίπτωση της χρήσης "mentors" η οποία συσχετίζει τις δύο κλάσεις "Student" και "Professor". Εδώ δηλώνεται ότι ο "Professor" εθελοντικά καθοδηγεί πολλά "Person", τα οποία "Person" μπορεί να είναι μαθητής-"Student" ή καθηγητής-"Professor". Και αντίθετα το άτομο-"Person" (είτε καθηγητής ή φοιτητής) καθοδηγείται προαιρετικά από πολλούς "Professor". Τέλος σημειώνουμε στη κλάση "Professor" την ένδειξη "mentor" για να δώσουμε έμφαση ότι οι "Professors" είναι σύμβουλοι "mentors" του πανεπιστημίου αλλά ότι το "Person" σε γενικές γραμμές μπορεί να μην είναι.

Τι θα γινόταν αν θα θέλαμε να τονίσουμε ότι ταυτόχρονα "Professor" και "Student" θα μπορούσαν να είναι "mentor"? Θα μπορούσαμε να αντικαταστήσουμε μία αντανakλαστική κλάση "Person" ως ακολούθως:

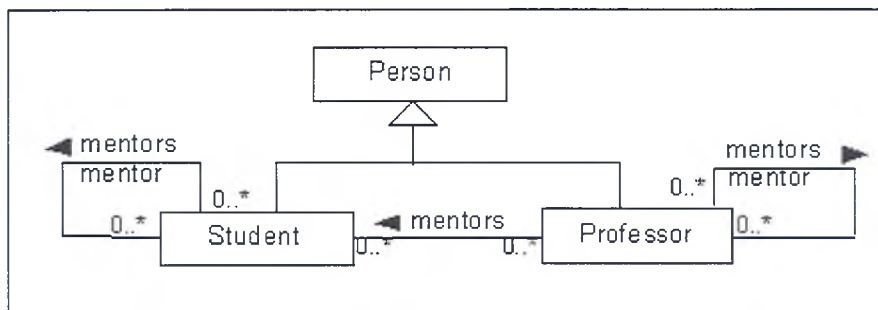


Το οποίο υπονοεί τέσσερις βασικές συσχετίσεις:

- Ο καθηγητής-"Professor" συμβουλεύει-"mentors" τον φοιτητή-"Student"

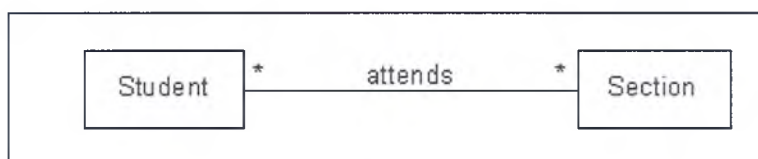
- Ο καθηγητής-"Professor" συμβουλεύει-"mentors" άλλο καθηγητή-"Professor"
- Ο φοιτητή-"Student" συμβουλεύει-"mentors" άλλο φοιτητή-"Student"
- Ο φοιτητή-"Student" συμβουλεύει-"mentors" τον καθηγητή-"Professor" (Δεν είναι και τόσο πιθανό!)

Εάν τώρα θα θέλαμε να ανακλάσουμε ότι μόνο οι τρεις πρώτες περιπτώσεις είναι πιθανές συσχετίσεις θα έπρεπε να καταλήξουμε στο πιο κάτω σύστημα:



Κλάσεις Συσχέτισης

Μερικές φορές ερχόμαστε αντιμέτωποι με μία κατάσταση όπου μία ιδιότητα αναγνωρίζεται σημαντική για το μοντέλο αλλά παρόλα αυτά δεν ανήκει με ιδανικό τρόπο σε μία κλάση. Για παράδειγμα έχουμε την παρακάτω συσχέτιση ενός μαθητή-"Student" που παρακολουθεί- "attends" ένα συγκεκριμένο τμήμα μαθημάτων-"Section".

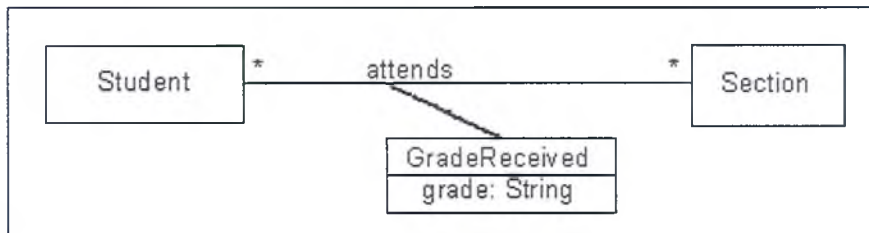


Στο τέλος κάθε εξαμήνου ο φοιτητής παίρνει βαθμολογία για κάθε μάθημα που παρακολούθησε σε ένα συγκεκριμένο τμήμα κατά την διάρκεια του εξαμήνου. Αποφασίζουμε ότι ο βαθμός θα παρουσιαστεί σαν μία αλυσίδα χαρακτηριστικών(δηλ. "A", "B", "C+" κ.ο.κ.). Όμως, που ανήκει το χαρακτηριστικό βαθμός?

- Δεν είναι χαρακτηριστικό της κλάσης φοιτητών "Student", επειδή ο φοιτητής δεν παίρνει ένα μόνο συγκεντρωτικό βαθμό για όλα τα διαφορετικά μαθήματα που παρακολούθησε, αλλά διαφορετικό βαθμό για κάθε μάθημα που παρακολούθησε.
- Δεν είναι χαρακτηριστικό της κλάσης τμήματος "Section" επειδή δεν παρακολουθούν όλοι οι φοιτητές ένα συγκεκριμένο τμήμα που περιλαμβάνει τα ίδια μαθήματα.

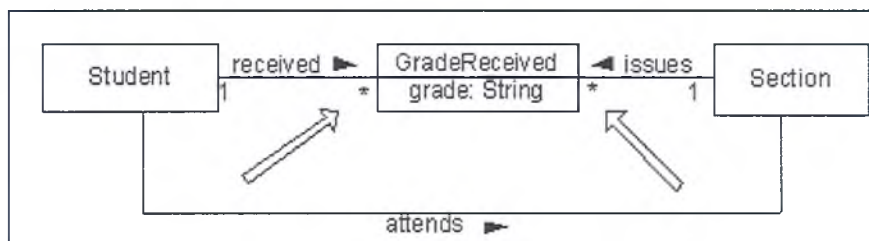
Έτσι αντιλαμβανόμαστε ότι ο βαθμός είναι ένα χαρακτηριστικό που ανήκει στη συσχέτισης που ενώνει τις δύο κλάσεις "Student" και "Section".

Με την UML, δημιουργούμε μία ξεχωριστή κλάση που είναι γνωστή σαν κλάση συσχέτισης για να στεγάσει τα χαρακτηριστικά που ανήκουν στη συσχέτιση μεταξύ των δύο αντικειμένων. Όπως φαίνεται και παρακάτω, την κλάση αυτή την προσθέτουμε με μία διακεκομμένη γραμμή που συνδέεται με την γραμμή συσχέτισης.



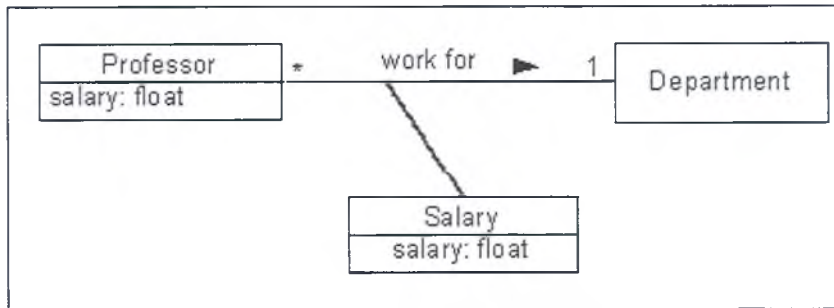
Έτσι κάθε φορά που βλέπεις μία κλάση συσχέτισης σε ένα διάγραμμα, αντιλαμβάνεσαι ότι υπάρχει ένας εναλλακτικά ισοδύναμος τρόπος να αναπαραστήσεις την ίδια κατάσταση χωρίς να χρησιμοποιήσεις μία κλάση συσχέτισης.

- Στην περίπτωση που έχουμε μία σχέση «πολλά προς πολλά» στην οποία εμπλέκεται μία συσχέτιση κλάσεως, θα μπορούσαμε να χωρίσουμε την σχέση αυτή σε δύο σχέσεις του τύπου «ένα προς πολλά», εισάγοντας την κλάση συσχέτισης που είχαμε προηγουμένως σαν μία κανονική κλάση μεταξύ των άλλων δύο κλάσεων. Κάνοντας αυτό, έχουμε την ακόλουθη ισοδύναμη και εναλλακτική λύση:

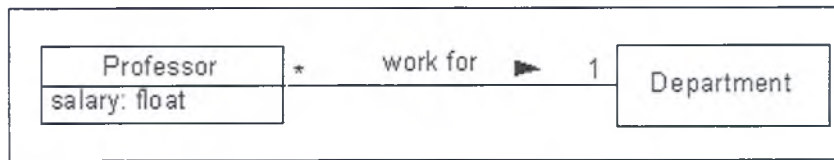


Ένα σημαντικό σημείο για να σημειώσουμε είναι ότι η πολλαπλότητα «πολλά»(*) στο τέλος των δύο συσχετίσεων συνυπάρχει με την νεοεισαχθείσα κλάση για το λόγο ότι ο φοιτητής "Student" παίρνει πολλούς βαθμούς και το τμήμα "Section" καταχωρεί πολλούς βαθμούς.

- Εάν τώρα τύχει να έχουμε μία σχέση της μορφής «ένα προς πολλά» όπως στη περίπτωση που ένας καθηγητής-"Professor" δουλεύει για-"works for" ένα τμήμα-"Department":

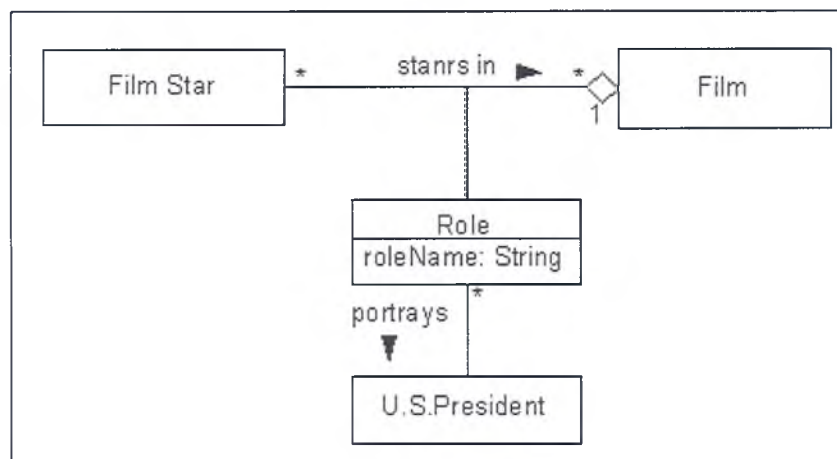


τότε τα χαρακτηριστικά της συσχέτισης μπορούν θεωρητικά να ταξινομηθούν στη κλάση με πολλαπλότητα «πολλά» και να μην χρησιμοποιήσουμε τη συσχέτιση καθόλου, όπως παρακάτω:



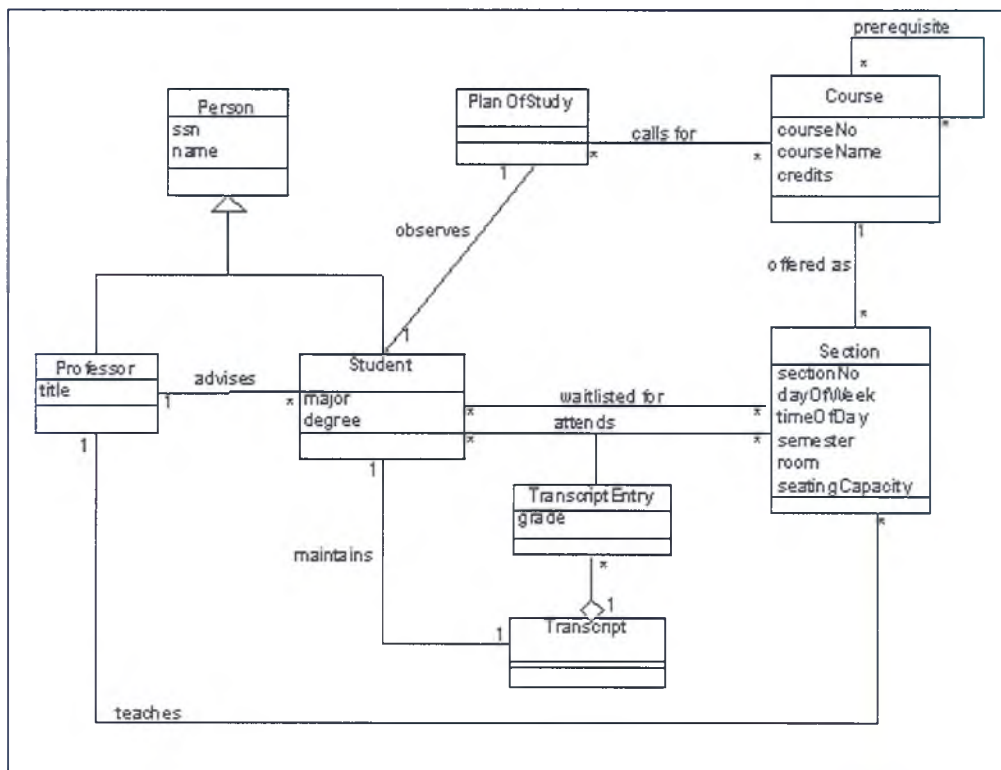
- Σε περίπτωση συσχέτισης «ένα προς ένα» μπορούμε να θέσουμε τα χαρακτηριστικά της συσχέτισης σε οποιαδήποτε από τις δύο κλάσεις.

Επίσης σημειώνουμε ότι οι κλάσεις συσχέτισης είναι κανονικές κλάσεις που μπορούν οι ίδιες να λάβουν μέρος σε συσχετίσεις με άλλες κλάσεις. Στο διάγραμμα πιο κάτω, για παράδειγμα, δείχνουμε την συσχέτιση μεταξύ της κλάσης "Role" που παίρνει μέρος στη συσχέτιση «ένα προς πολλά» με την κλάση "U.S.President". ένα παράδειγμα που να δείχνει αυτό το μοντέλο θα ήταν αυτό του ηθοποιού Anthony Hopkins που πρωταγωνιστούσε στη ταινία "Nixon" στο ρόλο του Richard M.Nixon , απεικονίζοντας έτσι στη ταινία τον πραγματικό πρώην πρόεδρο.



Η "Ολοκληρωμένη" Κλάση Συσχέτισης για το SRS

Εφαρμόζοντας όλα όσα έχουμε μάθει στην ενότητα αυτή σχετικά με τα στατικά μοντέλα, έχουμε φτιάξει το διάγραμμα UML για το Σύστημα Εγγραφής Φοιτητών (SRS), το οποίο φαίνεται πιο κάτω. Φυσικά, όπως έχουμε επανειλημμένα πει, αυτός δεν είναι ο μόνος ορθός τρόπος για να μοντελοποιήσουμε τις απαιτήσεις και ούτε κατά ανάγκη ο καλύτερος τρόπος μοντέλου που θα μπορούσαμε να είχαμε φτιάξει. Είναι όμως ένα ακριβές σύστημα και ορθό μοντέλο των στατικών χαρακτηριστικών του προβλήματος που θα αυτοματοποιηθεί.



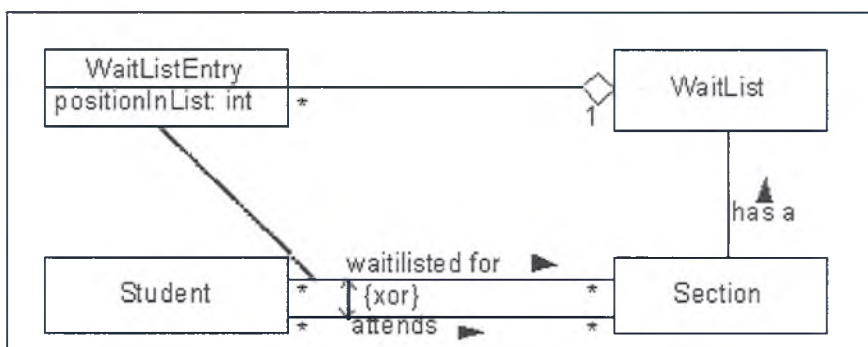
Ορισμένα σημεία που αξίζει να σημειωθούν είναι:

- Προτιμήσαμε να χρησιμοποιήσουμε τον γενικό συμβολισμό «πολλά» (*) για την UML παρά να ορίσουμε 0..* ή 1..*. Αυτό συχνά είναι αρκετό κατά την διάρκεια των αρχικών σταδίων της μοντελοποίησης.
- Σημειώστε ότι έχουμε αντικατοπτρίσει δύο διαφορετικές συσχετίσεις «πολλά προς πολλά» μεταξύ των κλάσεων "Student" και "Section": "waitlisted for" και "attends". Ένας συγκεκριμένος φοιτητής-"Student" μπορεί να είναι στη λίστα αναμονής για διάφορα τμήματα-"Section" και μπορεί να εγγραφεί και να παρακολουθεί πολλά άλλα μαθήματα. Τι δεν αντικατοπτρίζει αυτό το μοντέλο είναι το γεγονός ότι

ένας φοιτητής δεν μπορεί να είναι στη λίστα αναμονής και να παρακολουθεί το ίδιο μάθημα. Περιορισμοί όπως αυτόν μπορούν να αποδοθούν σαν λεκτικές σημειώσεις στο διάγραμμα και να περικλείονται σε άγκιστρα ή να παραληφθούν από το διάγραμμα και να τοποθετηθούν στο λεξικό δεδομένων. Στο ακόλουθο απόσπασμα διαγράμματος χρησιμοποιούμε τον συμβολισμό {x or} για να απεικονίσουμε μία αποκλειστική κατάσταση μεταξύ δύο συσχετίσεων:

Ένας φοιτητής-"Student" μπορεί να είναι είτε στην λίστα αναμονής-"waitlisted for" είτε-{x or} να παρακολουθεί-"attends" το μάθημα-"Section", αλλά όχι και τα δύο.

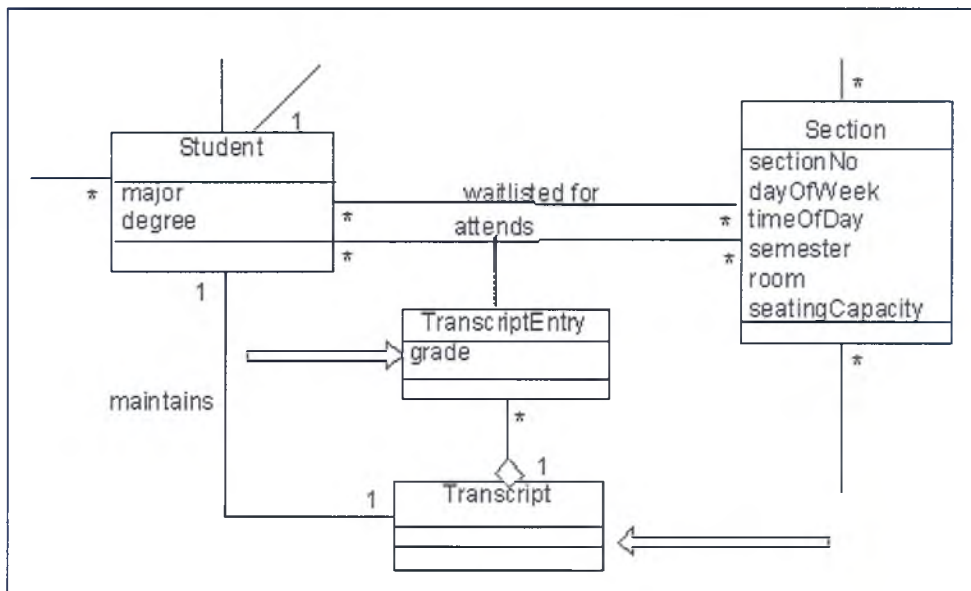
- Όπως αναφέρθηκε νωρίτερα σε αυτή την ενότητα, είμαστε ικανοί να ξεφύγουμε με μία μόνο συσχέτιση "attends" για να χειρίζεται ταυτόχρονα τα "sections" που ο φοιτητής παρακολουθεί μαζί με αυτά που ήδη έχει παρακολουθήσει στο παρελθόν. Η ημερομηνία παρακολούθησης- παρελθόν ή παρόν- φαίνεται από το χαρακτηριστικό του "semester" από το τμήμα "Section". Επίσης για όποια μαθήματα είναι ήδη σε εξέλιξη, η τιμή της ιδιότητας "grade" της κλάσης "TranscriptEntry" δεν θα έχει ήδη υπολογιστεί.
- Θα μπορούσαμε να είχαμε αντικατοπτρίσει μία κλάση συσχέτισης επί του καταλόγου αναμονής "waitlist" που να δείχνει την δεδομένη θέση του φοιτητή στον κατάλογο για το συγκεκριμένο μάθημα και μετέπειτα θα μπορούσε να συνεχίσει με το μοντέλο του συμβολισμού "waitlist" σαν μία συσσωμάτωση σε αντικείμενο "WaitListEntry":



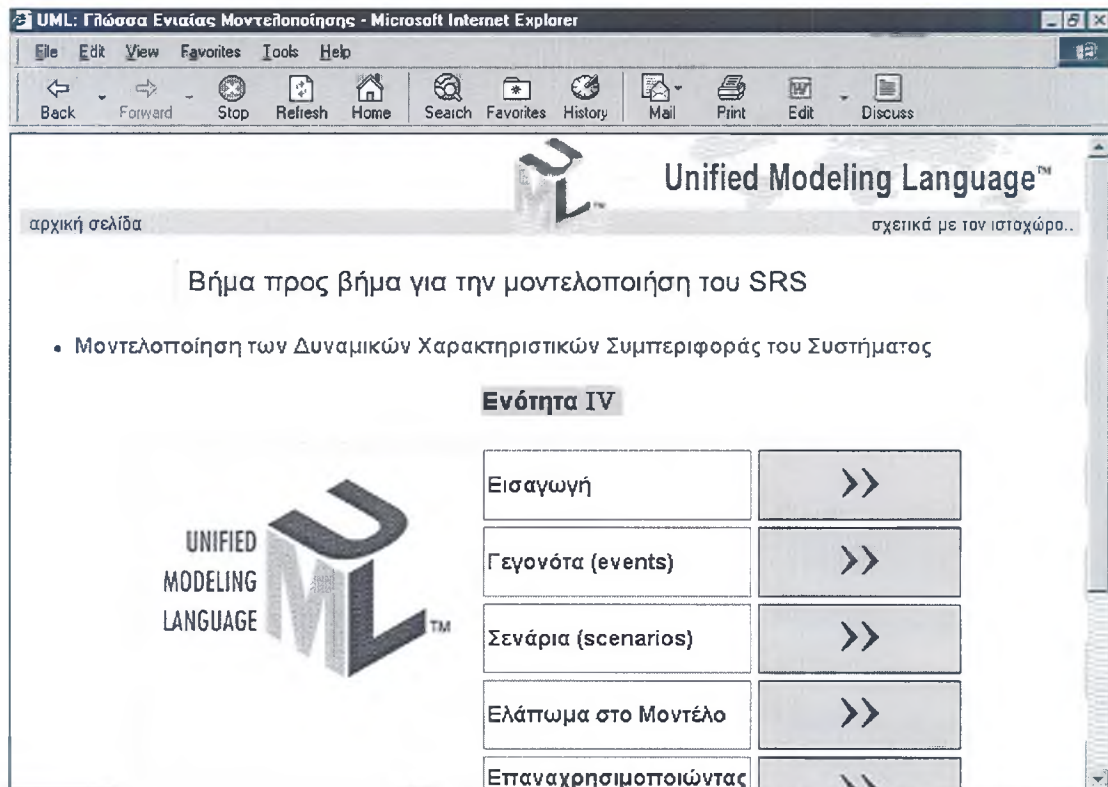
Εφόσον πρόκειται να θέλουμε να χρησιμοποιήσουμε αυτό το μοντέλο για να εξασφαλίσουμε πληροφορίες για τους χρήστες και τις αρχικές τους ανάγκες, δεν χρειάζεται να περιπλέξουμε το διάγραμμα με τέτοιου είδους λεπτομέρειες και ενέργειες στο παρασκήνιο.

- Επίσης έχουμε μετονομάσει την κλάση συσχέτισης σε "attends". Είχε εισαχθεί νωρίτερα σε αυτή την ενότητα σαν "GradeReceived" αλλά τώρα λέγεται "TranscriptEntry". Επίσης είχαμε δημιουργήσει μία

σχέση συσσωμάτωσης μεταξύ "TranscriptEntry" και μίας νέας κλάσης που λέγεται "Transcript".



Ενότητα IV



Εισαγωγή

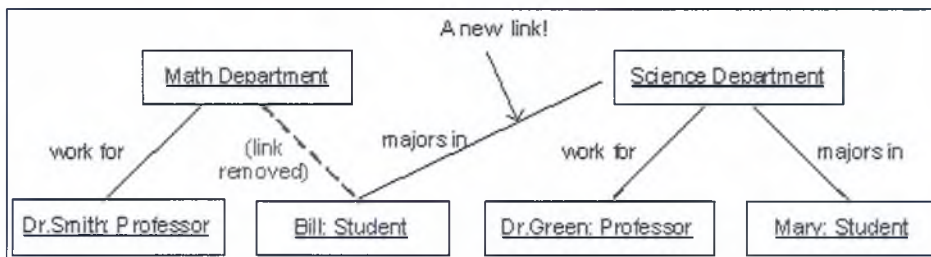
Μέχρι τώρα, είχαμε εστιάσει στην στατική δομή(static structure) του προβλήματος που μοντελοποιούμε. Όπως έχουμε μάθει στην Ενότητα III, αυτή η στατική δομή μεταβιβάζεται μέσω ενός διαγράμματος κλάσεων(class diagram) συν τους αντίστοιχους δεσμούς εξάρτησης, γενίκευσης και σύνδεσης.

Τα δομικά τμήματα ενός διαγράμματος κλάσεων είναι:

- Κλάσεις(Classes)
- Συσχετίσεις(Associations)/ Συναθροίσεις (Aggregations)
- Ιδιότητες(Attributes)
- Λειτουργίες(Operations)/Μέθοδοι(Methods). Αυτά ήταν απόντα από τα προηγούμενα διάγραμμα κλάσεων. Γιατί? Επειδή δεν αποτελούν τμήμα της στατικής δομής, έτσι δεν έχουμε συζητήσει ακόμα πως μπορούν να προσδιοριστούν. Αυτό θα είναι το θέμα της παρούσας ενότητας.

Όπως έχουμε πει αρκετές φορές μέχρι τώρα, ένα ΟΟ πρόγραμμα είναι μβα δέσμη από συνεργαζόμενα αντικείμενα που το καθένα έχει τη δική του ζωή. Εάν κάθε αντικείμενο συνέχιζε να ζεί κάνοντας την δική του δουλειά χωρίς αναφορά στα υπόλοιπα, θα επικρατούσε γενικό και απόλυτο χάος. Ο μόνος τρόπος με τον οποίο τα αντικείμενα μπορούν να συνεργαστούν στην εκτέλεση της γενικής αποστολής του συστήματος - όπως συμβαίνει για παράδειγμα στην εγγραφή ενός μαθητή σε ένα κύκλο σπουδών(SRS) - είναι εάν κάθε είδος μπορούσε να προσδιορίζει τις πρέπουσες μεθόδους ή υπηρεσίες που θα καθιστούσαν δυνατή την εκπλήρωση των αντίστοιχων ρόλων στην συνεργασία αυτή.

Για να προσδιορίσουμε ποιες πρέπει να είναι οι μέθοδοι/ υπηρεσίες αυτές, θα πρέπει να συμπληρώσουμε τις γνώσεις μας για τη στατική δομή του συστήματος που θα φτιαχτεί με το να μοντελοποιήσουμε τις δυναμικές πτυχές της κατάστασης: Δηλαδή, τον τρόπο με τον οποίο ταυτόχρονα θα μπορούν να αλληλεπιδράσουν τα ενεργά στοιχεία με την πάροδο του χρόνου - συμπεριφορά- και πώς αυτές οι αλληλοεπιδράσεις επηρεάζουν την κατάσταση του κάθε αντικειμένου. Κατασκευάζοντας ένα δυναμικό μοντέλο για να αντικαταστήσει το στατικό μοντέλο, όχι μόνο θα μας δώσει την δυνατότητα να προσδιορίσουμε τις μεθόδους που απαιτούνται για κάθε είδος αλλά θα μας δώσει νέες βλέψεις και τρόπους για βελτίωση πάνω στο στατικό μοντέλο.



Έτσι μπορούμε να δούμε ότι οι δυναμικές δραστηριότητες του αντικειμένου μπορούν να προκαλέσουν αλλαγές στη στατική δομή του συστήματος ενώ άλλου τέτοιου είδους δραστηριότητες δεν θα χρειαζόταν να επηρεάσουν την κατάσταση όλων των εμπλεκόμενων αντικειμένων στην συνεργασία αυτή.

Γεγονότα

Ξέρουμε ότι η συνεργασία των διάφορων στοιχείων αρχίζει από τα γεγονότα. Για λόγους μελέτης, ένα γεγονός είναι μια εξωτερική διεργασία του συγκεκριμένου αντικειμένου που αποτελείται στο στοιχείο αυτό με την μορφή ενός μηνύματος. Ένα γεγονός μπορεί να είναι:

- ✱ Προεπιλεγμένο από τον χρήστη(για παράδειγμα με το πάτημα ενός κουμπιού στο GUI).
- ✱ Προεπιλεγμένο από ένα άλλο σύστημα υπολογιστών(όπως για παράδειγμα την άφιξη πληροφοριών από την μεταφορά δεδομένων από το Σύστημα Εγγραφής Φοιτητών(SRS).
- ✱ Προεπιλεγμένο από άλλο αντικείμενο μέσα στο ίδιο σύστημα(όπως για παράδειγμα ένα αντικείμενο του μαθήματος που απαιτείται για κάποια υπηρεσία κάποιου transcript αντικείμενου).

Όταν ένα αντικείμενο πληροφορηθεί για ένα συμβάν μέσω ενός μηνύματος, μπορεί να αντιδράσει με ποικίλους τρόπους:

- ✱ Ένα αντικείμενο μπορεί να αλλάξει την κατάσταση του(τις τιμές των απλών του ιδιοτήτων ή και των συνδέσμων του με άλλα αντικείμενα), όπως στην περίπτωση του αντικείμενου "Professor" λαμβάνοντας ένα μήνυμα για να πάρει ένα νέο μαθητή να συμβουλευεί, αυτό επιδεικνύεται από τον ακόλουθο κώδικα:

```
Professor p = new Professor();
Student s = new Student();
// details omitted
p.addAdvisee(s);
```

Ας κοιτάξουμε στον κώδικα για την κλάση "Professor" και την μέθοδο addAdvisee() για να δούμε πώς θα αντιδράσει σε αυτό το μήνυμα. Βλέπουμε ότι το αντικείμενο "Professor" τροφοδοτεί την αναφορά προς το αντικείμενο "Student" το οποίο έχει δοθεί σαν επιχείρημα σε μια συλλογή αντικειμένων "Student" ,σαν αναφορά που λέγεται "advisees" :

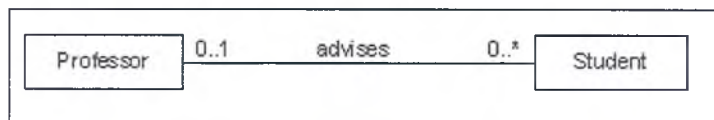
```
Class Professor {
    // Attributes.
    Collection advisees;
    // Holds Student object references.
    //details omitted
```

```

public void addAdvisee(Student s) {
    // Insert s into the advisees collection.
    // (pseudocode)
    advisees.insert(s);
}
}

```

Κάνοντας αυτό, το αντικείμενο "Professor" = p θα έχει σχηματίσει ένα νέο σύνδεσμο του τύπου "advise" με το αντικείμενο "Student" = s.



Τυπικές μέθοδοι για ενεργοποίηση εμπίπτουν σε αυτή τη κατηγορία αντίδρασης.

- ✱ Ένα αντικείμενο μπορεί να κατευθύνει ένα γεγονός(μήνυμα) προς ένα άλλο αντικείμενο(περιλαμβάνοντας ίσως ακόμα και τον αποστολέα του αυθεντικού μηνύματος), όπως στην περίπτωση του τμήματος που λαμβάνει ένα μήνυμα για την εγγραφή ενός μαθητή, το οποίο βλέπουμε από τον κάτωθι κώδικα:

```

Section x = new Section();
Student s = new Student();
// details omitted
x.register(s);

```

Εάν στη συνέχεια κοιτάξουμε τον κώδικα της κλάσης "Section" της μεθόδου register() για να δούμε πως αντιδρά σε αυτό το μήνυμα, βλέπουμε ότι το αντικείμενο "Section" με την σειρά του αντιδρά στέλλοντας ένα μήνυμα προς τον μαθητή(Student) προς εγγραφή για να ελέγξει ότι ο μαθητής αυτός έχει συμπληρώσει το αναγκαίο προαπαιτούμενο κύκλο σπουδών:

```

Class Section {
    // details omitted

    boolean register(Student s) {

```



```
// Verify that the student has completed a
//necessary prerequisite course.
//(pseudocode)
boolean completed =
s.successfullyCompleted(someprerequisite);
if (completed) register the student
and return a value of true;
else return a value of false to
signal that the registration
request has been rejected;
}
}
```

Αυτό τυχαίνει να είναι ένα παράδειγμα αντιπροσώπευσης, όπου ένα αντικείμενο(ο μαθητής - "Student", στην περίπτωση αυτή) βοηθά να εκπληρωθεί η ζήτηση για παροχή υπηρεσίας που έγινε αρχικά στο αντικείμενο "Section".

- ✱ Ένα αντικείμενο μπορεί να επιστρέψει μία τιμή η οποία να είναι:
 - Η τιμή ενός χαρακτηριστικού(object s private attributes).
 - Κάποια υπολογιζόμενη τιμή.
 - Μια τιμή που αποκτήθηκε από άλλο αντικείμενο μέσω αντιπροσώπευσης.
 - Ένας κώδικας κατάστασης(π.χ ο κώδικας true/false ο οποίος μας γνωστοποιεί επιτυχία ή αποτυχία q με την μέθοδο boolean).
- ✱ Ένα αντικείμενο μπορεί να αντιδράσει με τα εξωτερικά σύνορα του συστήματος: δηλαδή, μπορεί να μας απεικονίσει κάποιες πληροφορίες πάνω σε GUI ή να αναγκάσει κάποιες πληροφορίες να εκτυπωθούν.
- ✱ Τελικά , ένα αντικείμενο μπορεί φαινομενικά να αγνοήσει ένα γεγονός όπως θα ήταν στην περίπτωση που ο "Professor" λάμβανε ένα μήνυμα να προσθέσει ένα συμβουλευόμενο μαθητή -"Student"

```
Student s = new Student();
Professor p = new Professor();
// details omitted
p.addAdvisee(s);
```

αλλά προσδιόρισε ότι ο φοιτητής που προτάθηκε ήταν ήδη εγγεγραμμένος. Ας κοιτάξουμε μία κάπως διαφορετική εκδοχή της μεθόδου `addAdvisee()` από αυτή που είδαμε προηγουμένως:

```
class Professor {
    Collection advisees // Holds Student object
                        // references.
                        //details omitted

    public void addAdvisee(Student s) {
        // Only insert into the 'advisees'
        // collection IF IT ISN'T ALREADY //IN
        THERE.
        // (pseudocode)
        if (s is already in collection) return;
        // do nothing
        else advisees.insert(s);
    }
}
```

Πραγματικά, για να πούμε ότι το αντικείμενο "Professor" δεν κάνει τίποτα είναι μια εξακρίβωση της περίπτωσης αυτής. Το ελάχιστο είναι πως το αντικείμενο εκτελεί τον σωστό κώδικα, ο οποίος εκτελεί κάποιους εσωτερικούς ελέγχους ('Is this student already one of my advisees?'). Απλούστατα, όταν η ερώτηση αυτή βρίσκει αρνητική απάντηση τότε το αντικείμενο "Professor" δεν αλλάζει κατάσταση αλλά ούτε και εκπέμπει μήνυμα προς άλλα αντικείμενα και έτσι φαίνεται ότι τίποτα δεν γίνεται.

Φυσικά, αυτοί οι διάφοροι τρόποι αντίδρασης προς τα γεγονότα μπορεί να γίνουν ταυτόχρονα ή ακόμα και σε συνδυασμούς.

Σενάρια

Τα γεγονότα που ενεργοποιούνται εξωτερικά προς το σύστημα γίνονται τυχαία, δηλαδή δεν μπορούμε να προβλέψουμε για παράδειγμα πότε ο χειριστής του υπολογιστή θα πατήσει ένα πλήκτρο στο GUI.

Τα μηνύματα όμως που τα αντικείμενα ανταλλάσσουν κατά την εκτέλεση κάποιων λειτουργιών του συστήματος δεν μπορούν να αφεθούν και να πραγματοποιηθούν τυχαία. Αντίθετα θα πρέπει να ενορχηστρώνονται με τέτοιο τρόπο, έτσι ώστε να οδηγούν στο ποθητό αποτέλεσμα. Κατά τον ίδιο τρόπο που ένα μουσικό πεντάγραμμο δείχνει πια νότα θα πρέπει να παιχτεί από τα διάφορα μουσικά όργανα για να παραχθεί η μελωδία, έτσι και ένα σενάριο προκαθορίζει την σειρά με την οποία θα πρέπει να γίνουν τα διάφορα εσωτερικά μηνύματα και γεγονότα από την αρχή μέχρι το τέλος.

Το λεξικό "Merriam-Webster's" περιγράφει τον όρο "scenario" σαν:

«μία σειρά γεγονότων(ειδικά όταν είναι
φανταστικά τα γεγονότα αυτά)ή ακόμα
σαν ένα λογαριασμό που περιλαμβάνει
σειρά ενεργειών ή γεγονότων»

Το **σενάριο** είναι μία υποθετική περίπτωση για την χρησιμοποίηση ενός συγκεκριμένου "Use Case". Όπως ένα αντικείμενο(Object) είναι μία περίπτωση κλάσης(class) και ο σύνδεσμος(link) είναι μία περίπτωση συνεργασίας(association), το σενάριο(scenario) μπορεί επομένως να είναι μία περίπτωση χρήσης(use case). Επίσης εκφραζόμενο με άλλα λόγια, όπως μία κλάση(class) είναι ένα ταμπλό(template) δημιουργίας αντικειμένων, έτσι και η συνεργασία(association) είναι ένα ταμπλό δημιουργίας συνδέσμων και κατά επέκταση "Use Case" είναι ένα ταμπλό δημιουργίας σεναρίων.

Μία μόνο περίπτωση χρήσης "Use Case" εμπνέει την δημιουργία πολλών διαφορετικών σεναρίων, με τον ίδιο τρόπο που ένα ταξίδι από μία πόλη σε μία άλλη μπορεί να περιλαμβάνει πολλές διαφορετικές διαδρομές.

Περιγράφουμε τα σενάρια σαν μία σειρά από βήματα που ξεκινάνε από ένα σημείο αναφοράς ενός υποθετικού παρατηρητή που είναι ικανός να δει όχι μόνο το τι συμβαίνει στον εξωτερικό κόσμο(το τι δηλαδή εξωτερικεύεται κατά την διάρκεια που το σύστημα εκτελεί μία συγκεκριμένη εργασία), αλλά επίσης το τι συμβαίνει πίσω από το εμφανές σκηνικό δηλαδή στο εσωτερικό του συστήματος.

Το ακόλουθο είναι ένα δείγμα σεναρίων που αντιπροσωπεύει την "Εγγραφή σε ένα κύκλο σπουδών" σαν μία περίπτωση "Use Case", μία από τις πολλές που έχουν υποδειχθεί για το SRS στη Ενότητα I.

Σενάριο #1 - Εγγραφή φοιτητή σε κύκλο σπουδών

- 1) Ο Fred είναι φοιτητής και μπαίνει στο SRS.
- 2) Βλέπει το πρόγραμμα για τα διάφορα τμήματα του παρόντος εξαμήνου έτσι ώστε να προσδιορίσει με βάση το τμήμα σε ποια από τα μαθήματα επιθυμεί να εγγραφεί.
- 3) Ο Fred ζητά μία θέση σε ένα συγκεκριμένο τμήμα κύκλου σπουδών που ονομάζεται 'Beginning Objects', με αριθμό/ κωδικό μαθήματος OBJ101 και τμήμα 1.
- 4) Ο Fred ελέγχει το πλάνο σπουδών για να βεβαιωθεί ότι ο ζητούμενος κύκλος είναι κατάλληλος για τους γενικούς στόχους του διπλώματος του.(Υποθέτουμε ότι δεν επιτρέπεται στους φοιτητές να πάρουν μαθήματα εκτός του πλάνου σπουδών τους.)
- 5) Ελέγχετε το αντίγραφο σπουδών του για να βεβαιωθεί ότι εκπληρώνει όλες τις απαιτούμενες προαπαιτήσεις για το ζητηθέν μάθημα- αν υπάρχουν.
- 6) Επιβεβαιώνεται η ύπαρξη θέσεων στο συγκεκριμένο ζητηθέν μάθημα.
- 7) Προστίθεται το συγκεκριμένο μάθημα στη καρτέλα του Fred.

Ο Fred που κάθεται μπροστά από τον υπολογιστή του για να κάνει την εγγραφή, δεν αντιλαμβάνεται τι γίνεται στο σύστημα. Μετά την είσοδο του στο SRS, δείχνει ότι επιθυμεί να εγγραφεί στο μάθημα με κωδικό OBJ101, τμήμα 1, το οποίο και επιλέγει από τον κατάλογο "Course catalog" και μετά πατά το κουμπί "add" για να του προστεθεί το επιλεγμένο μάθημα. Μερικές λεπτά αργότερα αφού γίνει η επεξεργασία των στοιχείων του Fred από το σύστημα, ο Fred λαμβάνει ένα μήνυμα επιβεβαίωσης θέσης.

Όπως βλέπουμε ο Fred δεν είναι ενήμερος για τις πλείστες επεξεργασίες εντός του υπολογιστή και για τα βήματα του προγράμματος και πώς εκτελούνται.

Το προαναφερθεί σενάριο απεικονίζει μια εξιδανικευμένη περίπτωση όπου όλα κυλούν ομαλά και ο Fred τελειώνει επιτυχώς την εγγραφή του στο ζητηθέν μάθημα. Αλλά όπως όλοι γνωρίζουμε καλά, τα πράγματα δεν κυλούν πάντα τόσο ομαλά, όπως θα δούμε στο εναλλακτικό σενάριο που ακολουθεί για την ίδια περίπτωση. Τα δύο σενάρια #1 και #2 όπως θα διαφοροποιούνται μόνο στα βήματα 6, 7, 8.

Σενάριο #2 - Εγγραφή φοιτητή σε κύκλο σπουδών

- 1) Ο Fred είναι φοιτητής και μπαίνει στο SRS.
- 2) Βλέπει το πρόγραμμα για τα διάφορα τμήματα του παρόντος εξάμηνου έτσι ώστε να προσδιορίσει με βάση το τμήμα σε ποια από τα μαθήματα επιθυμεί να εγγραφεί.
- 3) Ο Fred ζητά μία θέση σε ένα συγκεκριμένο τμήμα κύκλου σπουδών που ονομάζεται "Beginning Objects", με αριθμό/ κωδικό μαθήματος OBJ101 και τμήμα 1.
- 4) Ο Fred ελέγχει το πλάνο σπουδών για να βεβαιωθεί ότι ο ζητούμενος κύκλος είναι κατάλληλος για τους γενικούς στόχους του διπλώματος του. (Υποθέτουμε ότι δεν επιτρέπεται στους φοιτητές να πάρουν μαθήματα εκτός του πλάνου σπουδών τους.)
- 5) Ελέγχεται το αντίγραφο σπουδών του για να βεβαιωθεί ότι εκπληρώνει όλες τις απαιτούμενες προαπαιτήσεις για το ζητηθέν μάθημα- αν υπάρχουν.
- 6) Ελέγχεται εάν υπάρχει θέση και διαπιστώνεται ότι το μάθημα αυτό είναι πλήρες.
- 7) Ο Fred ερωτάται από το σύστημα εάν επιθυμεί να μπει στο κατάλογο αναμονής.
- 8) Ο Fred επιλέγει να μπει στον κατάλογο αναμονής.

Με λίγη φαντασία θα μπορούσαμε να σκεφτούμε αμέτρητα σενάρια για την περίπτωση αυτή που να περιλαμβάνουν τέτοιες καταστάσεις, όπως για παράδειγμα την επιλογή μαθήματος εκτός κύκλου σπουδών του Fred ή την επιλογή για την οποία δεν πληρεί τις αναγκαίες προαπαιτήσεις. Υπάρχουν επίσης και πολλές άλλες περιπτώσεις "Use Case" για να ληφθούν υπόψη όπως συζητήσαμε στην Ενότητα I.

Υπάρχουν πρακτικά όρια στον αριθμό των εναλλακτικών σεναρίων που θα μπορούσε κάποιος να λάβει υπόψη για μία δεδομένη "Use Case"?

Όπως για όλες τις αναλύσεις απαιτήσεων, τα κριτήρια στο πού σταματάμε είναι υποκειμενικά: Σταματάμε όταν φαίνεται πώς δεν μπορούμε να δημιουργήσουμε περαιτέρω σενάρια με σημαντικές διαφορές. Ασήμαντες διαφορές θα πρέπει να αποφεύγονται.

Γιατί τα σενάρια είναι τόσο σημαντικά?

Επειδή είναι τα μέσα με τα οποία ξεκίνα η απόκτηση της εσωτερικής γνώσης και συμπεριφοράς που θα απαιτήσει το αντικείμενο μας. Θα χρειαστούμε ένα τρόπο για να επισημοποιήσουμε αυτά τα σενάρια έτσι ώστε η πραγματικές μέθοδοι που απαιτούνται για κάθε μία από τις κλάσεις μας να γίνουν

εμφανείς. Τα διαγράμματα ροής(sequence diagram)της UML είναι τα μέσα με τα οποία μπορούμε να το πετύχουμε αυτό.

Ελάττωμα στο Μοντέλο

Ξαναβλέποντας τη δήλωση της υπόθεσης του SRS, θα διαπιστώσουμε ότι έχουμε αφήσει πίσω μια αναγκαία απαίτηση.

Συγκεκριμένα:

"Το SRS θα ελέγχει κατά πόσο το πόσο το προτεινόμενο πλάνο σπουδών ικανοποιεί τις απαιτήσεις του διπλώματος που προσδοκεί ο φοιτητής"

Δεν έχουμε μοντελοποιήσει το "Degree" σαν κλάση. Επαναφέρουμε στη μνήμα μας την συζήτηση του κατά πόσο να το συμπεριλάβουμε ή όχι(ενότητα III) και σε τελευταία ανάλυση αποφασίσαμε ενάντιο αυτού. Επίσης για το λόγο αυτό δεν αντικατοπτρίζουμε τις απαιτήσεις κάποιου συγκεκριμένου διπλώματος-κύκλου σπουδών στο μοντέλο μας. Ας κοιτάξουμε τι χρειάζεται για να το πραγματοποιήσουμε.

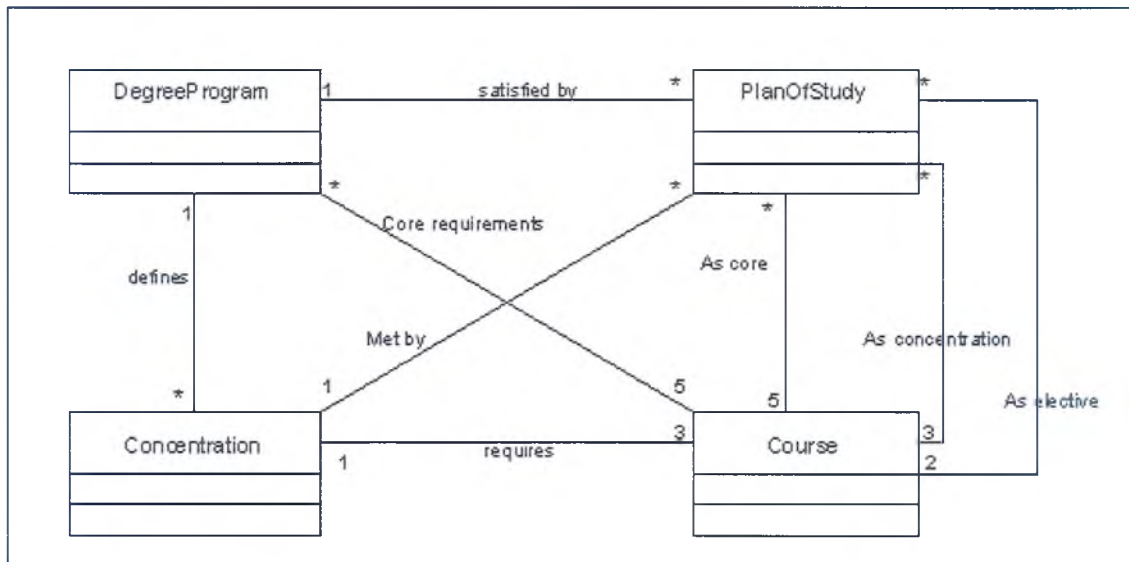
Ερευνώντας τον τρόπο με τον οποίο το πανεπιστήμιο, μας ορίζει τις απαιτήσεις για το κάθε πρόγραμμα-κύκλο σπουδών, μαθαίνουμε τα ακόλουθα:

- Κάθε κύκλος σπουδών-degree απαιτεί πέντε μαθήματα κοινού κορμού, δηλαδή μαθήματα τα οποία οι φοιτητές πρέπει να πάρουν. Για παράδειγμα, για τον κύκλο σπουδών "Master of Science in Information Technology", οι φοιτητές πρέπει να πάρουν τα ακόλουθα πέντε μαθήματα:
 - Ø Analysis of Algorithms
 - Ø Application Programming Design
 - Ø Data Structures
 - Ø Information Systems Project Management
- Κατά δεύτερο, οι φοιτητές αναμένεται να επιλέξουν ένα τομέα εξειδίκευσης στα πλαίσια του προγράμματος του κύκλου σπουδών τους, τα οποία είναι γνωστά σαν μαθήματα συγκέντρωσης. Για το μάθημα "Master of Science in Information Technology", το πανεπιστήμιο προσφέρει τρεις διαφορετικές συγκεντρώσεις:
 - Ø Object Technology
 - Ø Database Management Systems
 - Ø Networking and Communications
- Από την άλλη πλευρά κάθε συγκέντρωση με την σειρά της ορίζει τρία υποχρεωτικά μαθήματα. Για το "Master of Science in Information Technology" με προσοχή στην "Object Technology" τα απαιτούμενα συγκεντρωτικά μαθήματα είναι:
 - Ø Object Methods for Software Development
 - Ø Advanced Java Programming

Ø Object Database Management Systems

- Τελικά ο φοιτητής πρέπει να πάρει δύο επιπρόσθετα μαθήματα έτσι ώστε να συμπληρώσει τον συνολικό αριθμό μαθημάτων για τον κύκλο σπουδών.

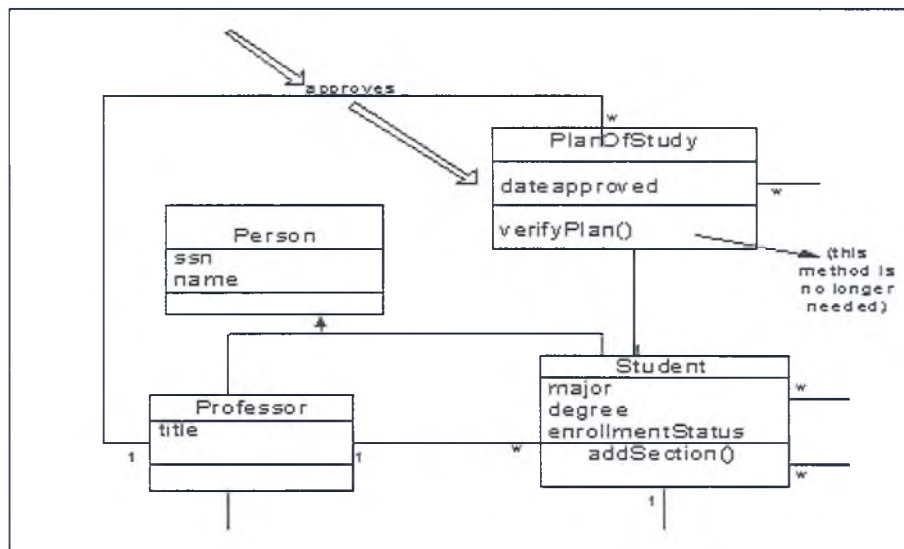
Για να μοντελοποιήσουμε όλα τα παραπάνω ανεξάρτητα γεγονότα, απαιτείται ένα σχετικά πολύπλοκο διάγραμμα δομής όπως πιο κάτω:



Πηγαίνοντας στους χορηγούς της μελέτης μας- τους μελλοντικούς χρήστες του SRS- και ανακοινώνοντας τους τα νέα ότι μόλις έχουμε ανακαλύψει μια ελλείπουσα μέχρι τώρα ανάγκη για να καλύψει το πρόγραμμα , η οποία θα αυξήσει σημαντικά το κόστος και την πολυπλοκότητα του αυτοματοποιημένου συστήματος μας. Οι χορηγοί μας αποφάσισαν ότι με το να έχουμε το SRS να πιστοποιεί την ορθότητα του πλάνου σπουδών του φοιτητή είναι πολύ φιλόδοξο σαν στόχος. Αντί αυτού αποφάσισαν ότι ο φοιτητής θα χρησιμοποιεί το SRS για να αποδώσει ένα προτεινόμενο πλάνο, αλλά ο σύμβουλος του φοιτητή θα είναι υπεύθυνος να εξακριβώσει χειρονακτικά και να αποδεκτή την αίτηση του φοιτητή. Έτσι το μόνο που θα χρειάζεται τώρα για να διορθώσουμε το διάγραμμα κλάσης είναι να προσθέσουμε μια ιδιότητα(attribute) στη κλάση "PlanOfStudy" που να αντανakλά την ημερομηνία αποδοχής της αίτησης και το νέο "approves" σύνδεσμο που ενώνει τη κλάση "Professor" με την κλάση "PlanOfStudy" και με αυτό τελειώνουμε την διόρθωση.

Σημειώστε ότι δεν χρειάζεται να προσθέσουμε ένα approvePlan() μέθοδο στο "PlanOfStudy" κλάση, επειδή όπως συζητήθηκε στην Ενότητα III , μπορούμε να υποθέσουμε την παρουσία των μεθόδων 'set' για όλες τις

ιδιότητες. Η `setDateApproved()` μέθοδος θα δείχνει/ σημειώνει ότι το πλάνο/ αίτηση έχει εγκριθεί. Και ο σύνδεσμος "approves" μεταξύ της κλάσης "PlanOfStudy" και της κλάσης "Professor" διαβεβαιώνει ότι κάθε αντικείμενο "PlanOfStudy" θα διατηρεί το πάνω χέρι ελέγχου στο αντικείμενο "Professor" το οποίο ουσιαστικά εγκρίνει την αίτηση κατά την ημερομηνία που δείχνει.



Επαναχρησιμοποιώντας Μοντέλα

Όπως συζητήσαμε και σε άλλη ενότητα, όταν μπαίνουμε σε κάτι καινούργιο, αυτόματα ερευνούμε το πνευματικό μας αρχείο για άλλα μοντέλα που έχουμε φτιάξει προηγουμένως, για να κοιτάξουμε τυχόν ομοιότητες πάνω στις οποίες να βασιστούμε. Η τέχνη της σύγκρισης χαρακτηριστικών και το να βρεις μια περίληψη που είναι αρκετά όμοια για να επαναχρησιμοποιηθεί αποδοτικά, είναι γνωστή σαν επαναχρησιμοποίηση προτύπου. Όπως εξάγεται, η επαναχρησιμοποίηση προτύπου είναι σημαντική τεχνική για την ανάπτυξη προγραμμάτων που σχετίζονται με τα αντικείμενα.

Ας πούμε ότι μετά που τελειώσαμε το διάγραμμα κλάσης SRS, μας καλούν να μοντελοποιήσουμε ένα σύστημα για ένα μικρό ταξιδιωτικό γραφείο "Wild Blue Yonder" (WBY). Σαν μια καινούργια εταιρεία επιθυμούν να προσφέρουν εξυπηρέτηση στους πελάτες τους πάνω και πέρα από τους καλά εδραιωμένους ανταγωνιστές τους, έτσι αποφασίζουν να δώσουν την δυνατότητα στους πελάτες τους να προκρατούν την διαμονή τους μέσω του WEB. Για κάθε δεδομένο πακέτο ταξιδιού- ας πούμε ένα δεκαήμερο ταξίδι στην Ιρλανδία- η WBY προσφέρει αναρίθμητα ταξίδια σε όλη την διάρκεια του χρόνου. Κάθε ταξίδι έχει κάποια μέγιστη πληρότητα έτσι εάν ένας πελάτης δεν μπορεί να πάρει επιβεβαίωση για την θέση του σε κάποιο ταξίδι θα μπορεί να ζητήσει να εγγραφεί στην λίστα αναμονής(αυτός που έρχεται πρώτος εξυπηρετείται πρώτος).

Για να διατηρήσει επαφή με τους πελάτες της η WEB ακολουθεί μετά από κάθε ταξίδι ένα πρόγραμμα συμπλήρωσης μιας φόρμας αξιολόγησης ζητώντας από τους πελάτες της να βαθμολογήσουν την εμπειρία τους από το 1-10 με το 10 να είναι το εξαιρετο. Με αυτό τον τρόπο η WEB μπορεί να προσδιορίσει πια ταξίδια είναι πιο πετυχημένα για να τα προσφέρει πιο συχνά στο μέλλον και παράλληλα να απαλείψει αυτά που δεν είναι αρκετά δημοφιλή.

Η WBY θα μπορεί να παρέχει πληρέστερες περιγραφές/ εισηγήσεις στα μελλοντικά ταξίδια που ένας πελάτης είναι πιθανό να απολαμβάνει με την μελέτη του ιστορικού της κατάστασης/ αναφοράς ικανοποίησης πελατών.

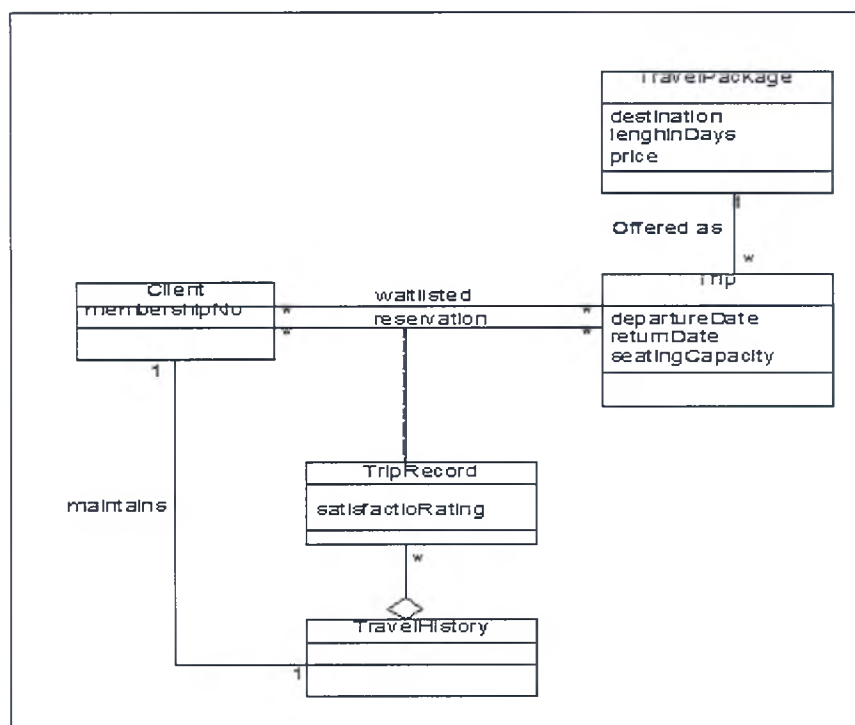
Αντικατοπτρίζοντας τις απαιτήσεις για το σύστημα αυτό, διαπιστώνουμε και αναγνωρίζουμε ότι πολλοί τομείς του συστήματος WBY είναι όμοιες με αυτές του SRS.

Στην ουσία είμαστε να ξαναχρησιμοποιήσουμε την όλη δομή ή πρότυπο του SRS με το να κάνουμε τις πιο κάτω αντικαταστάσεις.

- Αντικαθιστούμε το "TravelPackage" με το "Section"
- Αντικαθιστούμε το "Trip" με το "Section"
- Αντικαθιστούμε το "Client" με το "Student"
- Αντικαθιστούμε το "TripRecord" με το "TranscriptEntry"
- Αντικαθιστούμε το "TravelHistory" με το "Transcript"

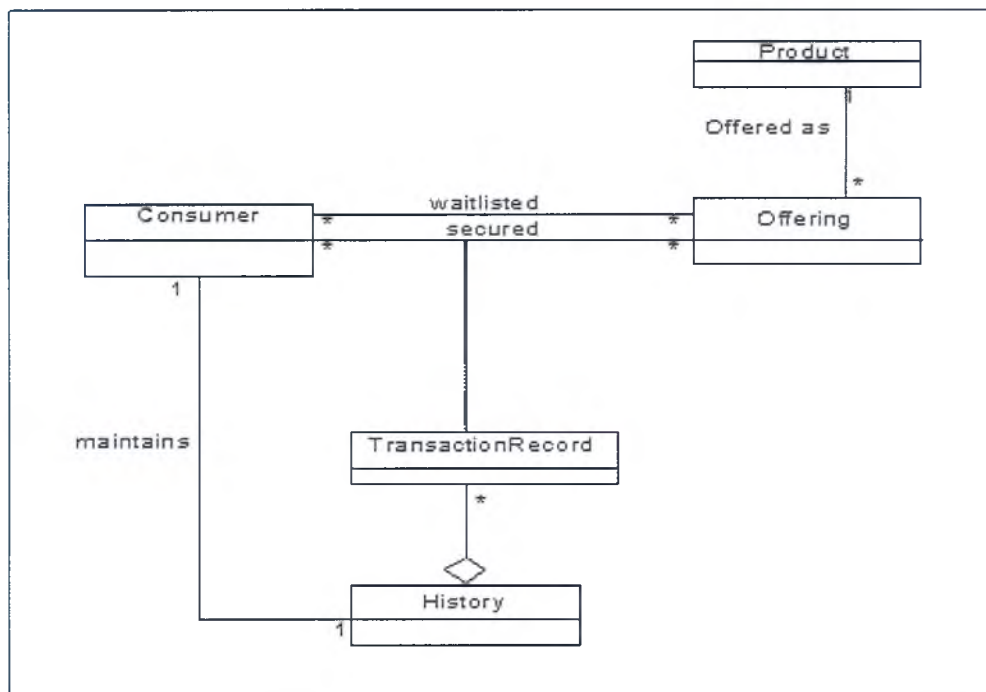
Σημειώστε ότι όλες οι σχέσεις μεταξύ αυτών των κλάσεων- τα ονόματα τους, τύπος τους ακόμα και η πολυπλοκότητα τους, παραμένουν αμετάβλητες από το διάγραμμα κλάσεων του SRS.

"Τέτοιο ακριβές ταίριασμα είναι πάρα πολύ σπάνιο όταν χρησιμοποιούμε πρότυπα σχεδιασμού. Μην φοβάσαι να αλλάξεις κάποια πράγματα(διάγραψε τις κλάσεις ή τις σχέσεις, άλλαξε την πολυπλοκότητα και ούτω καθεξής) για να μπορεί να γίνει εφικτή η επαναχρησιμοποίηση του προτύπου" .



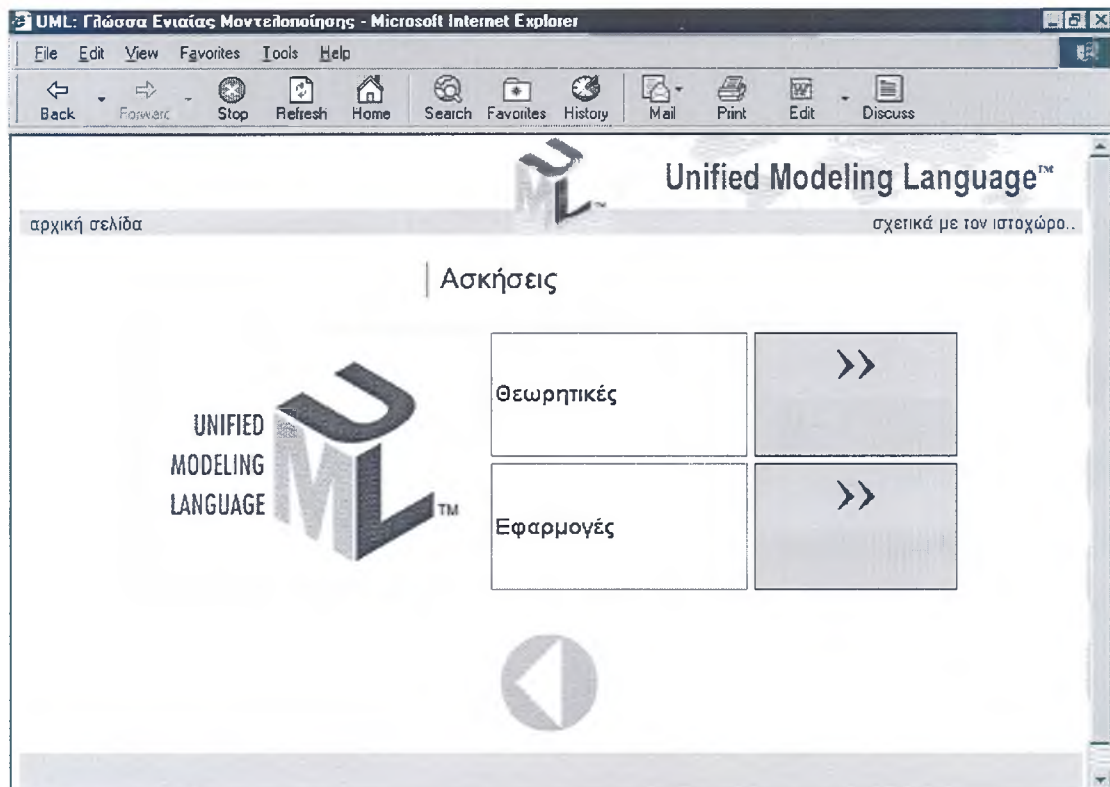
Έχοντας αναγνωρίσει τις ομοιότητες μεταξύ των δύο σχεδιασμών, είμαστε έτοιμοι να εκμεταλλευτούμε για λίγο την επαναχρησιμοποίηση σε σχέση με τον κώδικα των δύο συστημάτων. Στην ουσία, έχοντας αντιληφθεί την ανάγκη ανάπτυξης των δύο αυτών συστημάτων πριν την ανάπτυξη κάποιου από αυτά, θα μπορούσαμε να είχαμε κάποια βασικά βήματα ανάπτυξης ενός γενικού μοτίβου που θα μπορούσε να είχε χρησιμοποιηθεί σαν η βάση και για τα δύο

συστήματα καθώς επίσης και για οποιοδήποτε σύστημα κράτησης θέσεων που θα μας ζητιόταν να μοντελοποιήσουμε.



Πολλά χρήσιμα και επαναχρησιμοποιούμενα μοτίβα έχουν περιγραφεί: «Πριν ξεκινήσουμε για την δημιουργία ενός αντικειμένου και την μελέτη μοντελοποίησης άλλου αντικειμένου αξίζει τον κόπο να εξερευνήσουμε κατά πόσο κάποιο από αυτά θα μπορούσε να είναι κατάλληλο σαν σημείο έναρξης».

Ασκήσεις



Ασκήσεις-Θεωρητικές

1. Τι είναι και τι δεν είναι η UML?
2. Τι είναι ένα σενάριο? Δημιουργείστε ένα σενάριο για την "ανάληψη μετρητών" (withdraw cash) από ένα ΑΤΜ.
3. Τι είναι μια περίπτωση χρήσης? Δώστε παράδειγμα
4. Ποιά η διαφορά ανάμεσα σε μια περίπτωση χρήσης και σε ένα σενάριο?
5. Ποιους ονομάζουμε χειριστές ή αλλιώς ηθοποιούς?
6. Φτιάξτε ένα διάγραμμα περιπτώσεων χρήσης για ένα τραπεζικό ΑΤΜ. α)Ποιοι είναι οι χειριστές? β)Ποιες είναι οι περιπτώσεις χρήσης?
7. Να ορίσετε τους ακόλουθους όρους και να δώσετε αντίστοιχα ένα παράδειγμα: α)Συσχέτιση, β)Πολλαπλότητα, γ)Συσσώρευση, δ)Κληρονομικότητα
8. Έστω μία κλάση Α αποτελεί συσσώρευση μίας κλάσης Β. Τι σημαίνει αυτό?
9. Περιγράψτε τη διαφορά ανάμεσα στη συσσώρευση και την κληρονομικότητα με ένα συγκεκριμένο παράδειγμα.
10. Τι είναι ένα αντικείμενο?

Απαντήσεις Θεωρητικών Ασκήσεων

1. Τι είναι και τι δεν είναι η UML?

Η UML ή αλλιώς Γλώσσα Ενιαίας Μοντελοποίησης(Unified Modeling Language) είναι μια διαγραμματική τεχνική για τον προσδιορισμό, την οπτικοποίηση, την κατασκευή και την τεκμηρίωση των συνιστωσών συστημάτων λογισμικού. Η UML δεν είναι μια γλώσσα προγραμματισμού.

2. Τι είναι ένα σενάριο? Δημιουργείστε ένα σενάριο για την "ανάληψη μετρητών" (withdraw cash) από ένα ΑΤΜ.

Σενάριο είναι μια αφηγηματική περιγραφή μιας συγκεκριμένης κατάστασης που είναι σημαντική για το υπό εξέταση σύστημα. Δίνει μια σταδιακή περιγραφή των γεγονότων καθώς συμβαίνουν και χρησιμοποιεί απλή γλώσσα που προέρχεται από το πεδίο του προβλήματος.

Σενάριο για την "ανάληψη μετρητών" από ένα ΑΤΜ:

- ο Εισαγωγή τραπεζικής κάρτας στο ΑΤΜ.
- ο Πληκτρολόγηση του PIN.
- ο Επιλογή της κατάλληλης υπηρεσίας ανάληψης μετρητών.
- ο Εισαγωγή του επιθυμητού ποσού.
- ο Αφαίρεση της τραπεζικής κάρτας.
- ο Απομάκρυνση των χρημάτων.

3. Τι είναι μια περίπτωση χρήσης? Δώστε παράδειγμα.

Μια περίπτωση χρήσης είναι η γενικότερη κατάσταση που μοντελοποιείται από ένα σενάριο.

Παράδειγμα: "πληρωμή ταμιά", "ανάληψη μετρητών"

4. Ποιά η διαφορά ανάμεσα σε μια περίπτωση χρήσης και σε ένα σενάριο?

Η διαφορά ανάμεσα σε μια περίπτωση χρήσης και σε ένα σενάριο έγκειται στο ότι : Η περίπτωση χρήσης καλύπτει όλα τα ενδεχόμενα που είναι δυνατό να προκύψουν από μια κατάσταση- και τις εξαιρέσεις ενώ το σενάριο είναι μόνο μία συγκεκριμένη ακολουθία γεγονότων.

5. Ποιους ονομάζουμε χειριστές ή αλλιώς ηθοποιούς?

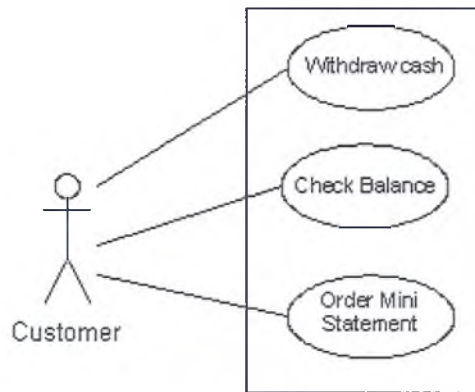
Ένας χειριστής(actor) είναι κάποιος ή κάτι που αλληλεπιδρά με ένα σύστημα. Παριστάνεται με ένα παιδικό σκίτσο ανθρώπου. Οι χειριστές απεικονίζονται

έξω από το σύστημα, αλλά συνδέονται με γραμμές με τις σχετικές περιπτώσεις χρήσης.

6. Φτιάξτε ένα διάγραμμα περιπτώσεων χρήσης για ένα τραπεζικό ΑΤΜ.

α) Ποιοι είναι οι χειριστές? β) Ποιες είναι οι περιπτώσεις χρήσης?

Διάγραμμα περίπτωσης χρήσης για ένα τραπεζικό ΑΤΜ:



α) Υπάρχει μόνο ένας χειριστής, ο "Customer"

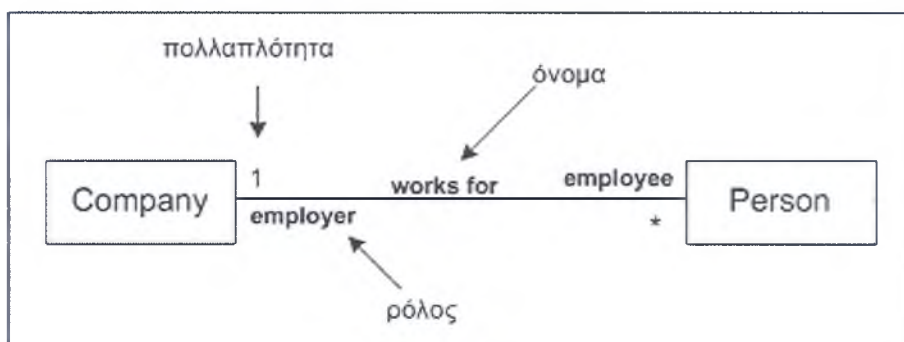
β) Υπάρχουν τρεις περιπτώσεις χρήσεις: "Withdraw cash", "Check Balance", "Order Mini Statement"

7. Να ορίσετε τους ακόλουθους όρους και να δώσετε αντίστοιχα ένα παράδειγμα: α) Συσχέτιση, β) Συσσώρευση, γ) Κληρονομικότητα

α) Συσχετίσεις (associations) είναι δυνατό να υπάρχουν ανάμεσα σε κλάσεις.

Μπορούν να οριστούν ως μια εννοιολογική σύνδεση ή σχέση ανάμεσα σε δύο ή περισσότερες κλάσεις.

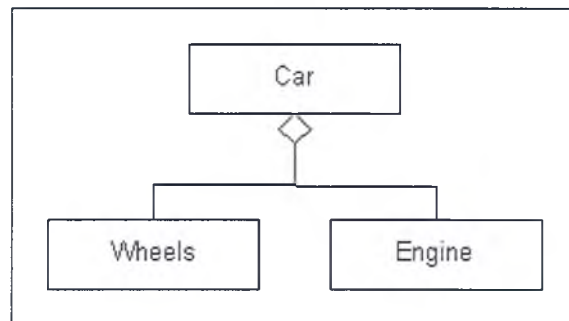
Παράδειγμα:



β) Συσσώρευση (aggregation) είναι μια ιδιαίτερη μορφή συσχέτισης που προσδιορίζει μια σχέση όλου-μέρους (whole-part) ανάμεσα σε μια σύνθετη κλάση (aggregate) και τις συνιστώσες κλάσεις της. Για την απεικόνιση της χρησιμοποιείται το σύμβολο του διαμαντιού. Το διαμάντι είναι προσαρτημένο

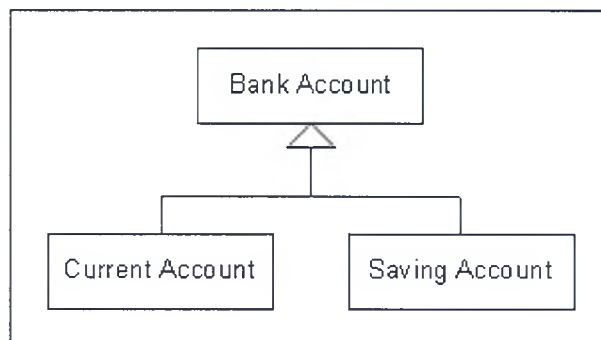
στην κλάση που απεικονίζει την οντότητα που είναι το αποτέλεσμα της συσσώρευσης.

Παράδειγμα:



γ) Κληρονομικότητα είναι η διεργασία μέσω της οποίας ένα αντικείμενο μπορεί να αποκτήσει ("κληρονομήσει", inherit) τις ιδιότητες κάποιου άλλου. Επίσης είναι μια ιδιαίτερη μορφή συσχέτισης, η οποία απεικονίζει και εκφράζει τη σχέση "είναι-ένα-είδος"("is-a-kind-of")

Παράδειγμα:



8. Έστω μία κλάση A αποτελεί συσσώρευση μίας κλάσης B. Τι σημαίνει αυτό?

Αυτό σημαίνει:

- Ένα στιγμιότυπο της κλάσης A θα πρέπει να περιέχει ή να περικλείει ένα στιγμιότυπο της κλάσης B.
- Οι ιδιότητες της κλάσης A θα πρέπει να περιέχουν, είτε ένα αντικείμενο της κλάσης B, είτε μια αναφορά σε ένα αντικείμενο της κλάσης B.

9. Περιγράψτε τη διαφορά ανάμεσα στη συσσώρευση και την κληρονομικότητα με ένα συγκεκριμένο παράδειγμα.

- Ένα διαθέσιμο αυτοκίνητο, ένα φορτηγάκι είναι είδη => αυτοκινήτου. (Κληρονομικότητα)
- Αυτοκίνητο => μηχανή, κιβώτιο ταχυτήτων, ρόδες.(Συσσώρευση)

10. Τι είναι ένα αντικείμενο? Δώστε μερικά παραδείγματα αντικειμένων.

Τα αντικείμενα είναι στοιχεία ή οντότητες που χρησιμοποιούνται για την αναπαράσταση του κόσμου που μας περιβάλλει. Ένα αντικείμενο παριστάνεται με ένα κουτί στο οποίο αναγράφεται το όνομα του αντικειμένου, στη συνέχεια μια άνω κάτω τελεία και τέλος το όνομα της τάξης στην οποία ανήκει το αντικείμενο αν αυτή έχει προσδιοριστεί.

Ένα αντικείμενο μπορεί να είναι:

- Κάτι χειροπιαστό, π.χ. ένα αυτοκίνητο
- Μια διεργασία, π.χ. ο έλεγχος μιας μηχανής(engine test)
- Μια σχέση, π.χ. ένα συμβόλαιο
- Μια θεωρητική έννοια, π.χ. οι μιγαδικοί αριθμοί

Ασκήσεις-Εφαρμογές

1. Σχεδιάστε σε UML ένα τουλάχιστον αντιπροσωπευτικό διάγραμμα κάθε τύπου που να αφορά το πληροφοριακό σύστημα της βιβλιοθήκης του πανεπιστημίου.
2. Απεικονίστε σε διάγραμμα καταστάσεων τον τρόπο λειτουργίας ενός κινητού τηλεφώνου.
3. Σχεδιάστε ένα διάγραμμα δραστηριότητας που να περιγράφει τον τρόπο εκτέλεσης μιας μαγειρικής συνταγής.
4. Σχεδιάστε ένα διάγραμμα κατάστασης για τον τρόπο λειτουργίας ενός πλυντηρίου.

Βιβλιογραφία

Βιβλιογραφικές αναφορές:

- ✦ Terry Quatrani. Visual Modeling with rational rose 2000 and UML. Upper Saddle River, 2000
- ✦ Martin Fowler and Kerdall Scott. Εισαγωγή στη UML(δεύτερη Αμερικάνικη έκδοση). Addison-Wesley Longman, 2000
- ✦ Richard C. Lee and William M. Tepfenhart. UML and C++:A Practical Guide to Object-Oriented Development(second edition). Prentice Hall-Upper Saddle River, 2001
- ✦ Jacquie Barker. Beginning Java objects from Concepts to Code. Wrox-Birmingham, 2000
- ✦ Αλεξανδρος Ν.Χατζηγεωργίου. Αντικειμενοστραφής Τεχνολογία Λογισμικού, Θεσσαλονίκη, Μάρτιος 2003
- ✦ Grady Booch, James Rumbaugh, and Ivar Jacobson. The Unified Modeling Language User Guide. Addison-Wesley, 1999.

Ηλεκτρονικές διευθύνσεις:

- ✦ <http://umlcenter.visual-paradigm.com>
- ✦ http://www.rtems.com/~joel/object_oriented.html
- ✦ <http://www.scs.carleton.ca/~csqs/resources/oop.html>
- ✦ www.rational.com/uml
- ✦ www.planet-sourse-code.com
- ✦ <http://javaboutique.intrnet.com>
- ✦ <http://cgi.di.uom.gr/~ys01>
- ✦ www.cgiscript.net
- ✦ <http://javascript.internet.com>